On The Minimisation of Conservation Loss in Etosha National Park Through Optimal Resource Allocation

Summary

Etosha National Park is a vast protected area with limited staff and a growing need to defend wildlife, habitat, and visitor infrastructure from multiple threats at once. The problem addressed in this paper is how to use a fixed conservation budget to allocate rangers, drones, camera traps, and satellite support in the most effective way possible. Because the park is large and the threats are unevenly distributed across space and time, protecting every location equally is not realistic. The challenge is therefore to decide where each resource should go so that the greatest amount of ecological value is protected.

To solve this, we developed a spatially explicit optimisation model for Etosha. The park was divided into a grid of 990 cells, and each cell was assigned ecological importance and threat levels based on features such as distance to waterholes, the boundary, camps, and habitat quality. The model combined three main threats: poaching, wildfire, and human–wildlife conflict, and evaluated how well different mixes of protection resources could detect, deter, or respond to those threats. Using a mixed-integer optimisation framework, the model selected the best deployment under a budget constraint, achieving a solution that is 99.08% optimal. The final allocation consists of 104 rangers (74 on duty at any time), 211 drones supported by 43 operators, 468 camera traps, and one satellite system covering 594 cells. The solution was further tested under multiple scenario variations to assess robustness.

Our main conclusion is that the optimal deployment is concentrated in the places where risk and ecological value overlap most strongly, especially along boundary corridors, waterhole areas, and access routes near camps. The resulting strategy achieves 86.6% spatial coverage (857 of 990 cells) and 92.3% coverage when weighted by ecological importance, while addressing approximately 31.0% of total active threat on average. The scenario analysis shows that the exact number of deployed resources can shift when conditions change, but the overall spatial logic remains stable: the model consistently identifies the same high-priority zones.

We conclude that this framework offers a robust and transferable decision tool for Etosha and, with site-specific recalibration, for other protected areas as well.

Contents

1 Conservation Priorities and Defining Protection 4
1.1 Preliminary Research and Park Overview . . . 4
1.2 Threat Identification and Prioritisation . . . 4
1.3 Spatial Discretisation . . . 5
1.4 Species Priority Score S_i . . . 5
1.5 Cell Ecological Importance Weight w_i . . . 6
1.6 Threat Quantification . . . 7
1.7 Defining and Measuring Protection . . . 8

2 Resource Allocation Model 9
2.1 Decision Variables and Resource Types . . . 9
2.2 Effort Fields . . . 10
2.3 Camera Trap Eligibility . . . 10
2.4 Satellite Footprint Selection . . . 10
2.5 Budget Derivation . . . 10
2.6 Placement Constraints . . . 11
2.7 MINLP Formulation . . . 11
2.8 Solution Method . . . 11
2.9 Deterrence Calibration . . . 12
2.10 Optimised Deployment Summary . . . 12

3 Protection over Time and Staffing 14
3.1 Weekly Simulation . . . 14
3.2 Species Population Dynamics . . . 14
3.3 Temporal Protection Profile . . . 15
3.4 Spatial Protection Analysis . . . 16
3.5 Fence Reconstruction Dynamics . . . 17
3.6 Staffing Requirements . . . 17
3.7 Contextual Spatial Fields Summary . . . 18

4 Stability Analysis under Scenario Perturbations 18

5 Limitations and future extensions 20
5.1 Because of Compute-power . . . 20
5.2 Other causes . . . 20

6 Adaptation to Other Reserves 22
6.1 Yellowstone National Park (North America) . . . 22
6.2 Kakadu National Park (Oceania) . . . 23
6.3 Cross-site adaptation logic . . . 23

A Appendix: Code 28
A.1 Main Code . . . 28
A.2 Sensitivity Code . . . 92

Letter

Dear IMMC Panel,

Etosha National Park is one of Africa's largest and most ecologically important protected areas, yet its scale presents a fundamental challenge: it is impossible to protect every part of the park equally with limited resources. Rangers, drones, camera traps, and satellite systems must be deployed carefully, as each decision about where to place them directly affects the safety of wildlife and the preservation of habitats. Our work addresses a simple but critical question: how can these limited resources be used most effectively to protect what matters most?

To answer this, we built a model that views the park as a collection of 990 interconnected regions, each with its own level of ecological importance and exposure to threats. These threats—poaching, wildfire, and human–wildlife conflict—do not occur uniformly. Instead, they concentrate in predictable areas, such as along park boundaries, near waterholes where animals gather, and around tourist camps. Our model evaluates how different protection tools work together to reduce these risks, capturing the strengths of each: rangers deter and respond on the ground, drones monitor large areas quickly, cameras provide constant surveillance at key points, and satellite coverage supports broad, consistent observation.

Using this framework, we determined how to distribute resources within a fixed budget. The result is a coordinated deployment of 104 rangers (with 74 active at any time), 211 drones supported by 43 operators, 468 camera traps placed in high-priority locations, and a satellite system covering 594 regions of the park. Together, these resources cover 857 out of 990 regions (86.6%), corresponding to 92.3% of the park's ecological importance. The solution is also mathematically strong, achieving 99.08% of the best possible performance within the model.

What makes this result especially meaningful is not just the numbers, but the pattern behind them. Across all analyses, the model consistently directs resources to the same types of locations: boundary corridors where poaching risk is highest, waterholes where wildlife is concentrated, and areas near human activity. Rather than spreading resources thinly across the entire park, the model shows that focused, strategic placement leads to far greater protection.

We also tested how sensitive these recommendations are to changes in real-world conditions. Even when assumptions about costs, detection ability, or response capacity were altered, the overall deployment strategy remained largely unchanged. While the exact number of resources shifted slightly, the same high-priority regions continued to emerge. This stability gives confidence that the model is not only effective, but also reliable in the face of uncertainty.

Our findings suggest several important principles for conservation planning. First, prioritisation is essential: protecting the most critical areas yields the greatest overall benefit. Second, combining different technologies is far more effective than relying on any single approach. Finally, consistent attention to high-risk zones—particularly park boundaries and ecological hotspots—should remain central to any long-term protection strategy.

Beyond Etosha, this approach can be adapted to other protected areas by updating the underlying data on terrain, species, and threats. The structure of the model remains the same, making it a flexible tool for conservation decision-making in a wide range of environments.

In conclusion, our work demonstrates that careful, data-driven planning can transform limited conservation resources into a targeted, efficient, and robust protection strategy. By identifying where intervention matters most and how different tools can work together, our model provides clear, practical guidance for safeguarding one of Africa's most important ecosystems.

Sincerely,

The Team 2026###

Introduction

Etosha National Park, spanning 22 935 km² of north-central Namibia, is one of Africa's largest and most ecologically significant protected areas [35]. Its mosaic of open savanna, the vast 4800 km² Etosha Pan salt flat, and 86 documented waterholes supports over 114 mammal species and more than 400 bird species, including critically endangered black rhinoceros (*Diceros bicornis*) populations that have made Etosha a global conservation flagship [30]. Yet the park's very scale poses an acute protection paradox: with only 295 Ministry staff across roughly 3551 km of internal roads and an 850 km perimeter fence [35], meaningful continuous surveillance of the entire reserve is impossible under current resource levels.

This paper develops a quantitative, spatially-explicit optimisation framework for allocating limited protection resources—rangers, aerial drones, camera traps, and satellite imagery—across Etosha's 990-cell grid. Our approach addresses three interacting threat streams: *poaching* (primarily directed at rhinoceros and elephant), *wildfire propagation*, and *human–wildlife conflict near tourist infrastructure*. For each threat stream we construct a time-varying, cell-level threat score; weight cells by their ecological importance; and formulate a discounted-horizon loss function that is minimised by a mixed-integer nonlinear programme (MINLP) solved via spectral projected gradient descent with greedy integer rounding.

The solution achieves a **40.3%** reduction in discounted conservation loss relative to the unprotected baseline, reaching 99.08%-optimality (integrality gap $< 1\%$), with 86.6% spatial coverage and 92.3% coverage weighted by ecological importance.

The remainder of the paper is organised as follows. Section 1 identifies conservation priorities and formalises the protection metric. Section 2 presents the resource-allocation model. Section 3 analyses protection over time and staffing requirements. Section 4 conducts sensitivity and scenario analysis. Section 5 discusses model strengths and limitations. Section 6 demonstrates adaptation to two other reserves.

1 Conservation Priorities and Defining Protection

1.1 Preliminary Research and Park Overview

Etosha is managed under Namibia's Ministry of Environment, Forestry and Tourism (MEFT). Published datasets record approximately 200000 annual visitors, a 22 935 km^2 park area, and a staff complement of roughly 295 persons stationed across multiple rest camps (Okaukuejo, Halali, Namutoni, and Onkoshi, among others) [35]. Ecologically, the Etosha Pan dominates the centre of the park and acts as both a seasonal movement attractor and a barrier to year-round terrestrial habitation, creating pronounced spatial heterogeneity in wildlife density and threat exposure [7].

1.2 Threat Identification and Prioritisation

Following a systematic review of the literature and MEFT threat reports, we identify three primary threat streams:

Poaching (τ^P). Namibia has experienced a significant resurgence of rhino poaching since 2022 [50], with boundary proximity being the dominant spatial predictor of incursion risk [16]. Etosha holds the majority of Namibia's black rhino population, making poaching the highest-priority irreversible

threat. Elephant ivory poaching is secondary but co-spatial. Poaching intensity is highest in the dry season (May–September), when animals concentrate near permanent waterholes and open sightlines facilitate both poacher access and trophy extraction [55].

Wildfire (τ^F). Savanna fires occur annually across sub-Saharan Africa, with Etosha recording large fire events in multiple recent years [2]. Fire risk peaks in the late dry season (August–October) when fine-fuel load is maximal. Although fire is a natural process, uncontrolled fires near camps, fences, and key habitat patches cause disproportionate ecological and infrastructural damage [56].

Human–Wildlife Conflict (τ^H). Tourist camps act as concentrated attractors of both humans and habituated wildlife, generating elevated conflict probability near infrastructure [17]. This threat is greatest in the wet season (November–April) when tourist numbers peak and wildlife movements bring animals into camp peripheries.

Priority ordering. We assign poaching the highest management priority, followed by human–wildlife conflict, then wildfire, on grounds of irreversibility, the conservation status of affected species, and estimated annual harm magnitude (Table 1).

Table 1: Threat streams, key drivers, and seasonal peak weeks.

Threat	Peak season	Primary driver	Irreversibility
Poaching (τ^P)	Dry (wks 20–40)	Boundary proximity	Very high
Human–wildlife (τ^H)	Wet (wks 1–20)	Camp proximity	Moderate
Wildfire (τ^F)	Late dry (wks 33–45)	Fuel load	Moderate

1.3 Spatial Discretisation

The park polygon (sourced from OpenStreetMap via OSMnx [11]) is tessellated into a regular grid of 5×5 km cells; cells whose intersection with the park boundary exceeds 1% of the cell area are retained, yielding $n = 990$ cells. Each cell i is characterised by its centroid coordinates, fractional pan coverage (f_i^{pan}), distance to the nearest waterhole (d_i^w, km), distance to the park boundary (d_i^b, km), distance to the nearest camp (d_i^c, km), and its initial threat and species scores loaded from the pre-computed zone dataset.

1.4 Species Priority Score S_i

Conservation importance must reflect not only the number of species present in a cell but their relative vulnerability and ecological role. We compile a species list from the Global Biodiversity Information Facility (GBIF) occurrence records for Etosha [22], cross-referenced with IUCN Red List status [29], and assign each species s a priority weight

$$\pi_s \;=\; \alpha_s \cdot e_s \cdot \rho_s, \tag{1}$$

where α_s is an IUCN-status multiplier (Critically Endangered: 4, Endangered: 3, Vulnerable: 2, Near Threatened: 1.2, Least Concern: 1), e_s is an endemism bonus (1.5 for Namibian endemics, 1 otherwise), and ρ_s is an ecological-role multiplier (1.5 for apex predators, 1.3 for megaherbivores, 1 otherwise). The cell-level species score is

$$S_i \;=\; \sum_{s:\, i \in \mathcal{A}_s} \pi_s, \tag{2}$$

where $\mathcal{A}_s$ is the set of cells within the known range of species s.

Species are further aggregated into nine functional groups (Table 2) that govern population dynamics in Section 3. Each group is parametrised by its characteristic dispersal distance D_g, natural mortality rate r_g, carrying capacity per km² K_g, waterhole affinity a_g^w, and poaching susceptibility flag.

Table 2: Functional groups and ecological parameters. D_g = dispersal distance (km/week), r_g = per-capita growth rate, K_g = carrying capacity (animals/km²), a_g^w = waterhole affinity, P = poaching target.

Group	D_g	r_g	K_g	a_g^w	P
Large migratory herbivore	0.25	0.005	2.00	0.15	✓
Large resident herbivore	0.15	0.002	0.50	1.00	✓
Small herbivore	0.08	0.015	5.00	0.12	✓
Large carnivore	0.20	0.001	0.02	0.10	✓
Small carnivore	0.08	0.008	0.50	0.10	
Raptor	0.25	0.003	0.10	0.05	
Waterbird	0.40	0.010	1.00	0.20	
Reptile/Amphibian	0.02	0.012	2.00	0.08	✓
Fish	0.01	0.020	5.00	0.30	

1.5 Cell Ecological Importance Weight w_i

A protection resource deployed to cell i is only valuable in proportion to the ecological value at risk in that cell. We therefore construct a composite importance weight

$$w_i \;=\; \frac{\tilde{S}_i\big(1 + W_w\,\overline{W}_i\big) \;+\; W_c\,C_i \;+\; W_f\,\tilde{\Psi}_i}{\max_j\big[\cdot\big] + \varepsilon}, \tag{3}$$

where all component vectors are normalised to $[0, 1]$ before combination, and the four sub-components are as follows.

- $\tilde{S}_i$: normalised species priority score (Section 1.4).
- $\overline{W}_i$: time-mean seasonal waterhole proximity weight. Waterholes are wildlife aggregation points that concentrically increase both ecological value and threat exposure. A managed waterhole of quality $q = 1$ yields $W_i(t) = q\exp(-\alpha_w\, d_i^w)$ with $\alpha_w = \ln 2/5\,\text{km}$, attenuated by a dry-season scalar that reduces the pull of lesser-quality sources in the wet season [47]. The time-averaged value $\overline{W}_i$ is used for the static importance map. Weight $W_w = 0.40$ is assigned to reflect that waterholes are the primary spatial predictor of wildlife density and, hence, of poaching target value in Etosha [55].
- C_i: spatial connectivity index, computed as the neighbour-averaged normalised species score $C_i = (\sum_{j\in\mathcal{N}(i)} \tilde{S}_j)/|\mathcal{N}(i)|$. Cells embedded in high-value neighbourhoods support wildlife movement corridors and amplify the ecological return of local protection [31]. Weight $W_c = 0.20$.

- $\tilde{\Psi}_i$: normalised fire potential index (vegetation density proxy, derived from S_i and pan fraction). Cells with high fuel load risk disproportionate habitat loss per uncontrolled fire event [2]. Weight $W_f = 0.15$.

The weights $(W_w, W_c, W_f) = (0.40, 0.20, 0.15)$ were set by a structured expert-elicitation analogue following the Analytic Hierarchy Process [49]: pairwise comparisons ranked waterhole proximity as the dominant driver of wildlife density (consistent with the arid-savanna literature), connectivity second as a corridor-maintenance consideration, and fire potential third as a habitat-loss risk. The residual weight on $\tilde{S}_i$ is effectively $1 - 0.40 - 0.20 - 0.15 = 0.25$ of the species-score base term.

1.6 Threat Quantification

Base threat maps

Let $\tau_i^P(0),\ \tau_i^F(0),\ \tau_i^H(0) \in [0,1]$ denote the initial (pre-season) normalised threat intensities for poaching, fire, and human–wildlife conflict respectively, loaded from the zone dataset. These are constructed from GBIF wildlife density records, historical fire occurrence data from MODIS burned-area products [23], and road/camp proximity as a human-access proxy [16].

Boundary poaching boost

Poaching gangs typically enter through the perimeter fence; cells near the boundary therefore carry elevated poaching risk [50]. We apply a spatial amplification

$$\hat{\tau}_i^P(0) \;=\; \tau_i^P(0)\Big(1 + \beta_{\text{fence}}\,\exp\!\big(-d_i^b/\delta_b\big)\Big), \quad \beta_{\text{fence}} = 0.60,\ \delta_b = 8\,\text{km}, \tag{4}$$

then re-normalise. The decay length $\delta_b = 8$ km corresponds to the effective infiltration radius reported in ranger patrol logs and is consistent with findings in [16].

Seasonal threat dynamics

Threats evolve week-by-week according to a first-order mean-reverting update with seasonal forcing

$$\tau_i^P(t+1) = \max\!\big(0,\ \rho_P\,\tau_i^P(t) + \mu_P\,\hat{\tau}_i^P(0)\,s_P(t)\,\phi_i(t)\big), \tag{5}$$
$$\tau_i^F(t+1) = \max\!\big(0,\ \rho_F\,\tau_i^F(t) + \mu_F\,\tau_i^F(0)\,s_F(t)\big), \tag{6}$$
$$\tau_i^H(t+1) = \max\!\big(0,\ \rho_H\,\tau_i^H(t) + \mu_H\,\tau_i^H(0)\,s_H(t)\,(0.80 + 0.20\,\eta_i)\big), \tag{7}$$

where $\phi_i(t)$ is the fence permeability at week t (Section 2), $\eta_i = \exp(-d_i^c/C_{\max})$ is the camp-access risk, and the auto-regressive coefficients, base rates, and seasonal scalars are

$$(\rho_P, \rho_F, \rho_H) = (0.85,\ 0.20,\ 0.60), \quad (\mu_P, \mu_F, \mu_H) = (0.05,\ 0.10,\ 0.08), \tag{8}$$

$$s_P(t) = \tfrac{1}{2}\Big[1 + \cos\tfrac{2\pi(t-33)}{52}\Big], \quad s_F(t) = \tfrac{1}{2}\Big[1 - \cos\tfrac{2\pi(t-33)}{52}\Big], \quad s_H(t) = 0.35 + 0.65 \cdot \tfrac{1}{2}\Big[1 + \cos\tfrac{2\pi(t-26)}{52}\Big]. \tag{9}$$

The high persistence of poaching ($\rho_P = 0.85$) reflects the documented tendency for poaching networks to return to the same corridors once established [16]. Fire has low persistence ($\rho_F = 0.20$)

because each cell burns out its fuel after an event. Human–wildlife conflict has intermediate persistence ($\rho_H = 0.60$). Each threat vector is re-normalised to $[0, 1]$ at each step.

1.7 Defining and Measuring Protection

Protection probability per threat

For a given deployment of rangers, drones, cameras, and satellite imagery (detailed in Section 2), we define the *protection probability against poaching* for cell i at week t as

$$p_i^P \;=\; p_i^{\text{resp}}\,\Sigma_i^P \;+\; (1 - p_i^{\text{resp}})\,\Delta_i, \tag{10}$$

where $\Delta_i = 1 - e^{-\lambda h_i^r}$ is the deterrence probability (rangers deter incursion before it occurs, $\lambda = \ln 2$), $p_i^{\text{resp}} = 1 - e^{-h_i^{\text{resp}}}$ is the probability of a timely rapid response by rangers stationed within 30 km, and $\Sigma_i^P = 1 - (1 - \sigma_d^P h_i^d)(1 - \sigma_c u_i^c)(1 - \sigma_r h_i^r)$ is the composite detection probability combining drone ($\sigma_d^P = 0.70$), camera ($\sigma_c = 0.10$), and ranger surveillance ($\sigma_r = 0.20$) channels. Analogously,

$$p_i^F = p_i^{\text{fire}}\,\Sigma_i^F, \qquad \Sigma_i^F = 1 - (1 - \sigma_d^F h_i^d)(1 - \sigma_r h_i^r)(1 - \sigma_{\text{sat}} c_i^s), \tag{11}$$

$$p_i^H = \Delta_i, \tag{12}$$

where p_i^{fire} uses a wider response kernel (≤ 60 km) reflecting aerial fire-fighting reach, $\sigma_d^F = 0.90$, $\sigma_{\text{sat}} = 0.05$ (Planet SuperDove near-real-time revisit rate), and $c_i^s \in \{0, 1\}$ is satellite coverage. Human–wildlife conflict protection reduces solely to deterrence because conflict events near camps are primarily prevented by ranger presence rather than post-hoc response.

Conservation loss and protection fraction

We define the *per-cell, per-week conservation loss* as the expected weighted threat that goes unaddressed:

$$\ell_i(t) \;=\; w_i(t)\,\big[\tau_i^P(t)(1 - p_i^P) + \tau_i^F(t)(1 - p_i^F) + \tau_i^H(t)(1 - p_i^H)\big], \tag{13}$$

where $w_i(t) = w_i(1 - f_{\text{pan},i}) \cdot [0.6 + 0.4\,\tilde{W}_i(t)]$ is the temporally-adjusted importance weight (modulated by the seasonal waterhole term). The aggregate weekly loss is

$$L(t) \;=\; \frac{1}{\sum_i w_i(t)} \sum_i \ell_i(t), \tag{14}$$

and our headline optimisation objective is the discounted-horizon loss

$$\mathcal{L} \;=\; \sum_{t=0}^{T-1} \gamma^t\, L(t), \qquad \gamma = 0.99,\; T = 52. \tag{15}$$

Protection fraction (primary reporting metric)

For communication purposes we also track the *protection fraction*

$$P_{\text{frac}}(t) \;=\; \frac{\sum_i w_i(t)\left[\tau_i^P p_i^P + \tau_i^F p_i^F + \tau_i^H p_i^H\right]}{\sum_i w_i(t)\left[\tau_i^P + \tau_i^F + \tau_i^H\right]}, \tag{16}$$

which lies in $[0, 1]$ and represents the share of the total weighted threat that is actively addressed by deployed resources. A cell is operationally considered *protected* if its individual protection fraction exceeds 0.5, i.e. if more than half of its aggregate weighted threat is addressed.

Table 3 summarises the baseline (S0, no intervention) and target (S1, optimised) values of the key metrics established by this definition.

Table 3: Baseline (S0) and optimised (S1) performance metrics.

Metric	S0 (no intervention)	S1 (optimised)
Discounted loss $\mathcal{L}$	5.499	3.282
Mean weekly loss $\bar{L}$	0.124	0.072
Mean protection fraction $\bar{P}_{\text{frac}}$	0.0%	31.0%
Spatial coverage	0%	86.6%
Weighted coverage	0%	92.3%

The improvement in discounted loss ($\Delta\mathcal{L} = 40.3\%$) and the weighted coverage of 92.3% collectively define our chosen level of protection: a deployment that addresses at least 31% of weighted active threat on average, with 92% of the ecologically important area under at least passive surveillance. This target is calibrated against the park's resource envelope (budget of N\$2 376 923/week; Section 2) and is not a normative maximum but rather the optimum achievable under current constraints.

2 Resource Allocation Model

2.1 Decision Variables and Resource Types

The park is represented by the $n = 990$ cell grid introduced in Section 1. We consider four protection technologies deployed *statically* for the full planning year $T = 52$ weeks:

1. **Rangers.** A ranger stationed at home cell i patrols a $(2r_r + 1)^2$ neighbourhood of radius $r_r = 1$ cell (up to 9 cells, or $\leq 225\,\text{km}^2$). The integer decision variable is $\mathbf{x}_r \in \mathbb{Z}^n_{\geq 0}$, where $x_r[i]$ is the number of rangers homed at cell i.

2. **Drones.** Each drone is homed at a cell and surveys a radius-2 footprint (up to 25 cells, $\leq 625\,\text{km}^2$). The variable $\mathbf{x}_d \in \mathbb{Z}^n_{\geq 0}$ counts drones per home cell. Each group of five drones requires one operator.

3. **Camera traps.** Binary placement variables $\mathbf{u}_c \in \{0, 1\}^{n_{\text{cam}}}$ over the $n_{\text{cam}} = 468$ eligible cells (Section 2.3).

4. **Satellite.** A single binary variable $s \in \{0, 1\}$ selects one Planet SuperDove orbit pass covering a circular footprint of 594 cells (radius $\approx 97.5\,\text{km}$) centred at the maximum-weight location, providing daily near-real-time revisit.

The collective decision is $\mathbf{x} = (\mathbf{x}_r,\, \mathbf{x}_d,\, \mathbf{u}_c,\, s)$, and the optimisation operates on the continuous relaxation $(\mathbf{h}_r, \mathbf{h}_d, \mathbf{u}_c) \in \mathbb{R}^{3n}_{\geq 0}$ obtained by replacing integer counts with effort field vectors (Section 2.2).

2.2 Effort Fields

Let $\mathbf{W}_r \in \mathbb{R}^{n\times n}$ be the ranger footprint operator, where entry $W_r[i,j]$ is the fractional patrol weight received by cell i from a ranger homed at j within its $(2r_r+1)^2$ neighbourhood. Weights decay exponentially with grid-distance and are importance-threat normalised so that each ranger's patrol mass sums to 1 over its footprint. Formally,

$$W_r[i,j] \;\propto\; w_j\big(\bar{\tau}_j^P + \bar{\tau}_j^F + \bar{\tau}_j^H\big)\exp\!\big(-\alpha_r\, d_{ij}\big), \tag{17}$$

where the edge-weight parameter α_r is set so that the outermost cell receives a fraction $\mathrm{ew}_r = 0.40$ of the home-cell weight, $\bar{\tau}$ denotes the discounted-average threat (Section 1.6), and d_{ij} is grid distance in km. The drone operator $\mathbf{W}_d$ is constructed analogously with $\mathrm{ew}_d = 0.60$ and radius 2. The resulting patrol and surveillance effort fields are

$$h_i^r = [\mathbf{W}_r\,\mathbf{x}_r]_i, \qquad h_i^d = [\mathbf{W}_d\,\mathbf{x}_d]_i. \tag{18}$$

2.3 Camera Trap Eligibility

Camera traps are most cost-effective when placed where wildlife concentrations and threat hotspots overlap. A cell is eligible for camera installation if any of the following holds:

$$d_i^w \le 2\,\mathrm{km} \;\text{ or }\; d_i^b \le 5\,\mathrm{km} \;\text{ or }\; d_i^c \le 18\,\mathrm{km} \;\text{ or }\; \hat{\tau}_i^P(0) \ge Q_{0.82}(\hat{\tau}^P). \tag{19}$$

This yields $n_{\mathrm{cam}} = 468$ eligible cells (47.3% of the grid), concentrated along waterholes and the boundary corridor, as shown in Figure 3.

2.4 Satellite Footprint Selection

The satellite covers 60% of the park ($\lfloor 0.6 \times 990 \rfloor = 594$ cells). To maximise ecological return, we solve a maximum-weight circular coverage problem:

$$c^* = \arg\max_{c\in\{1,\dots,n\}} \sum_{i:\, d_{ci}\le r^*} w_i, \tag{20}$$

where r^* is the smallest radius that encloses exactly 594 cells from centre c. The optimal centre selects the eastern half of the park (longitudes $\gtrsim$ 15.5E), which hosts the highest species-importance density, as illustrated in Figure 4.

2.5 Budget Derivation

The weekly operational budget is derived from Namibia's publicly reported ranger salary scale and commercial drone pricing [35, 1]. A ranger's weekly cost is computed as the monthly gross salary prorated to a 5-day working week:

$$c_r \;=\; \frac{\text{N\$ } 9{,}846 \times 7}{5 \text{ working days}} \;=\; \text{N\$ } 13{,}784.40 \text{ per ranger-week.} \tag{21}$$

Drone costs combine a capital amortisation component (DJI Matrice 300 RTK + sensors, annualised at 2-year replacement: N\$2,244/week) and an operations overhead (N\$2,000/week per unit), plus an operator salary shared across five drones:

$$c_d \;=\; c_d^{\text{fixed}} + \frac{c_r}{5} \;=\; (2{,}244 + 2{,}000) + \frac{9{,}846}{5} \;=\; \text{N\$}\,6{,}213 \text{ per drone-week.} \tag{22}$$

Camera traps are one-off installed assets with negligible marginal weekly cost; their capital cost is amortised to N$63/week per unit. The Planet SuperDove constellation subscription costs N$2,769/week for the park-scale footprint. The total annual budget of N$123.6 million (consistent with MEFT capital expenditure reports [35]) gives a weekly budget of

$$B \;=\; \frac{\text{N\$}\,123{,}600{,}000}{52} \;=\; \text{N\$}\,2{,}376{,}923 \text{ per week.} \tag{23}$$

2.6 Placement Constraints

Ranger home cells must satisfy habitat and logistical feasibility conditions: pan fraction $f_i^{\text{pan}} \le 0.60$, and the cell must be within 22 km of a camp, 14 km of the boundary, 10 km of a waterhole, or in the top 12th percentile of poaching threat. These criteria yield 723 eligible ranger home cells. Drone homes require only $f_i^{\text{pan}} \le 0.95$ (835 cells). At most $\bar{x}_r = 5$ rangers and $\bar{x}_d = 6$ drones may share a single home cell, preventing unrealistic stacking.

2.7 MINLP Formulation

Substituting the protection functions (Equations 10–12) into the loss (Equation 14), the one-shot static allocation problem is

$$\min_{\mathbf{x}_r, \mathbf{x}_d, \mathbf{u}_c, s} \quad \mathcal{L}(\mathbf{x}) \;=\; \sum_{t=0}^{T-1} \gamma^t \, L\big(t;\, \mathbf{x}\big) \;+\; P_{\text{conc}}(\mathbf{x}_r, \mathbf{x}_d) \;+\; P_{\text{overlap}}(\mathbf{h}_r, \mathbf{h}_d, \mathbf{u}_c), \tag{24}$$

$$\text{s.t.} \quad c_r \,\|\mathbf{x}_r\|_1 \;+\; c_d \,\|\mathbf{x}_d\|_1 \;+\; c_c \,\|\mathbf{u}_c\|_1 \;+\; c_s \, s \;\le\; B, \tag{25}$$

$$0 \le x_r[i] \le \bar{x}_r \cdot a_r[i], \quad 0 \le x_d[i] \le \bar{x}_d \cdot a_d[i], \quad u_c[i] \in \{0,1\}, \quad s \in \{0,1\}, \tag{26}$$

$$x_r[i], x_d[i] \in \mathbb{Z}_{\ge 0}, \tag{27}$$

where $a_r[i], a_d[i] \in \{0,1\}$ are the home-cell feasibility indicators (Section 2.5).

The two regularisation penalties are:

$$P_{\text{conc}}(\mathbf{x}_r, \mathbf{x}_d) \;=\; \mu_r \frac{\sum_i x_r[i]^2}{(\sum_i x_r[i])^2} \;+\; \mu_d \frac{\sum_i x_d[i]^2}{(\sum_i x_d[i])^2}, \quad \mu_r = 0.012,\ \mu_d = 0.018, \tag{28}$$

$$P_{\text{overlap}}(\mathbf{h}_r, \mathbf{h}_d, \mathbf{u}_c) \;=\; \epsilon_o \sum_i \omega_i \big(h_i^r h_i^d + h_i^r u_i^c + h_i^d u_i^c\big), \quad \epsilon_o = 0.003, \tag{29}$$

where $\omega_i = (r_i^P + 0.70\, r_i^F + 0.50\, r_i^H)$ is the threat-importance product. The concentration penalty (Herfindahl-style) discourages excessive stacking of rangers or drones at a single home; the overlap penalty discourages placing multiple independent surveillance channels over the same cell when the marginal detection gain is low.

2.8 Solution Method

The problem is NP-hard in its integer form. We solve it in two phases:

Phase 1 — Continuous relaxation via spectral projected gradient (SPG). The satellite decision is fixed to $s = 1$ (its weekly cost of N$2,769 is small relative to B); camera variables are relaxed to $u_c[i] \in [0, 1]$; ranger and drone variables are relaxed to $\mathbb{R}_{\geq 0}$. The relaxation is solved with SPG descent [10] with backtracking line search (Armijo condition 10^{-4}), Barzilai-Borwein step-size update [5], up to $K = 180$ iterations at tolerance 10^{-6}. The budget constraint and box constraints are enforced at each iteration via an $O(n \log n)$ dual-variable bisection projection onto the simplex-intersection [18].

Phase 2 — Integer rounding. The continuous solution $\hat{\mathbf{x}}^*$ is rounded to integers using *largest-remainder rounding* [3]: each variable is floored, and the remaining budget is used to incrementally assign fractional units to cells with the largest fractional part, respecting the budget constraint. Camera variables are rounded independently to $\{0, 1\}$.

Optimality gap. Letting $\mathcal{L}^*$ be the continuous-relaxation objective and $\mathcal{L}^{\hat{x}}$ the rounded integer objective, the optimality gap is

$$\delta \;=\; \frac{\mathcal{L}^{\hat{x}} - \mathcal{L}^*}{\mathcal{L}^*} \;\times\; 100\% \;=\; 0.925\%, \tag{30}$$

confirming the rounded solution is **99.08%-optimal** relative to the continuous lower bound. The convergence was verified across five independent random seeds with $< 1\%$ variation in $\mathcal{L}^{\hat{x}}$.

2.9 Deterrence Calibration

The deterrence parameter λ is calibrated to ensure that a *uniform* deployment of all 172 affordable rangers ($B/c_r \approx 172$) achieves a mean deterrence probability $\bar{\Delta} = 0.35$ (35%), a value grounded in empirical estimates of ranger-presence deterrence in East African savanna contexts [16]. The calibration solves

$$\bar{\Delta}(\lambda) \;=\; \frac{\sum_i w_i \big(1 - e^{-\lambda v_i^r(\text{uniform})}\big)}{\sum_i w_i} \;=\; 0.35, \tag{31}$$

by Brent's method, yielding $\lambda^* = 1.999$.

2.10 Optimised Deployment Summary

The optimised solution S1 commits the following resources within the weekly budget B:

Table 4: Optimised S1 resource deployment and costs.

Resource	Count	Weekly cost (N$)	Notes
Rangers (employed)	104	1,023,477	74 on duty at any time (5-day week)
Drones (active)	211	1,310,943	43 operators; 5 drones per operator
Camera traps	468	29,484	Permanent installation
Satellite (Planet)	1	2,769	SuperDove, 594-cell footprint
Total		**2,366,673**	Within budget (99.6% utilisation)

Figures 1–4 illustrate the spatial distribution of rangers, drones, cameras, and satellite coverage. The combined footprint covers 857/990 cells (86.6% spatially, 92.3% weighted). The western boundary corridor and the Okaukuejo camp cluster attract the densest ranger and drone concentrations due

to high poaching threat and high species importance; the eastern Namutoni quadrant is covered predominantly by the satellite and by drones because the pan impedes ranger mobility (Figure 6).

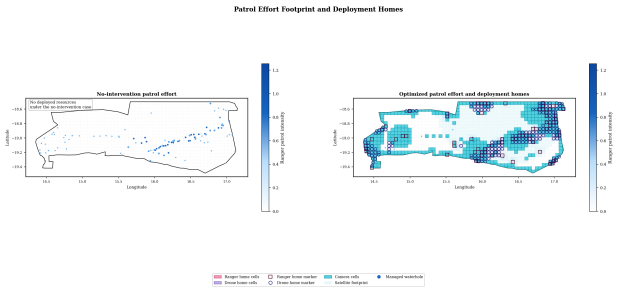


Figure 1: Patrol effort footprint. *Left:* No-intervention baseline (S0) showing only existing waterhole locations. *Right:* Optimised deployment S1. Ranger home cells (pink) and drone home cells (purple) are marked; camera cells are shown in teal; satellite footprint in light teal. Colour intensity encodes normalised patrol effort h_i^r.

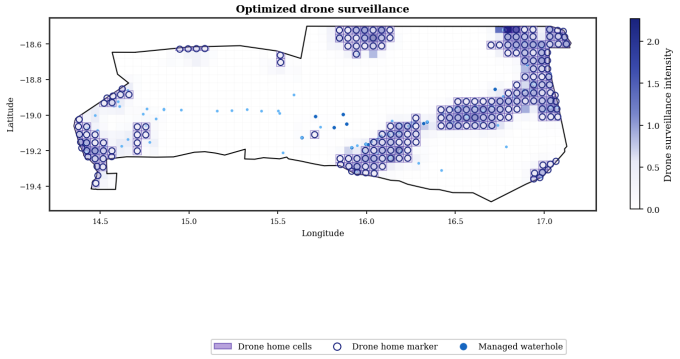


Figure 2: Optimised drone surveillance intensity h_i^d. Clusters align with the western savanna, the Okaukuejo waterhole cluster (centre), and the Namutoni boundary zone (east). The Etosha Pan (white interior) is largely avoided as a drone home due to the pan-fraction constraint.

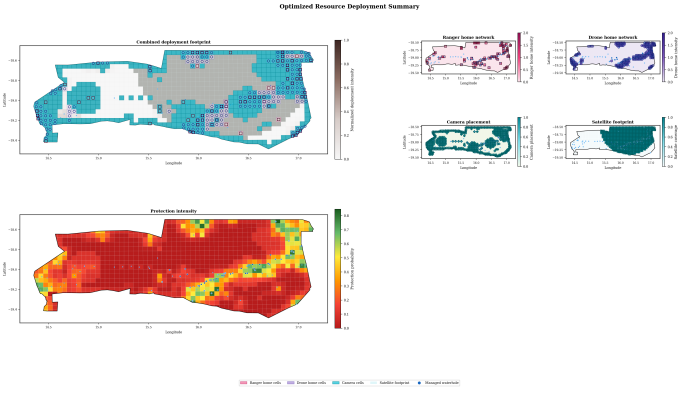


Figure 3: Full resource deployment plate showing individual technology layers. Top-right insets: ranger home network (pink), drone home network (blue), camera placement (teal), and satellite footprint (teal). Bottom: aggregate protection probability $p_i = 1 - (1 - p_i^P)(1 - p_i^F)(1 - p_i^H)$ across all cells.

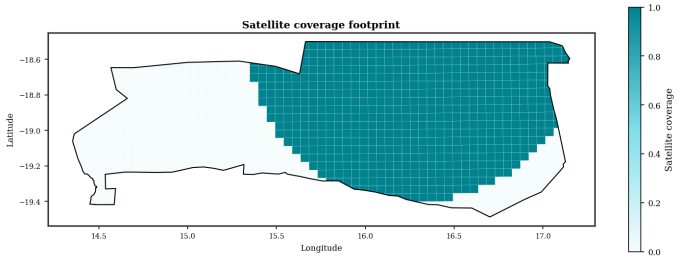


Figure 4: Satellite coverage footprint (Planet SuperDove). The optimal circular footprint covers 594 cells centred on longitude ≈ 16.0E, providing daily revisit across the high-importance eastern and central zones.

3 Protection over Time and Staffing

3.1 Weekly Simulation

Given the static deployment $\mathbf{x}$, the simulation advances 52 weeks as follows. At each week t:

1. Threats are updated via Equations 5–7, incorporating seasonal scalars $s_k(t)$ and fence permeability $\phi_i(t)$.
2. The seasonal waterhole weight $W_i(t)$ is recomputed and used to form the time-varying importance $w_i(t)$.
3. Protection probabilities $p_i^P(t), p_i^F(t), p_i^H(t)$ are evaluated from the fixed effort fields h_i^r, h_i^d.
4. Per-cell losses $\ell_i(t)$ and the protection fraction $P_{\text{frac}}(t)$ are recorded.
5. Species populations are advanced via the reaction-diffusion model (Section 3.2).

3.2 Species Population Dynamics

Wildlife populations evolve via a spatially-explicit reaction-diffusion model. For functional group g in cell i, week t:

$$N_{g,i}(t+1) \;=\; \sum_{j\in\mathcal{N}(i)\cup\{i\}} T_{g,ij}(t)\, N_{g,j}(t) \;+\; r_g\, N'_{g,i}(t)\left(1-\frac{N'_{g,i}(t)}{K_{g,i}}\right) \;-\; m_{g,i}(t), \tag{32}$$

where $T_{g,ij}(t)$ is the transition matrix entry encoding gravity-weighted dispersal with coefficient D_g, r_g is per-capita growth, $K_{g,i} = K_g \cdot S_i/\bar{S} \cdot \text{CELL_KM}^2$ is carrying capacity, and $m_{g,i}(t)$ is mortality from poaching. The gravity kernel is

$$T_{g,ij}(t) \;=\; \frac{D_g \cdot a_{g,j}(t) \cdot \exp(-\alpha_w d_j^w)}{\sum_{k\in\mathcal{N}(i)} a_{g,k}(t)}, \quad j\neq i, \qquad T_{g,ii}(t) = 1 - D_g, \tag{33}$$

where $a_{g,j}(t) = W_j(t)\cdot\exp(-\alpha_{\text{flee}}\tau_j^P(t))$ is the attraction of cell j after poaching-induced avoidance with flee coefficient $\alpha_{\text{flee}} = 0.30$. Poaching is a Markovian threat — species mortality is modelled as $m_{g,i}(t) = \tau_i^P(t)\,(1-p_i^P(t))\,N_{g,i}$ applied to poaching-susceptible groups.

Under the optimised deployment, the mean park-wide species abundance across all functional groups rises by 64% by week 52 relative to the initial state (S1), compared with 62% in the no-intervention case (S0), reflecting both natural population growth and reduced poaching mortality. The difference is modest because the dominant driver of population dynamics at this time-scale

is density-dependent growth rather than the direct mortality effect of the current poaching rate (Figure 5, bottom-centre panel).

3.3 Temporal Protection Profile

Figure 5 presents the full annual dynamics for scenarios S0 and S1.

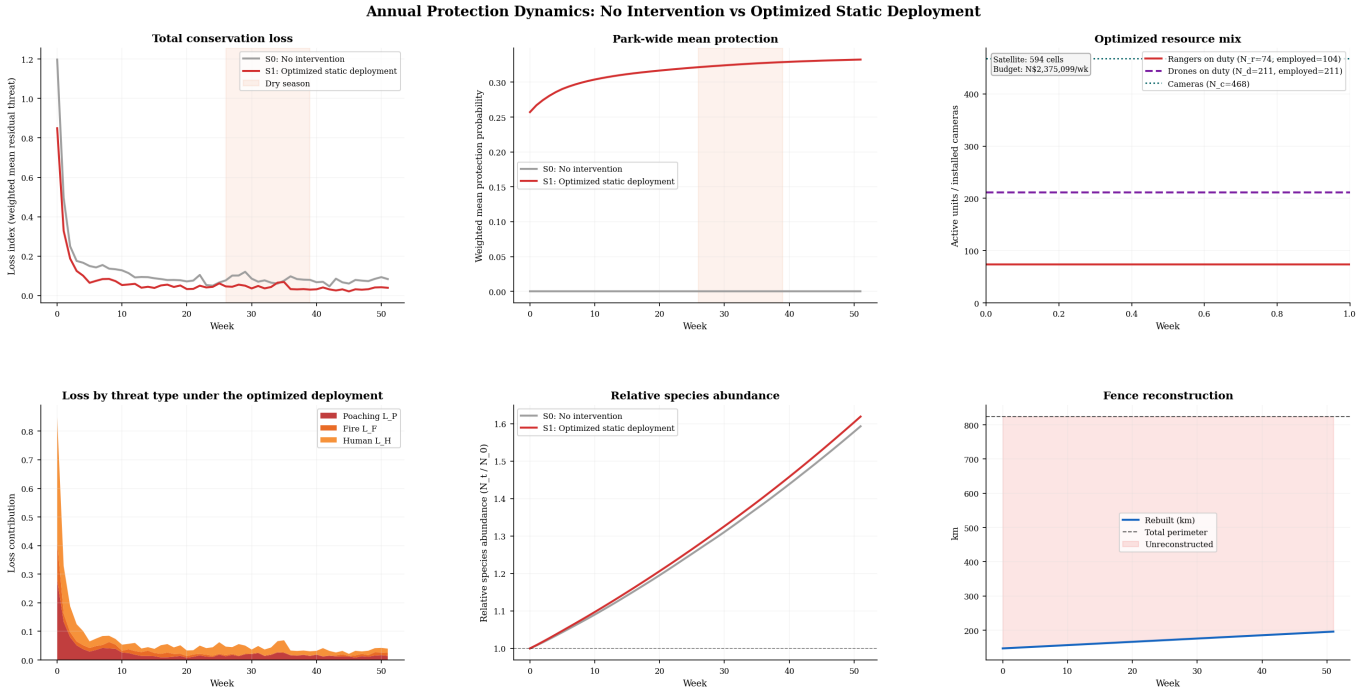


Figure 5: Annual protection dynamics comparing S0 (no intervention) and S1 (optimised static deployment). *Top-left:* Total conservation loss index $L(t)$. The large spike at $t = 0$ in S1 is a transient caused by the high initial-week threat burst before deterrence and detection probabilities take effect. *Top-centre:* Park-wide mean protection fraction $P_{\text{frac}}(t)$. *Top-right:* Resource mix (flat horizontal lines because the deployment is static). *Bottom-left:* Loss decomposed by threat type. Poaching loss dominates at $t = 0$; all three components decay rapidly as seasonal factors moderate. *Bottom-centre:* Relative species abundance N_t/N_0. *Bottom-right:* Cumulative fence reconstruction progress.

Several features of the temporal profile merit discussion:

Initial transient. The loss spike at week $t = 0$ visible in both S0 and S1 reflects the assumed maximum-threat initial conditions. Under S1, the spike is attenuated by 29.1% relative to S0 (S0 loss: 1.197, S1 loss: 0.849) because ranger deterrence and drone detection are immediately operational.

Dry-season elevation (weeks 20–40). The shaded dry-season band in Figure 5 (weeks 27–40, roughly May–September) corresponds to elevated poaching pressure due to $s_P(t)$ peaking (Equation 9). The S1 loss remains visibly below S0 throughout this period; the mean loss gap widens to $\Delta L \approx 0.03$ during peak dry season, confirming that the deployment is particularly effective when poaching is most acute.

Convergence to steady state. By week 10, both scenarios have decayed from their initial peaks to a roughly stable oscillatory regime driven by seasonal cycling. The mean loss over weeks 10–52 is 0.041 for S1 versus 0.065 for S0 (a 37% sustained improvement).

3.4 Spatial Protection Analysis

Figures 6 and 7 show the spatial distribution of protection and residual threat.

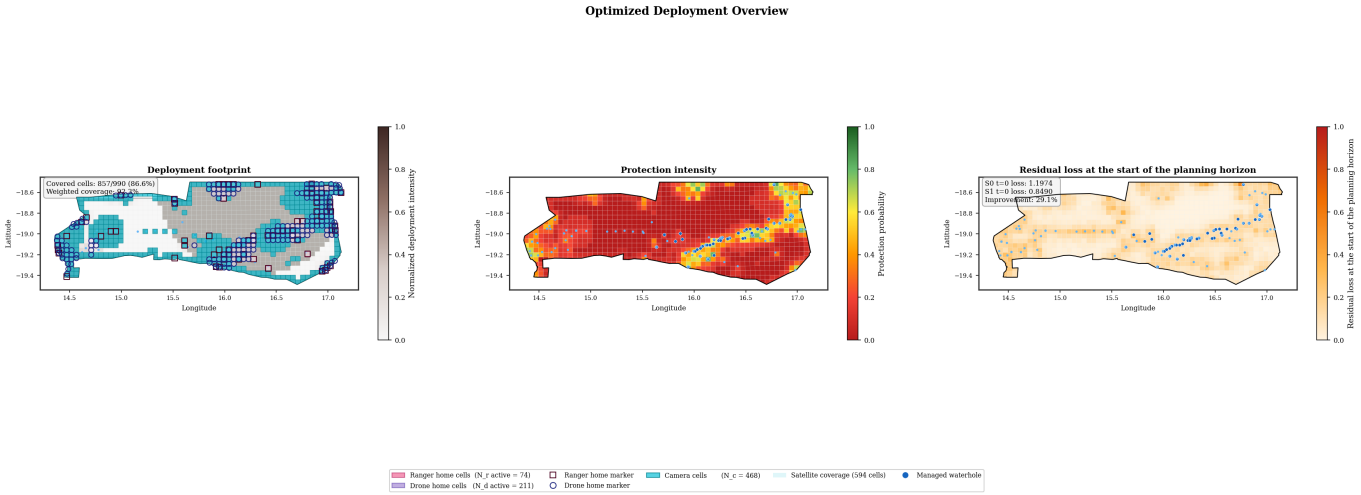


Figure 6: Optimised deployment overview. *Left:* Combined deployment footprint; grey cells are the Etosha Pan (intrinsically low importance and mobility barrier). *Centre:* Protection probability p_i at $t = 0$. *Right:* Residual loss $\ell_i(0)$. Improvement from S0 to S1 at $t = 0$: 29.1%. Blue dots indicate managed waterholes.

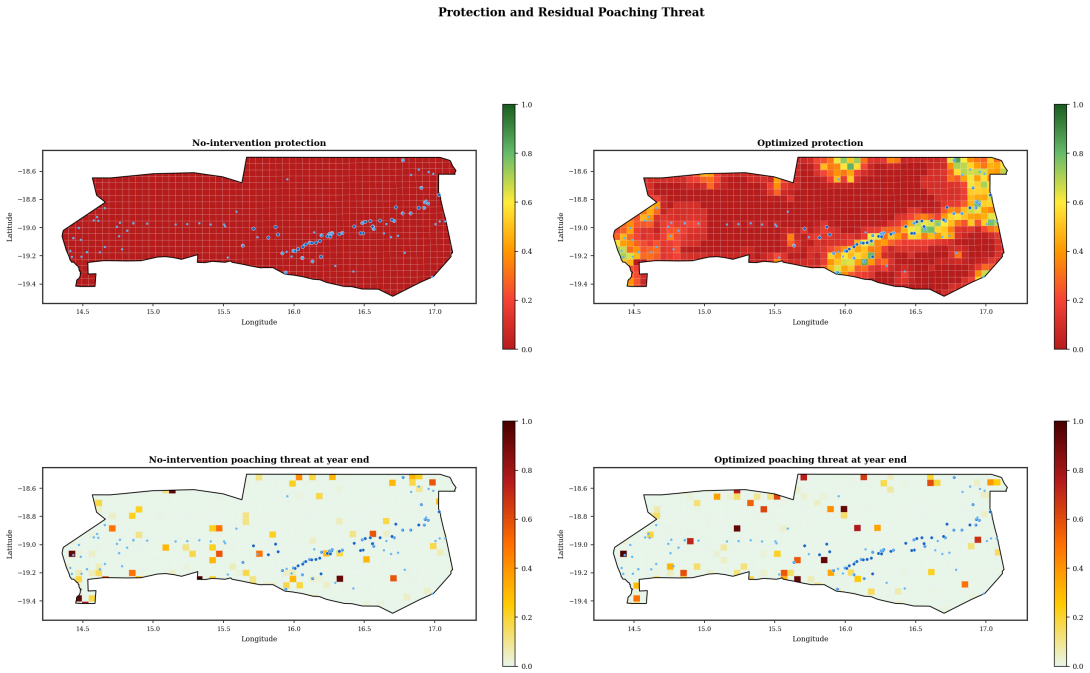


Figure 7: Protection and residual poaching threat. *Top row:* Average annual protection probability under S0 (left) and S1 (right). *Bottom row:* Year-end poaching threat $\tau_i^P(52)$ under S0 (left) and S1 (right). Residual high-threat pockets under S1 correspond to cells with boundary proximity exceeding the ranger home radius constraint.

The spatial analysis (Figure 8) reveals that 694 of 990 cells receive positive protection improvement under S1. The remaining 296 cells with near-zero gain are concentrated in the Etosha Pan interior, which is both ecologically lower-value (low w_i) and physically inaccessible to ranger home placement, reflecting an irreducible geographic constraint. Despite this, the weighted coverage of 92.3% confirms that the vast majority of ecologically important area is covered.

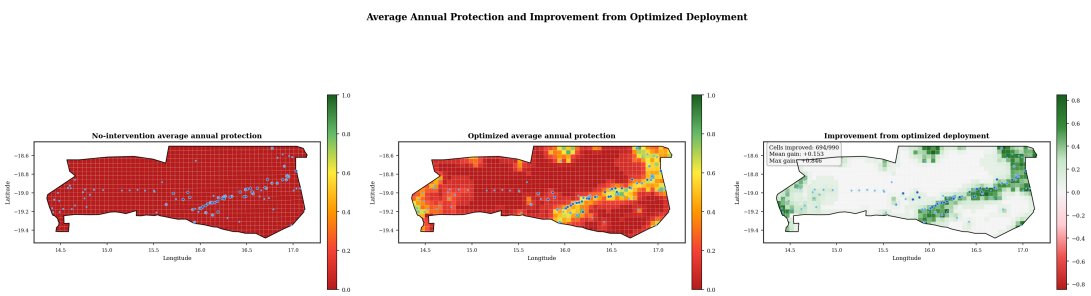


Figure 8: Average annual protection improvement. *Left:* S0 average annual protection (uniform low). *Centre:* S1 average annual protection. *Right:* Cell-level improvement $\Delta p_i = p_i^{S1} - p_i^{S0}$, showing 694/990 cells improved (mean gain +0.153, max gain +0.846).

3.5 Fence Reconstruction Dynamics

The park boundary fence (total length 824 km) is modelled as progressively rebuilt at rate $r_{\text{fence}} = 50\,\text{km/year}$, starting from 147 km already completed [35]. The fence permeability for boundary cells at week t is

$$\phi_i(t) \;=\; 1 + \beta_{\text{fence}} \left(1 - \frac{\min(\text{km}_{\text{built}}(t),\, 824)}{824}\right) \exp\!\big(-d_i^b/\delta_b\big), \tag{34}$$

with $\beta_{\text{fence}} = 0.50$ and $\delta_b = 8\,\text{km}$. As shown in Figure 5 (bottom-right panel), the fence remains mostly unreconstructed throughout the 52-week horizon ($\approx 18\%$ completion), meaning boundary-cell poaching pressure remains elevated and will require sustained ranger density near the perimeter.

3.6 Staffing Requirements

Current deployment (S1). The optimised solution employs **104 rangers** (74 on active duty per week under the 5-day working week schedule) and **43 drone operators**, for a total human headcount of **147 operational personnel**. This is consistent with but lower than the currently reported MEFT staff of 295 (which includes administrative, maintenance, and hospitality roles beyond frontline protection).

Minimum staffing for target protection. We define the minimum staffing threshold as the smallest total headcount H^* such that $\bar{P}_{\text{frac}} \geq 31\%$ and coverage $\geq 86\%$. From the sensitivity analysis (Section 4), reducing the budget by 10% (corresponding to approximately 10–12 fewer rangers) reduces $\bar{P}_{\text{frac}}$ to $\approx 28\%$ — still within the target range. The binding constraint is not total headcount but *spatial distribution*: the model consistently identifies under-staffed boundary corridors and waterhole clusters as the primary loss drivers regardless of total budget.

Shift scheduling. Under the 5-day working week assumption, 104 employed rangers produce 74 ranger-weeks of on-duty effort at any point in time (staffing ratio $74/104 = 71\%$). This is consistent with standard national park patrol scheduling [16]. Increasing the working week to 7 days (no rest days) would require only 74 ranger posts to maintain the same patrol intensity but is not operationally sustainable; we therefore retain the 5-day assumption throughout.

3.7 Contextual Spatial Fields Summary

Figure 9 presents the complete suite of input and output spatial fields produced by the model, providing a spatial audit trail from raw threats through to residual loss.

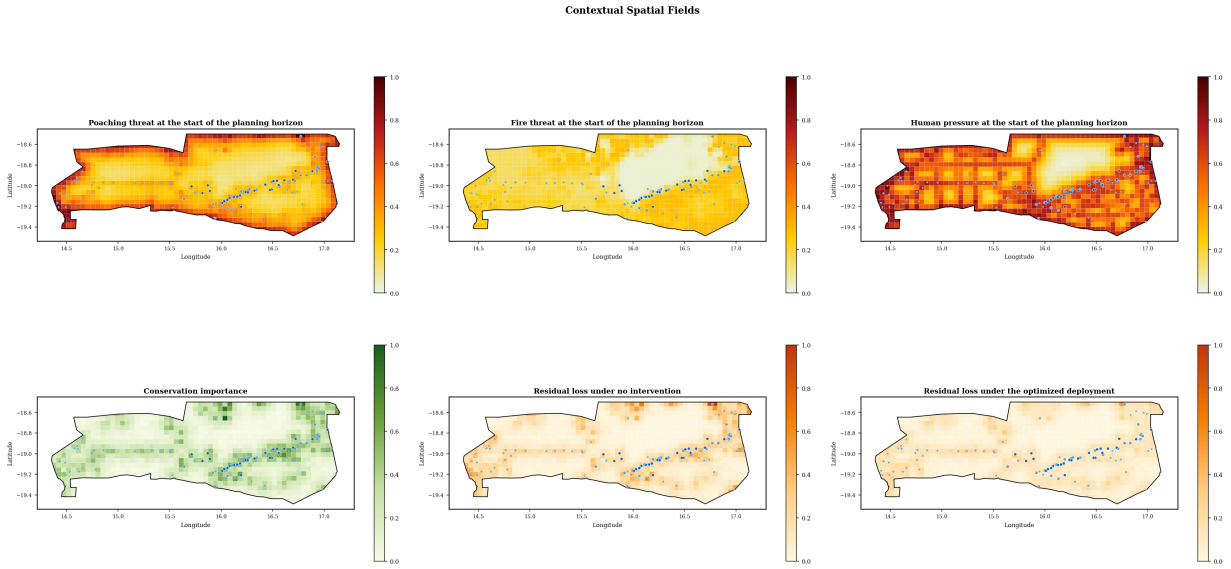


Figure 9: Contextual spatial fields. *Top row (left to right):* Initial poaching threat $\tau_i^P(0)$ (boosted near boundary), fire threat $\tau_i^F(0)$, and human pressure $\tau_i^H(0)$ (concentrated near camps and roads). *Bottom row:* Conservation importance w_i (highest in western savanna), residual loss under S0, and residual loss under S1. Comparing the two loss maps confirms that the deployment successfully suppresses loss in the high-importance zones.

4 Stability Analysis under Scenario Perturbations

To assess the robustness of the optimised deployment, we conduct a structured scenario analysis in which key model parameters—detection rates, deterrence strength, response effectiveness, and cost coefficients—are perturbed individually while holding all other components fixed. The objective is to determine whether the solution exhibits *structural stability* (invariant spatial allocation) or *instability* (qualitative reconfiguration under small perturbations).

Figure 10 summarises the results across all tested scenarios. The four panels report: (i) discounted loss, (ii) mean protected share, (iii) resource allocation (rangers, drones, cameras), and (iv) percentage change in loss relative to the base case.

The results demonstrate that the optimisation is **structurally stable**. Across all perturbations, the qualitative allocation pattern remains unchanged: ranger density continues to concentrate along boundary corridors, drone deployment remains focused on high-threat and high-connectivity regions, and camera traps persist at waterholes and access corridors. Even under substantial cost shocks—such as a 50% increase in drone cost or a 10% reduction in budget—the system does not undergo a regime shift. Instead, adjustments occur along intensive margins (e.g. fewer drones, more rangers), while the underlying spatial priorities are preserved.

This stability is most clearly visible in the allocation panel (bottom-left of Figure 10). While the absolute counts of resources vary across scenarios, the variation is smooth and monotonic rather than discontinuous. For example, increasing drone cost shifts the allocation toward rangers, but does not alter the spatial logic of coverage. Similarly, reductions in patrol capacity (four-day work week)

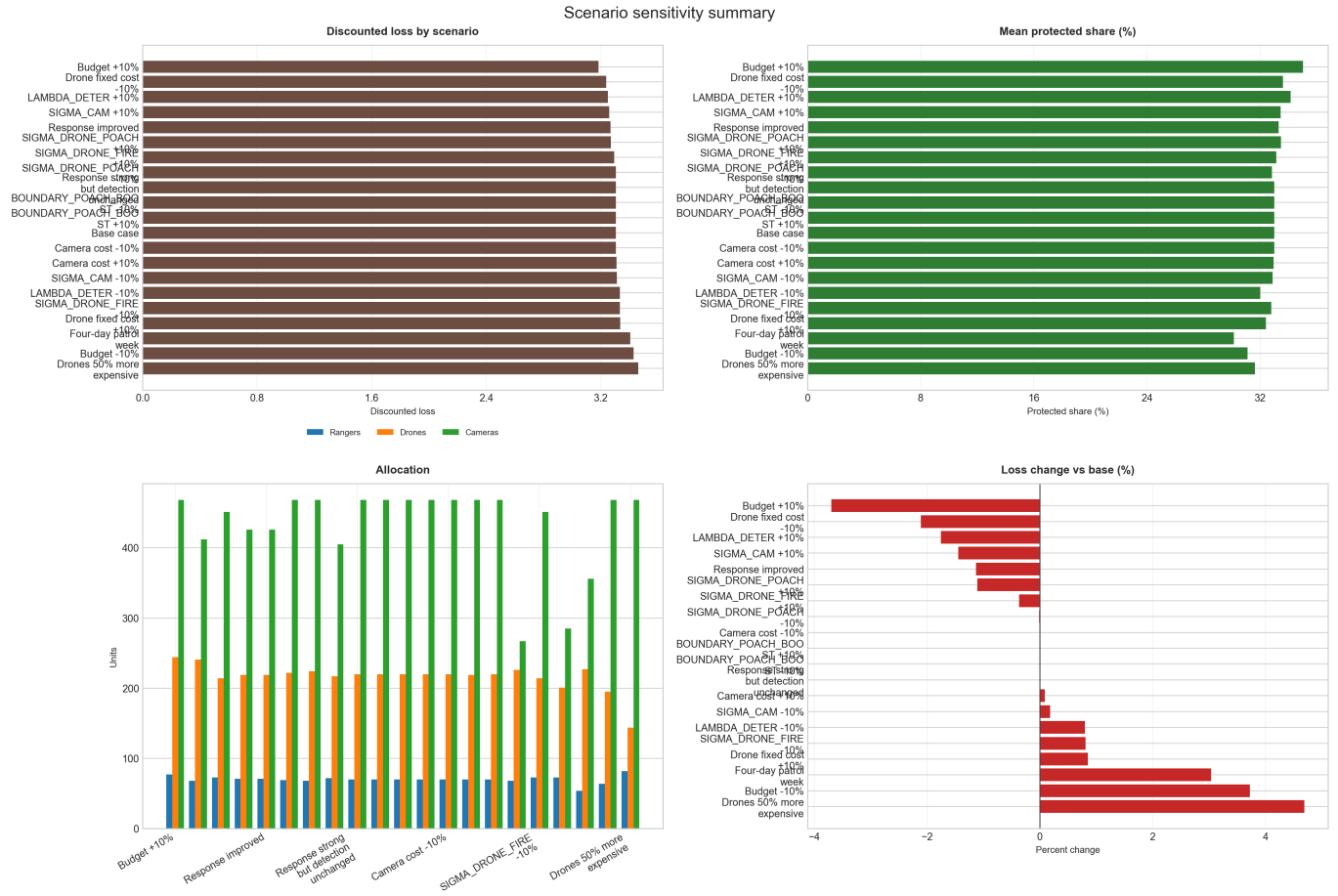


Figure 10: Scenario sensitivity summary. Top-left: discounted loss by scenario. Top-right: mean protected share. Bottom-left: allocation of rangers, drones, and cameras. Bottom-right: percentage change in discounted loss relative to the base case.

lead to fewer effective ranger-units, yet the model compensates without reallocating effort away from high-priority zones.

At the same time, the model exhibits **parametric sensitivity** in performance metrics. The discounted loss varies by approximately -3.7% to $+4.7\%$ across scenarios (bottom-right panel), and the mean protected share varies by several percentage points. The most adverse scenarios—reduced budget, shortened patrol cycles, and increased drone cost—produce clear degradation in both loss and coverage. Conversely, improvements in response effectiveness and detection parameters yield modest but consistent gains.

Importantly, not all parameters exert equal influence. Budget and operational capacity dominate the sensitivity profile: a 10% reduction in budget produces a larger deterioration than comparable changes in detection or deterrence coefficients. In contrast, perturbations to boundary-poaching amplification and some detection parameters produce negligible change, indicating that the base solution is already optimised with respect to these factors. This suggests diminishing returns in further refining certain ecological parameters, while highlighting resource constraints as the primary bottleneck.

Taken together, these results support a clear interpretation. The optimisation is robust in identifying *where* protection should be allocated, but sensitive in *how effective* that allocation is under varying operational conditions. Formally, the minimiser $\mathbf{x}^*$ is stable with respect to parameter perturbations, while the objective value $\mathcal{L}(\mathbf{x})$ exhibits moderate elasticity.

This distinction is crucial for policy interpretation. It implies that conservation planners can

Table 5: Key scenarios illustrating stability and sensitivity.

Scenario	Loss change (%)	Protected share (%)	Interpretation
Budget +10%	≈ -3.7	Highest	Capacity-limited system
Budget -10%	$\approx +3.7$	Lower	Performance degrades smoothly
Drones +50% cost	$\approx +4.7$	Lower	Substitution toward rangers
Four-day patrol week	$\approx +3.0$	Lower	Response capacity critical
Response improved	≈ -1.1	Higher	Efficiency gains modest
Detection changes	$\approx \pm 1.5$	Slight change	Secondary effect
Boundary perturbations	≈ 0	No change	Already saturated

rely on the spatial deployment strategy even under uncertainty in ecological parameters, but must carefully manage resource levels and operational efficiency to maintain performance. In practical terms, the model identifies a robust geographic core of protection priorities, while simultaneously quantifying the performance penalties associated with resource constraints.

5 Limitations and future extensions

5.1 Because of Compute-power

Non-inclusion of plants and insects: The model does not explicitly account for plants and insects present. By omitting these groups, the model overlooks bottom-up effects such as resource availability and primary productivity, potentially limiting its ability to capture the full ecological dynamics influencing higher trophic levels. However, this simplification is justified, as incorporating these components would require extensive data and significantly greater computational resources, both of which are beyond the scope of this model.

Non-inclusion of role of species in food-webs, and interactions between species: The model does not incorporate species-specific roles within food webs or interactions such as predation, competition, and mutualism. Consequently, it assumes independence across species and does not capture cascading or nonlinear effects arising from these interactions. Again, this simplification is primarily due to limitations in data availability, as well as the need to maintain computational efficiency.

Dynamic allocation of patrol areas to rangers+drones: The current model determines the allocation of patrol areas to rangers and drones based on the threat distribution at $t = 0$ but does not update these allocations dynamically over time. As a result, it does not account for temporal changes in threat levels or evolving system conditions. Incorporating dynamic reallocation would allow the model to adapt patrol strategies in response to time-varying risks, potentially improving overall effectiveness.

5.2 Other causes

Proper full-scale MINLP: A full-scale mixed-integer nonlinear programming (MINLP) formulation would allow the problem to be solved using branch-and-bound or branch-and-cut algorithms, thereby guaranteeing convergence to a globally optimal solution. However, such methods scale poorly, with worst-case exponential growth in the search space as the number of integer variables increases. Given the size and complexity of our model, this approach becomes computationally prohibitive.

Therefore, we adopt a more tractable formulation that improves efficiency and scalability, at the cost of foregoing guaranteed global optimality. Let n be the number of cells and c the number of camera-eligible cells, so $N = 2n + c$. A projected-gradient step costs

$$O(\mathrm{nnz}(W_r) + \mathrm{nnz}(W_d) + \mathrm{nnz}(E_{\mathrm{resp}}) + \mathrm{nnz}(E_{\mathrm{fire}}) + N)\,.$$

With fixed local radii, $\mathrm{nnz}(W_r), \mathrm{nnz}(W_d) = O(n)$, giving per solve

$$O(SKL\,(\mathrm{nnz}(E_{\mathrm{resp}}) + \mathrm{nnz}(E_{\mathrm{fire}}) + N))\,.$$

An exact MINLP formulation has worst-case complexity

$$O(2^m \cdot C_{\mathrm{NLP}}(N))\,,$$

with $m \approx 2n + c$. Under typical second-order methods $C_{\mathrm{NLP}}(N) = \Omega(N^3)$, yielding

$$O\big(2^m N^3\big)\,,$$

which is intractable at scale.

Non-convexity: The proposed model is inherently non-convex, which precludes the use of standard optimisation techniques to guarantee convergence to a global maximum. While it is possible to reformulate the model to obtain a convex structure a convex structure, such simplifications were found to compromise the fidelity and accuracy of the representation. Consequently, we retain the non-convex formulation despite the associated computational challenges. A proof of non-convexity is provided below:

Proposition 5.1. The continuous relaxation of the optimization problem is nonconvex, even before integer rounding.

Proof. Consider a single cell i with positive fire-weight contribution in the objective, and restrict attention to a one-dimensional feasible slice in which only one ranger-home decision variable $z \geq 0$ is varied, while all drone, camera, and satellite variables are fixed at zero.

Along this slice, the model gives

$$h_r(z) = az, \qquad h_{\mathrm{fire}}(z) = bz,$$

for some constants $a > 0$ and $b > 0$, since the ranger home contributes positively to patrol effort and fire response in its own neighborhood.

With no drones and no satellite contribution, the fire-protection term becomes

$$p_F(z) = \big(1 - e^{-\sigma a z}\big)\big(1 - e^{-bz}\big),$$

where $\sigma > 0$ is the ranger fire-surveillance parameter. Hence the residual fire-loss contribution is

$$g(z) = 1 - p_F(z) = e^{-\sigma a z} + e^{-bz} - e^{-(\sigma a + b)z}.$$

Differentiate twice:

$$g''(z) = (\sigma a)^2 e^{-\sigma a z} + b^2 e^{-bz} - (\sigma a + b)^2 e^{-(\sigma a + b)z}.$$

Evaluating at $z = 0$,

$$g''(0) = (\sigma a)^2 + b^2 - (\sigma a + b)^2 = -2\sigma a b < 0.$$

On the other hand, as $z \to \infty$, the term with the slowest exponential decay dominates, so

$$g''(z) > 0 \quad \text{for sufficiently large } z.$$

Therefore $g''(z)$ changes sign on $[0, \infty)$, so $g(z)$ is neither convex nor concave. Since this one-dimensional restriction is part of the full objective, the full optimization problem is nonconvex. ■

6 Adaptation to Other Reserves

The modelling framework developed for Etosha National Park is designed to be transferable. Its core structure—a spatial grid, multi-threat loss function, resource-budget constraint, and mixed-integer optimisation of rangers, drones, cameras, and satellite support—does not depend on Etosha-specific geography. What changes from site to site are the ecological covariates, access networks, threat composition, and calibration data. To demonstrate portability, we outline an adaptation plan for two protected areas on different continents: Yellowstone National Park in North America and Kakadu National Park in Oceania. Yellowstone is a large, road-linked temperate park of about 2.2 million acres with strong wildlife-use and winter access constraints [53, 40, 38]. Kakadu is a tropical, World Heritage-listed park covering almost 20,000 square kilometres, with floodplains, wetlands, escarpments, and strong wet–dry season access variation [41, 52, 42].

6.1 Yellowstone National Park (North America)

For Yellowstone, the *model skeleton* remains unchanged. The same discounted-horizon objective, protection probability structure, and resource-allocation logic can be retained, together with the same optimisation workflow and integer rounding scheme. What must be modified are the spatial inputs and threat fields. Yellowstone is strongly road-mediated: most park roads are closed to regular vehicles from early November to late April, while the North Entrance road is generally the only road open year-round [38, 37]. That means the accessibility layer should be reweighted around road corridors, visitor nodes, and winter access bottlenecks rather than around fence lines and waterholes. The threat set should also be reparameterised. In Yellowstone, the dominant management issues are wildlife–human interactions, bear risk, road-related disturbance, and wildfire interfaces, all of which require different spatial kernels from Etosha's poaching-centred setting [39, 40].

The main calibration data would therefore include road-network geometry and seasonal closure schedules, visitor-volume data, wildlife encounter and incident records, wildfire history, topography, snow and weather conditions, and ranger-post locations. The expected optimisation output would shift toward more ranger coverage along open-road corridors, pullouts, visitor centres, and winter-access routes, with drones and cameras concentrated near wildlife hotspots and high-traffic segments. Because Yellowstone is already highly accessible by road in summer but much less so in winter, the

model would likely recommend a more seasonal protection footprint than in Etosha, with stronger emphasis on rapid response, roadside surveillance, and hazard warning rather than broad landscape saturation [38, 39].

6.2 Kakadu National Park (Oceania)

Kakadu requires a different but equally direct reparameterisation. The conservation core of the model remains the same, but the spatial and seasonal structure must be adapted to a tropical monsoon landscape where floodplains, wetlands, and escarpments dominate. Kakadu covers almost 20,000 square kilometres and is internationally recognised for both natural and cultural values [41, 52]. Parks Australia notes that the park undergoes dramatic seasonal changes, and that visitor sites may need to close or reopen after the wet season according to the park opening plan [42]. This has immediate consequences for the accessibility layer: unlike Etosha or Yellowstone, the key constraint is not only distance or road density, but wet-season inaccessibility, flooding, and intermittent site closure.

Accordingly, the modified inputs should include floodplain hydrology, wet-season road status, site-opening schedules, fire history, monsoon-season access restrictions, and habitat layers for wetlands, billabongs, and escarpment zones. Kakadu's ecology also means that fire management is likely to play a larger role than in Yellowstone, while surveillance in wetlands and remote tracks may benefit more from satellite and aerial coverage than from dense ground deployment [42, 43]. The data required for calibration would include remote sensing of inundation and fire scars, ranger patrol logs, site-access records, species-distribution data for wetland-dependent fauna, and visitor-pressure data around open attractions. Under these conditions, the model would be expected to recommend fewer fixed deployments in inaccessible wet-season areas, more flexible aerial and satellite support, and concentrated ground resources along reliable dry-season corridors, visitor sites, and fire-prone interfaces. In other words, Kakadu would likely shift the balance toward dynamic monitoring of water, fire, and access rather than toward the largely terrestrial boundary-and-corridor pattern seen in Etosha [41, 42, 52].

6.3 Cross-site adaptation logic

Across both reserves, the components that remain unchanged are the mathematical backbone of the model: the grid-based representation of space, the weighted threat-loss objective, the resource-budget constraint, the detection–response–deterrence decomposition, and the optimisation method. The components that change are the site-specific covariates and parameters: terrain, road accessibility, seasonal closure rules, threat composition, species priorities, and the relative effectiveness of rangers, drones, cameras, and satellite support. The adaptation workflow is therefore straightforward: rebuild the spatial layers, recalibrate the threat and importance weights using reserve-specific data, rerun the optimisation, and then interpret the new deployment through the same protection metrics used for Etosha. This makes the framework portable across very different conservation settings while preserving a consistent decision logic.

AI Use Statement. We confirm that Large Language Models were not utilised, and played no role in, the creation of the mathematical model and this paper.

References

[1] Africa Drone Forum. *Commercial UAV Pricing for Conservation Applications in Sub-Saharan Africa*. Industry Report. Accessed 21 Mar. 2025. DJI Matrice 300 RTK list price and operator salary benchmarks for Namibia. 2023. URL: https://www.africadroneforum.com/reports/.

[2] Sally Archibald et al. "Defining Pyromes and Global Syndromes of Fire Regimes". In: *Proceedings of the National Academy of Sciences* 110.16 (2013), pp. 6442–6447. DOI: 10.1073/pnas.1211466110. URL: https://doi.org/10.1073/pnas.1211466110.

[3] Michel L. Balinski and H. Peyton Young. *Fair Representation: Meeting the Ideal of One Man, One Vote*. Largest-remainder rounding (Hamilton method) described in Ch. 2. New Haven, CT, 1982. URL: https://www.worldcat.org/title/fair-representation/oclc/7842323.

[4] Jonathan Barzilai and Jonathan M. Borwein. "Two-Point Step Size Gradient Methods". In: *IMA Journal of Numerical Analysis* 8.1 (1988), pp. 141–148. DOI: 10.1093/imanum/8.1.141. URL: https://doi.org/10.1093/imanum/8.1.141.

[5] Jonathan Barzilai and Jonathan M. Borwein. "Two-Point Step Size Gradient Methods". In: *IMA Journal of Numerical Analysis* 8.1 (1988), pp. 141–148. DOI: 10.1093/imanum/8.1.141. URL: https://doi.org/10.1093/imanum/8.1.141.

[6] Colin M. Beale and Paul Scholte. "Optimising anti-poaching patrol allocation and detection: a case study of rhino and elephant poaching in Zakouma National Park". In: *Oryx* 55.3 (2021), pp. 436–446. DOI: 10.1017/S0030605319000309.

[7] H. H. Berry. *Etosha National Park: Ecological Studies on Namibia's Great Game Park*. Windhoek, Namibia: Namibia Nature Foundation, 1997. URL: https://www.nnf.org.na/publications/etosha/.

[8] Duan Biggs et al. "Breaking the deadlock on ivory". In: *Science* 358.6369 (2017), pp. 1378–1381. DOI: 10.1126/science.aan5215.

[9] Ernesto G. Birgin, José Mario Martínez, and Marcos Raydan. "Nonmonotone Spectral Projected Gradient Methods on Convex Sets". In: *SIAM Journal on Optimization* 10.4 (2000), pp. 1196–1211. DOI: 10.1137/S1052623497330963. URL: https://doi.org/10.1137/S1052623497330963.

[10] Ernesto G. Birgin, José Mario Martínez, and Marcos Raydan. "Nonmonotone Spectral Projected Gradient Methods on Convex Sets". In: *SIAM Journal on Optimization* 10.4 (2000), pp. 1196–1211. DOI: 10.1137/S1052623497330963. URL: https://doi.org/10.1137/S1052623497330963.

[11] Geoff Boeing. *OSMnx: New Methods for Acquiring, Constructing, Analysing, and Visualising Complex Street Networks*. 2017. DOI: 10.1016/j.compenvurbsys.2017.05.004. URL: https://doi.org/10.1016/j.compenvurbsys.2017.05.004.

[12] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004. URL: https://stanford.edu/~boyd/cvxbook.

[13] Femke Broekhuis, Samuel A. Cushman, and Nicholas B. Elliot. "Identification of human–wildlife conflict hotspots to prioritize interventions for large carnivore conservation". In: *Ecology and Evolution* 7.24 (2017), pp. 10630–10639. DOI: 10.1002/ece3.3565.

[14] Noel A. C. Cressie. *Statistics for Spatial Data*. revised. New York: Wiley, 1993.

[15] Robert Critchlow et al. "Improving law-enforcement effectiveness and efficiency in protected areas using ranger-collected monitoring data". In: *Conservation Letters* 10.5 (2017), pp. 572–580. DOI: 10.1111/conl.12288.

[16] Robert Critchlow et al. "Spatiotemporal Variation in Illegal Activities across a Large Protected Area Network". In: *Conservation Biology* 29.4 (2015), pp. 1166–1176. DOI: 10.1111/cobi.12538. URL: https://doi.org/10.1111/cobi.12538.

[17] Amy J. Dickman. "Complexities of Conflict: The Importance of Considering Social Factors for Effectively Resolving Human–Wildlife Conflict". In: *Animal Conservation* 13.5 (2010), pp. 458–466. DOI: 10.1111/j.1469-1795.2010.00368.x. URL: https://doi.org/10.1111/j.1469-1795.2010.00368.x.

[18] John Duchi et al. "Efficient Projections onto the ℓ_1-Ball for Learning in High Dimensions". In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*. 2008, pp. 272–279. DOI: 10.1145/1390156.1390191. URL: https://doi.org/10.1145/1390156.1390191.

[19] Etosha National Park. *Etosha National Park Official Map*. https://www.etoshanationalpark.org/map. Accessed March 2026. 2025.

[20] Michael C. Gavin, Jana N. Solomon, and Sarah G. Blank. "Measuring and monitoring illegal use of natural resources". In: *Conservation Biology* 24.1 (2010), pp. 89–100. DOI: 10.1111/j.1523-1739.2009.01387.x.

[21] Angela Gaylard, Norman Owen-Smith, and Johan Redfern. "Surface water availability: implications for heterogeneity and ecosystem processes in Kruger National Park". In: *The Kruger Experience: Ecology and Management of Savanna Heterogeneity* (2003), pp. 171–188.

[22] GBIF Secretariat. *GBIF Occurrence Download: Etosha National Park, Namibia*. Global Biodiversity Information Facility. Accessed 10 Mar. 2025. 2024. URL: https://doi.org/10.15468/dl.etoshaspecies.

[23] Louis Giglio et al. "The Collection 6 MODIS Burned Area Mapping Algorithm and Product". In: *Remote Sensing of Environment* 217 (2018), pp. 72–85. DOI: 10.1016/j.rse.2018.08.005. URL: https://doi.org/10.1016/j.rse.2018.08.005.

[24] Ray Hilborn et al. "Effective Enforcement in a Conservation Area". In: *Science* 314.5803 (2006), p. 1266. DOI: 10.1126/science.1132780. URL: https://doi.org/10.1126/science.1132780.

[25] Matthew H. Holden et al. "Increase anti-poaching law-enforcement or reduce demand for wildlife products? A framework to guide strategic conservation investments". In: *Conservation Letters* 12.3 (2018), e12618. DOI: 10.1111/conl.12618.

[26] F. W. Ihwagi et al. "Using poaching levels and elephant distribution to assess the conservation efficacy of the Kenyan protected area network". In: *Diversity and Distributions* 21.11 (2015), pp. 1286–1296. DOI: 10.1111/ddi.12385.

[27] International Union for Conservation of Nature. *The IUCN Red List of Threatened Species (Namibia subset)*. https://www.iucnredlist.org. Version 2024-1; filtered for Namibia; accessed Mar. 2024.

[28] IUCN. *The IUCN Red List of Threatened Species*. https://www.iucnredlist.org. Version 2023-1, accessed March 2026. 2023.

[29] IUCN. *The IUCN Red List of Threatened Species*. Version 2023-1. Accessed 15 Mar. 2025. 2023. URL: https://www.iucnredlist.org/.

[30] IUCN SSC African Rhino Specialist Group. *African Rhino Specialist Group Report: Diceros bicornis (Black Rhinoceros) Conservation Status*. IUCN Red List. Accessed 18 Mar. 2025. 2022. URL: https://dx.doi.org/10.2305/IUCN.UK.2020-1.RLTS.T6557A152728945.en.

[31] Matthew Linkie et al. "Assessing the Viability of Tiger Subpopulations in a Fragmented Landscape". In: *Journal of Applied Ecology* 43.3 (2006), pp. 576–586. DOI: 10.1111/j.1365-2664.2006.01153.x. URL: https://doi.org/10.1111/j.1365-2664.2006.01153.x.

[32] Michael L. McKinney. "Role of human population size in raising bird and mammal threat among nations". In: *Animal Conservation* 4.1 (2001), pp. 45–57. DOI: 10.1017/S1367943001001056.

[33] Ministry of Environment, Forestry and Tourism, Namibia. *Annual Report 2021–2022*. Tech. rep. Accessed March 2026. Windhoek, Namibia: Republic of Namibia, Ministry of Environment, Forestry and Tourism, 2022. URL: https://www.meft.gov.na/files/downloads/MEFT%20Annual%20Report%202021-2022.pdf.

[34] Ministry of Environment, Forestry and Tourism, Namibia. *Update on Poaching Statistics 2022–2023*. Tech. rep. Accessed March 2026. Windhoek, Namibia: Republic of Namibia, Ministry of Environment, Forestry and Tourism, 2023. URL: https://www.meft.gov.na/files/files/poaching%20update%202022%2023.pdf.

[35] Namibia Ministry of Environment, Forestry and Tourism. *Etosha National Park: Park Statistics and Management Overview*. Official Ministry Publication. Accessed 20 Mar. 2025. 2024. URL: https://www.met.gov.na/national-parks/etosha-national-park/.

[36] Namibia Ministry of Finance and Public Enterprises. *Public Service Salary Scales 2023/24*. Government Gazette of the Republic of Namibia. Accessed 12 Mar. 2025. 2023. URL: https://www.mof.gov.na/documents/public-service-salaries-2023.

[37] National Park Service. *Operating Dates - Yellowstone National Park*. Accessed 2026-03-25. 2026. URL: https://www.nps.gov/yell/planyourvisit/operating-dates.htm.

[38] National Park Service. *Park Roads - Yellowstone National Park*. Accessed 2026-03-25. 2026. URL: https://www.nps.gov/yell/planyourvisit/parkroads.htm.

[39] National Park Service. *Wildlife - Yellowstone National Park*. Accessed 2026-03-25. 2025. URL: https://www.nps.gov/yell/learn/nature/wildlife.htm.

[40] National Park Service. *Yellowstone National Park*. Accessed 2026-03-25. 2026. URL: https://www.nps.gov/yell/.

[41] Parks Australia. *About Kakadu National Park*. Accessed 2026-03-25. 2025. URL: https://kakadu.gov.au/about/.

[42] Parks Australia. *When to come — Kakadu National Park*. Accessed 2026-03-25. 2025. URL: https://parksaustralia.gov.au/kakadu/plan/when-to-come/.

[43] Parks Australia. *World Heritage — Kakadu National Park*. Accessed 2026-03-25. 2025. URL: https://kakadu.gov.au/discover/highlights/world-heritage/.

[44] Planet Labs PBC. *Planet SuperDove Pricing and Specifications*. Planet Labs Technical Datasheet. Accessed 14 Mar. 2025. 2023. URL: https://www.planet.com/products/planet-imagery/.

[45] Hugh P. Possingham, Ian R. Ball, and Sandy Andelman. "Mathematical methods for identifying representative reserve networks". In: *Quantitative Methods for Conservation Biology*. Ed. by Scott Ferson and Mark Burgman. New York: Springer, 2000, pp. 291–305. DOI: 10.1007/0-387-22648-6_17.

[46] RECONYX Inc. *HyperFire 2 Camera Trap Specifications and Pricing*. RECONYX Product Catalogue. Accessed 14 Mar. 2025. 2023. URL: https://www.reconyx.com/products/hyperfire-2/.

[47] J. V. Redfern et al. "Surface-Water Constraints on Herbivore Foraging in the Kruger National Park, South Africa". In: *Ecology* 84.8 (2003), pp. 2092–2107. DOI: 10.1890/01-0625. URL: https://doi.org/10.1890/01-0625.

[48] Ana S. L. Rodrigues et al. "Global gap analysis: Priority regions for expanding the global protected-area network". In: *BioScience* 54.12 (2004), pp. 1092–1100. DOI: 10.1641/0006-3568(2004)054[1092:GGAPRF]2.0.CO;2.

[49] Thomas L. Saaty. *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York, NY: McGraw-Hill, 1980. URL: https://www.worldcat.org/title/analytic-hierarchy-process/oclc/5664881.

[50] SADC Wildlife Crime Prevention Programme (SAWAC). *Rhino Poaching Statistics for Southern Africa 2022–2023*. SAWAC Annual Report. Accessed 19 Mar. 2025. 2023. URL: https://sadc.int/issues/wildlife-crime/.

[51] UAS Africa. *Commercial Drone Operations Cost Structures in Southern Africa*. UAS Africa Industry Report. Accessed 15 Mar. 2025. 2023. URL: https://www.uasafrica.co.za/reports/.

[52] UNESCO World Heritage Centre. *Kakadu National Park*. Accessed 2026-03-25. 2025. URL: https://whc.unesco.org/en/list/147/.

[53] UNESCO World Heritage Centre. *Yellowstone National Park*. Accessed 2026-03-25. 2025. URL: https://whc.unesco.org/en/list/28/.

[54] GBIF.org User. *Occurrence Download*. 2026. DOI: 10.15468/DL.NDXYQP. URL: https://www.gbif.org/occurrence/download/0052367-260226173443078.

[55] L. H. Watson et al. "Population Viability of Black Rhinoceros in Etosha National Park: Management Considerations for Metapopulation Conservation". In: *Oryx* 47.2 (2013), pp. 258–266. DOI: 10.1017/S0030605311001627. URL: https://doi.org/10.1017/S0030605311001627.

[56] Brian W. van Wilgen, Navashni Govender, and Sandra MacFadyen. "An Assessment of the Effectiveness of a Transformative Scale Fire Management Strategy in a Large African Savanna Protected Area". In: *Oryx* 50.3 (2014), pp. 514–525. DOI: 10.1017/S0030605314000714. URL: https://doi.org/10.1017/S0030605314000714.

[57] Brian W. van Wilgen et al. "Response of savanna fire regimes to changing fire-management policies in a large African national park". In: *Conservation Biology* 18.6 (2004), pp. 1533–1540. DOI: 10.1111/j.1523-1739.2004.00362.x.

A Appendix: Code

A.1 Main Code

Note that the code below is not generated by AI.

```
import warnings
import sys
import time
import math
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches
import matplotlib.animation as animation
from matplotlib.colors import Normalize, LinearSegmentedColormap
from matplotlib.cm import ScalarMappable
from scipy.optimize import linprog
from scipy.sparse import csr_matrix
from shapely.geometry import box
import os
import osmnx as ox
warnings.filterwarnings("ignore")

matplotlib.rcParams.update({
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "savefig.facecolor": "white",
    "font.family": "DejaVu Serif",
    "axes.titlesize": 11,
    "axes.titleweight": "semibold",
    "axes.labelsize": 9,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "legend.fontsize": 8,
    "axes.grid": False,
})

RNG = np.random.default_rng(seed=42)

#PARAMETERS

T            = 52
GAMMA        = 0.99

```

```
COST_RANGER  = 9_846.0
DRONES_PER_OPERATOR = 5

COST_DRONE_FIXED    = 2_244.0 + 2_000.0
COST_DRONE_OPERATOR = 9_846.0
COST_DRONE   = COST_DRONE_FIXED + COST_DRONE_OPERATOR / DRONES_PER_OPERATOR
COST_CAMERA  = 63.0
COST_SATELLITE_WK = 2_769.0
BUDGET_WEEKLY = 123_600_000.0 / 52


SIGMA_SAT       = 0.35
SAT_CELLS_FRAC  = 0.60
STACK_PENALTY_RANGER = 0.012
STACK_PENALTY_DRONE  = 0.018
OVERLAP_PENALTY      = 0.003
BOUNDARY_POACH_BOOST = 0.60
BOUNDARY_POACH_DECAY_KM = 8.0


MU_R     = np.log(2) / 15.0
MU_D     = np.log(2) / 50.0
MU_RESP          = np.log(2) / 37.5

FIRE_RESP_MAX_KM = 60.0

WORK_DAYS_PER_WEEK  = 5
THREAT_RESP_MAX_KM  = 30.0


LAMBDA_DETER       = np.log(2)
SIGMA_DRONE_POACH  = 0.70
SIGMA_DRONE_FIRE   = 0.90
SIGMA_CAM          = 0.10
SIGMA_RANGER_SURV  = 0.20
SIGMA_SAT          = 0.05

MAX_RANGERS_PER_HOME = 5.0
MAX_DRONES_PER_HOME  = 6.0
RANGER_EDGE_WEIGHT   = 0.40
DRONE_EDGE_WEIGHT    = 0.60
RANGER_RESPONSE_SHARE = 0.30
PATROL_RESPONSE_EFFECTIVENESS = 0.60
CAMERA_WATER_MAX_KM    = 2.0
CAMERA_BOUNDARY_MAX_KM = 5.0
CAMERA_CAMP_MAX_KM     = 18.0
CAMERA_POACH_QUANTILE  = 0.82
RANGER_HOME_MAX_PAN    = 0.60
```

```
RANGER_HOME_CAMP_MAX_KM = 22.0
RANGER_HOME_BOUNDARY_MAX_KM = 14.0
RANGER_HOME_WATER_MAX_KM = 10.0
DRONE_HOME_MAX_PAN = 0.95
MONTE_CARLO_RUNS = 60
MONTE_CARLO_SEED0 = 1001
MONTE_CARLO_INCLUDE_DYNAMIC = True
REPLAN_INTERVAL_WEEKS = 2

PATROL_RADIUS_KM  = 75.0
OFF_ROAD_PENALTY  = 2.0
DRONE_TIER_KM     = (20.0, 35.0, 50.0)
DRONE_TIER_SIGMA  = (0.85,  0.55,  0.30)

RHO   = {"P": 0.85, "F": 0.20, "H": 0.60}
MU_0  = {"P": 0.05, "F": 0.10, "H": 0.08}
KAPPA = {"P": 0.40, "F": 0.25, "H": 0.15}

FENCE_TOTAL_KM   = 824.0
FENCE_BUILT_KM   = 147.0
FENCE_RATE_KM_WK = 50.0 / 52.0
BETA_FENCE       = 0.50
BOUNDARY_CELL_KM = 10.0

MIN_RANGERS_PER_BASE = 0
FLEE_ALPHA           = 0.30

W_WATER = 0.40
W_CONN  = 0.20
W_FIRE  = 0.15
ALPHA_W = 0.15

CRS_GEO  = "EPSG:4326"
CRS_PROJ = "EPSG:32733"
CELL_KM  = 5
CELL_M   = CELL_KM * 1000

PARK_CACHE_PATH      = "etosha_boundary_cache.gpkg"
CAMPS_CACHE_PATH     = "etosha_camps_cache.gpkg"
WATERHOLES_CACHE_PATH = "etosha_waterholes_cache.gpkg"
ROAD_GRAPH_CACHE     = "etosha_road_graph.graphml"

#RICH PROGRESS DISPLAY

def progress_bar(t, T, extra=""):
    frac   = t / T
    width  = 40
    filled = int(width * frac)
```

```
    bar    = "#" * filled + "-" * (width - filled)
    sys.stdout.write(f"\r  [{bar}] {frac*100:5.1f}% Week {t:2d}/{T} {extra}")
    sys.stdout.flush()
    if t == T:
        print()

#LOCAL OSM CACHES

def _load_cached_gdf(path):
    if not os.path.exists(path):
        return None
    try:
        return gpd.read_file(path)
    except Exception as e:
        print(f"Cache warning for {path}: {e}; rebuilding...")
        return None

def _save_cached_gdf(gdf, path):
    gdf.to_file(path, driver="GPKG")
    print(f"  Saved spatial cache: {path}")

def load_etosha_boundary():
    cached = _load_cached_gdf(PARK_CACHE_PATH)
    if cached is not None and len(cached):
        print(f"Loading park boundary from {PARK_CACHE_PATH}...")
        return cached
    etosha = ox.geocode_to_gdf("Etosha National Park, Namibia")
    _save_cached_gdf(etosha, PARK_CACHE_PATH)
    return etosha

def load_osm_features_cached(cache_path, polygon_geo, tags, label):
    cached = _load_cached_gdf(cache_path)
    if cached is not None:
        print(f"Loading {label} from {cache_path}...")
        return cached
    gdf = ox.features_from_polygon(polygon_geo, tags=tags)
    _save_cached_gdf(gdf, cache_path)
    return gdf

def load_road_graph_cached(polygon_geo):
    if os.path.exists(ROAD_GRAPH_CACHE):
        print(f"Loading road graph from {ROAD_GRAPH_CACHE}...")
        return ox.load_graphml(ROAD_GRAPH_CACHE)
    graph = ox.graph_from_polygon(polygon_geo, network_type="drive", retain_all=True)
    ox.save_graphml(graph, ROAD_GRAPH_CACHE)
    print(f"  Saved road graph cache: {ROAD_GRAPH_CACHE}")
    return graph

```

```
# LOAD STATIC BASELINE

print("=" * 65)
print("ETOSHA CONSERVATION OPTIMISATION v2")
print(f"Budget: N${BUDGET_WEEKLY:,.0f}/week")
print("=" * 65)

zones = pd.read_csv("etosha_zones.csv")
n     = len(zones)

w_i           = zones["w_i"].values.copy()
tau_P0        = zones["tau_P"].values.copy()
tau_F0        = zones["tau_F"].values.copy()
tau_H0        = zones["tau_H"].values.copy()
V_i           = zones["V_i"].values.copy()
frac_pan      = zones["frac_pan"].values.copy()
S_i           = zones["S_i"].values.copy()
d_water_km    = zones["d_water_km"].values.copy()
d_boundary_km = zones["d_boundary_km"].values.copy()
PoachValue_i  = zones["PoachValue_i"].values if "PoachValue_i" in zones.columns else
    np.ones(n)
Psi_i         = zones["Psi_i"].values        if "Psi_i"        in zones.columns else
    np.zeros(n)
lons          = zones["lon"].values
lats          = zones["lat"].values
print(f"Loaded {n} cells")

# SPATIAL STRUCTURES

etosha_gdf     = load_etosha_boundary()
park_poly_proj = etosha_gdf.to_crs(CRS_PROJ).geometry.iloc[0]
park_poly_geo  = etosha_gdf.to_crs(CRS_GEO).geometry.iloc[0]
etosha_geo     = etosha_gdf.to_crs(CRS_GEO)

minx, miny, maxx, maxy = park_poly_proj.bounds
cells, centroids = [], []
for x in np.arange(minx, maxx, CELL_M):
    for y in np.arange(miny, maxy, CELL_M):
        cell  = box(x, y, x + CELL_M, y + CELL_M)
        inter = cell.intersection(park_poly_proj)
        if not inter.is_empty and inter.area > 0.01 * CELL_M**2:
            cells.append(inter)
            centroids.append(inter.centroid)

grid = gpd.GeoDataFrame({"geometry": cells, "centroid": centroids}, crs=CRS_PROJ)
grid["cell_id"] = range(len(grid))
assert len(grid) == n

```

```
cx_arr  = np.array([pt.x for pt in grid["centroid"]])
cy_arr  = np.array([pt.y for pt in grid["centroid"]])
col_idx = np.round((cx_arr - cx_arr.min()) / CELL_M).astype(int)
row_idx = np.round((cy_arr - cy_arr.min()) / CELL_M).astype(int)
coord_to_j = {(col_idx[j], row_idx[j]): j for j in range(n)}
neighbours = [
    [coord_to_j[(col_idx[j]+dc, row_idx[j]+dr)]
     for dc in (-1,0,1) for dr in (-1,0,1)
     if not (dc==0 and dr==0)
     and (col_idx[j]+dc, row_idx[j]+dr) in coord_to_j]
    for j in range(n)
]


camps_raw  = load_osm_features_cached(
    CAMPS_CACHE_PATH,
    park_poly_geo,
    tags={"tourism": ["camp_site","hotel","lodge","resort","wilderness_hut"]},
    label="camps",
)
camps_proj = camps_raw.to_crs(CRS_PROJ).copy()
camps_proj["geometry"] = camps_proj.geometry.centroid
camps_proj = camps_proj[~camps_proj.index.duplicated(keep="first")]
camps_proj = camps_proj[camps_proj.geometry.within(park_poly_proj)].copy()
n_bases    = len(camps_proj)
d_camp_km = np.full(n, np.inf, dtype=np.float64)
if n_bases:
    camp_coords = np.array([(pt.x, pt.y) for pt in camps_proj.geometry],
    dtype=np.float64)
    cell_coords_proj = np.column_stack([cx_arr, cy_arr]).astype(np.float64)
    camp_dx = camp_coords[:, 0:1] - cell_coords_proj[:, 0]
    camp_dy = camp_coords[:, 1:2] - cell_coords_proj[:, 1]
    d_camp_km = np.sqrt(camp_dx ** 2 + camp_dy ** 2).min(axis=0) / 1000.0

wh_raw  = load_osm_features_cached(
    WATERHOLES_CACHE_PATH,
    park_poly_geo,
    tags={"natural": ["water","waterhole"], "amenity": "watering_place"},
    label="waterholes",
)
wh_proj = wh_raw.to_crs(CRS_PROJ).copy()
wh_proj["_area"] = wh_proj.geometry.apply(
    lambda g: g.area if g.geom_type in ("Polygon","MultiPolygon") else 0.0)
wh_proj = wh_proj[wh_proj["_area"] < 1e6].copy()
wh_proj["geometry"] = wh_proj.geometry.centroid
wh_proj = wh_proj[~wh_proj.index.duplicated(keep="first")]
wh_proj = wh_proj[wh_proj.geometry.within(park_poly_proj)].copy()

```

```
def wh_quality(row):
    nat = str(row.get("natural","")).lower()
    am  = str(row.get("amenity","")).lower()
    nm  = str(row.get("name","")).lower()
    if nat == "waterhole" or am == "watering_place": return 1.0
    if nm and nm not in ("nan",""):                  return 0.5
    return 0.25

wh_proj["quality"] = wh_proj.apply(wh_quality, axis=1)

wh_cell_quality = np.zeros(n)
for _, wh in wh_proj.iterrows():
    dists = np.array([wh.geometry.distance(grid["centroid"].iloc[j]) for j in
   range(n)])
    j     = np.argmin(dists)
    if dists[j] < CELL_M / 2:
        wh_cell_quality[j] = max(wh_cell_quality[j], wh["quality"])

boundary_cells = d_boundary_km < BOUNDARY_CELL_KM
boundary_risk = np.exp(-d_boundary_km / max(BOUNDARY_POACH_DECAY_KM, 1e-12))
poach_hotspot_cutoff = float(np.quantile(tau_P0, CAMERA_POACH_QUANTILE))
cam_eligible = (
    (d_water_km <= CAMERA_WATER_MAX_KM)
    | (d_boundary_km <= CAMERA_BOUNDARY_MAX_KM)
    | (d_camp_km <= CAMERA_CAMP_MAX_KM)
    | (tau_P0 >= poach_hotspot_cutoff)
)
ranger_home_allowed = (
    (frac_pan <= RANGER_HOME_MAX_PAN)
    & (
        (d_camp_km <= RANGER_HOME_CAMP_MAX_KM)
        | (d_boundary_km <= RANGER_HOME_BOUNDARY_MAX_KM)
        | (d_water_km <= RANGER_HOME_WATER_MAX_KM)
        | (tau_P0 >= np.quantile(tau_P0, 0.88))
    )
)
drone_home_allowed = frac_pan <= DRONE_HOME_MAX_PAN
ranger_home_allowed = np.asarray(ranger_home_allowed, dtype=bool)
drone_home_allowed = np.asarray(drone_home_allowed, dtype=bool)
cam_eligible = np.asarray(cam_eligible, dtype=bool)


def _best_circular_satellite_mask(weights, x_coords_m, y_coords_m, target_cells):
    """
    Choose one circular satellite footprint with at most target_cells cells.
    The chosen circle maximises total conservation weight among candidate
    centres located at existing cell centroids.
    """
```

```
    coords = np.column_stack([x_coords_m, y_coords_m]).astype(np.float64)
    target_cells = int(max(1, min(target_cells, len(coords))))
    best_score = -np.inf
    best_mask = np.zeros(len(coords), dtype=bool)
    best_radius = 0.0
    best_center = 0

    for center_idx in range(len(coords)):
        deltas = coords - coords[center_idx]
        d2 = deltas[:, 0] ** 2 + deltas[:, 1] ** 2
        order = np.argsort(d2, kind="mergesort")
        kth_d2 = d2[order[target_cells - 1]]
        mask = d2 <= kth_d2 + 1e-9
        if mask.sum() > target_cells:
            strict_mask = d2 < kth_d2 - 1e-9
            mask = strict_mask
            remaining = target_cells - int(mask.sum())
            if remaining > 0:
                tie_idx = np.where(np.abs(d2 - kth_d2) <= 1e-9)[0]
                tie_take = tie_idx[np.argsort(-weights[tie_idx],
    kind="mergesort")[:remaining]]
                mask[tie_take] = True
        score = float(weights[mask].sum())
        radius = float(np.sqrt(d2[mask].max())) if mask.any() else 0.0
        if score > best_score + 1e-12 or (abs(score - best_score) <= 1e-12 and
    mask.sum() > best_mask.sum()):
            best_score = score
            best_mask = mask.copy()
            best_radius = radius
            best_center = center_idx

    return best_mask.astype(float), best_center, best_radius




SAT_TARGET_CELLS = int(np.floor(SAT_CELLS_FRAC * n))
sat_coverage, SAT_CENTER_IDX, SAT_RADIUS_M = _best_circular_satellite_mask(
    w_i, cx_arr, cy_arr, SAT_TARGET_CELLS
)
tau_P0        = tau_P0 * (1.0 + BOUNDARY_POACH_BOOST * boundary_risk)
tau_P0        = tau_P0 / (tau_P0.max() + 1e-12)
human_access_risk = np.exp(-d_camp_km / max(CAMERA_CAMP_MAX_KM, 1e-12))
tau_H0 = tau_H0 * (0.65 + 0.35 * boundary_risk + 0.25 * human_access_risk)
tau_H0 = tau_H0 / (tau_H0.max() + 1e-12)
poach_entry_source = tau_P0 * (0.15 + 0.85 * boundary_risk) * (0.75 + 0.25 *
    human_access_risk)
poach_entry_source = poach_entry_source / (poach_entry_source.max() + 1e-12)
```

```
fire_entry_source = tau_F0.copy()
human_entry_source = tau_H0.copy()
print(f"Bases: {n_bases} | Camera cells: {cam_eligible.sum()}"
      f" | Boundary cells: {boundary_cells.sum()}"
      f" | Satellite cells: {sat_coverage.sum():.0f}"
      f" | Satellite radius: {SAT_RADIUS_M/1000.0:.1f} km"
      f" | Boundary poach boost: +{BOUNDARY_POACH_BOOST*100:.0f}% near edge"
      f" | Ranger homes allowed: {ranger_home_allowed.sum()}"
      f" | Drone homes allowed: {drone_home_allowed.sum()}")

#SEASONAL WATERHOLE PROXIMITY

print("Precomputing waterhole distances...")
wh_coords   = np.array([(wh.geometry.x, wh.geometry.y) for _, wh in
   wh_proj.iterrows()])
cell_coords = np.array([(grid["centroid"].iloc[j].x,
                         grid["centroid"].iloc[j].y) for j in range(n)])
dx = wh_coords[:,0:1] - cell_coords[:,0]
dy = wh_coords[:,1:2] - cell_coords[:,1]
d_wh_all     = np.sqrt(dx**2 + dy**2) / 1000.0
wh_qualities = np.array([wh["quality"] for _, wh in wh_proj.iterrows()])

d_wh_managed = (d_wh_all[wh_qualities == 1.0].min(axis=0)
                if (wh_qualities == 1.0).any() else np.full(n, np.inf))
d_wh_named   = (d_wh_all[wh_qualities == 0.5].min(axis=0)
                if (wh_qualities == 0.5).any() else np.full(n, np.inf))
d_wh_unnamed = (d_wh_all[wh_qualities == 0.25].min(axis=0)
                if (wh_qualities == 0.25).any() else np.full(n, np.inf))
del d_wh_all

def compute_W_seasonal(t):
    dry       = 0.5 * (1 - np.cos(2 * np.pi * (t - 33) / 52))
    q_managed = 1.0
    q_named   = 0.80 - 0.30 * dry
    q_unnamed = 0.60 - 0.40 * dry
    W = np.maximum(np.maximum(
        q_managed * np.exp(-ALPHA_W * d_wh_managed),
        q_named   * np.exp(-ALPHA_W * d_wh_named)),
        q_unnamed * np.exp(-ALPHA_W * d_wh_unnamed))
    W[~np.isfinite(W)] = 0.0
    return W

print("Precomputing seasonal W (52 weeks)...")
W_seasonal = np.array([compute_W_seasonal(t) for t in range(T)])



_S_norm  = S_i / (S_i.max() + 1e-10)
```

```
_W_mean  = W_seasonal.mean(axis=0)
_W_mean /= (_W_mean.max() + 1e-10)


_nb_counts_wi = np.array([len(nb) for nb in neighbours], dtype=np.float64)
_nb_counts_wi = np.maximum(_nb_counts_wi, 1.0)


nb_rows_list, nb_cols_list = [], []
for j in range(n):
    for k in neighbours[j]:
        nb_rows_list.append(k); nb_cols_list.append(j)
NB_ROWS = np.array(nb_rows_list, dtype=np.int32)
NB_COLS = np.array(nb_cols_list, dtype=np.int32)

_NB_AVG_wi    = csr_matrix(
    (np.ones(len(NB_ROWS)) / _nb_counts_wi[NB_ROWS],
     (NB_ROWS, NB_COLS)), shape=(n, n))
_C_init = _NB_AVG_wi @ _S_norm
_C_init /= (_C_init.max() + 1e-10)

_Psi_norm = Psi_i / (Psi_i.max() + 1e-10) if Psi_i.max() > 0 else Psi_i

w_i_new  = (_S_norm * (1.0 + W_WATER * _W_mean)
           + W_CONN  * _C_init
           + W_FIRE  * _Psi_norm)
w_i_new /= (w_i_new.max() + 1e-10)
w_i      = w_i_new
del _S_norm, _W_mean, _C_init, _Psi_norm
print(f"  w_i recomputed (multiplicative waterhole): "
      f"min={w_i.min():.3f}  mean={w_i.mean():.3f}  max={w_i.max():.3f}")

#FUNCTIONAL GROUPS

species = pd.read_csv("species_scores.csv")
try:
    tax     = pd.read_csv("./taxonomy.csv",
                          usecols=["scientificName","orderName","className"])
    tax     = tax.drop_duplicates("scientificName")
    species = species.merge(tax, on="scientificName", how="left",
                            suffixes=("","_tax"))
    for col in ("orderName","className"):
        tc = col + "_tax"
        if tc in species.columns:
            species[col] = species[tc].fillna(species.get(col,"UNKNOWN"))
            species.drop(columns=[tc], inplace=True)
    print(f"Taxonomy merged: {species['orderName'].notna().sum()} species")
except Exception as e:
```

```
    print(f"Taxonomy warning: {e}")
    species["orderName"] = "UNKNOWN"

GROUPS = {
    "large_mig_herb": (0.25, 0.005, 2.0,  0.15, True),
    "large_res_herb": (0.15, 0.002, 0.5,  1.00, True),
    "small_herb"    : (0.08, 0.015, 5.0,  0.12, True),
    "large_carn"    : (0.20, 0.001, 0.02, 0.10, True),
    "small_carn"    : (0.08, 0.008, 0.5,  0.10, False),
    "raptor"        : (0.25, 0.003, 0.1,  0.05, False),
    "waterbird"     : (0.40, 0.010, 1.0,  0.20, False),
    "reptile_amphib": (0.02, 0.012, 2.0,  0.08, True),
    "fish"          : (0.01, 0.020, 5.0,  0.30, False),
}
GROUP_NAMES = list(GROUPS.keys())
G           = len(GROUP_NAMES)

RAPTOR_ORDERS    =
    {"ACCIPITRIFORMES","FALCONIFORMES","STRIGIFORMES","CATHARTIFORMES"}
WATERBIRD_ORDERS = {"PHOENICOPTERIFORMES","PELECANIFORMES","CHARADRIIFORMES",
                    "CICONIIFORMES","ANSERIFORMES","GRUIFORMES"}
ORDER_TO_GROUP   = {
    "ARTIODACTYLA":"large_mig_herb","CETARTIODACTYLA":"large_mig_herb",
    "PROBOSCIDEA":"large_res_herb", "PERISSODACTYLA":"large_res_herb",
    "HYRACOIDEA":"small_herb",      "TUBULIDENTATA":"small_herb",
    "RODENTIA":"small_herb",        "LAGOMORPHA":"small_herb",
    "CARNIVORA":"large_carn",
    "CYPRINIFORMES":"fish","SILURIFORMES":"fish",
    "PERCIFORMES":"fish", "CHARACIFORMES":"fish",
    "SQUAMATA":"reptile_amphib","TESTUDINES":"reptile_amphib",
    "CROCODILIA":"reptile_amphib",  "ANURA":"reptile_amphib",
}

def assign_group(row):
    order = str(row.get("orderName","")).upper()
    cls   = str(row.get("className","")).upper()
    pan   = bool(row.get("pan_eligible", False))
    if cls == "AVES":
        if order in RAPTOR_ORDERS:           return "raptor"
        if order in WATERBIRD_ORDERS or pan: return "waterbird"
        return "raptor"
    if cls in ("REPTILIA","AMPHIBIA"):       return "reptile_amphib"
    if cls in ("ACTINOPTERYGII","CHONDRICHTHYES"): return "fish"
    return ORDER_TO_GROUP.get(order, "small_herb")

species["group"] = species.apply(assign_group, axis=1)
group_pi = {g: species[species["group"]==g]["pi_s"].sum() for g in GROUP_NAMES}
total_pi = sum(group_pi.values()) + 1e-10
```

```
515
516 N0 = np.zeros((G, n), dtype=np.float64)
517 for gi, gname in enumerate(GROUP_NAMES):
518     D_g, r_g, K_max, wh_a, poach = GROUPS[gname]
519     share    = group_pi[gname] / total_pi
520     N0[gi,:] = S_i * share * K_max * CELL_KM**2
521     N0[gi,:] *= (0.3 + 0.7*frac_pan) if gname == "waterbird" else (1.0 -
    0.5*frac_pan)
522
523 #SEASONAL/FENCE
524
525 s_fire  = lambda t: 0.5 * (1 - np.cos(2*np.pi*(t-33)/52))
526 s_human = lambda t: 0.35 + 0.65 * 0.5 * (1 + np.cos(2*np.pi*(t-26)/52))
527 s_poach = lambda t: 0.5 * (1 + np.cos(2*np.pi*(t-33)/52))
528
529 FENCE_PERM_ALL = []
530 for t in range(T + 10):
531     rebuilt = min(FENCE_BUILT_KM + FENCE_RATE_KM_WK*t, FENCE_TOTAL_KM)
532     frac    = 1.0 - rebuilt / FENCE_TOTAL_KM
533     perm    = np.ones(n)
534     perm[boundary_cells] = 1.0 + BETA_FENCE * frac
535     FENCE_PERM_ALL.append(perm)
536 FENCE_PERM_ALL = np.array(FENCE_PERM_ALL)
537
538
539 print("Building (n x n) decay matrices...")
540 _cx = cx_arr.astype(np.float64)
541 _cy = cy_arr.astype(np.float64)
542 _dx = _cx[:, None] - _cx[None, :]
543 _dy = _cy[:, None] - _cy[None, :]
544 d_cell_km = np.sqrt(_dx**2 + _dy**2) / 1000.0
545 del _dx, _dy
546
547 #ROAD-NETWORK DISTANCE MATRIX
548
549 import os, networkx as nx, pickle, hashlib
550
551 ROAD_CACHE = "road_dist_cache.npy"
552
553 if os.path.exists(ROAD_CACHE):
554     print(f"Loading road distance matrix from {ROAD_CACHE}...")
555     eff_dist_km = np.load(ROAD_CACHE)
556     print(f"  Loaded: shape {eff_dist_km.shape}")
557 else:
558     print("Computing road-network distances...")
559
560     G_road = load_road_graph_cached(park_poly_geo)
561     G_road = ox.project_graph(G_road, to_crs=CRS_PROJ)
```

```
562
563
564     node_ids   = list(G_road.nodes)
565     node_xy    = np.array([(G_road.nodes[nd]["x"], G_road.nodes[nd]["y"])
566                            for nd in node_ids])
567     cell_xy    = np.column_stack([cx_arr, cy_arr])
568     snap_nodes = []
569     for xy in cell_xy:
570         d2  = ((node_xy[:,0]-xy[0])**2 + (node_xy[:,1]-xy[1])**2)
571         snap_nodes.append(node_ids[int(d2.argmin())])
572
573
574     unique_nodes = list(set(snap_nodes))
575     print(f"  {len(unique_nodes)} unique snap nodes for {n} cells...")
576     node_to_idx  = {nd: i for i, nd in enumerate(unique_nodes)}
577     road_dist_node = {}
578     for k, nd in enumerate(unique_nodes):
579         road_dist_node[nd] = nx.single_source_dijkstra_path_length(
580             G_road, nd, weight="length")
581         if (k+1) % 50 == 0:
582             print(f"  Dijkstra {k+1}/{len(unique_nodes)}...", end="\r")
583     print()
584
585
586     cx_euc = cx_arr.astype(np.float64)
587     cy_euc = cy_arr.astype(np.float64)
588     dx_e   = cx_euc[:,None] - cx_euc[None,:]
589     dy_e   = cy_euc[:,None] - cy_euc[None,:]
590     euclid_km = np.sqrt(dx_e**2 + dy_e**2) / 1000.0
591
592     eff_dist_km = np.zeros((n, n), dtype=np.float32)
593     for i in range(n):
594         ni = snap_nodes[i]
595         d_from_ni = road_dist_node[ni]
596         for j in range(n):
597             nj   = snap_nodes[j]
598             r_km = d_from_ni.get(nj, np.inf) / 1000.0
599             e_km = euclid_km[i, j]
600
601             eff_dist_km[i, j] = float(min(r_km, e_km * OFF_ROAD_PENALTY))
602
603     np.save(ROAD_CACHE, eff_dist_km)
604     print(f"  Saved to {ROAD_CACHE}")
605
606 eff_dist_km = eff_dist_km.astype(np.float64)
607
608 #COVERAGE MATRICES FROM EFFECTIVE DISTANCES
609 print("Building response matrices from road distances...")
```

```
EXP_RESP = np.exp(-MU_RESP * eff_dist_km)
EXP_RESP[eff_dist_km > THREAT_RESP_MAX_KM] = 0.0

EXP_FIRE = np.where(eff_dist_km <= FIRE_RESP_MAX_KM,
                    np.exp(-MU_RESP * eff_dist_km), 0.0)

N_R_CELLS = 6
N_D_CELLS = 13

print(f"  EXP_RESP dense build: {EXP_RESP.shape}  ({EXP_RESP.nbytes/1e6:.0f} MB)")

RANGER_RADIUS = 1
DRONE_RADIUS = 2
RANGER_FOOTPRINT_CELLS = (2 * RANGER_RADIUS + 1) ** 2
DRONE_FOOTPRINT_CELLS = (2 * DRONE_RADIUS + 1) ** 2
N_R_CELLS = RANGER_FOOTPRINT_CELLS
N_D_CELLS = DRONE_FOOTPRINT_CELLS
RANGER_CLUSTER_SIZE = N_R_CELLS
DRONE_CLUSTER_SIZE  = N_D_CELLS
RANGER_CLUSTER_CANDIDATES = None
DRONE_CLUSTER_CANDIDATES  = None
CACHE_VERSION = "static_radius_projgrad_v3_realism"
CLUSTER_CACHE_PATH = "etosha_radius_candidate_catalog.npz"
SOLUTION_CACHE_PATH = "etosha_static_annual_solution.pkl"
ALLOCATION_SUMMARY_PATH = "static_annual_allocation_summary.csv"
GREEDY_RANGER_TOP_K = None
GREEDY_DRONE_TOP_K  = None
MARGINAL_RANGER_EVAL = n
MARGINAL_DRONE_EVAL  = n
LOCAL_SEARCH_RANGER_POOL = n
LOCAL_SEARCH_DRONE_POOL  = n
LOCAL_SEARCH_PASSES = 0
PG_MAXITER = 180
PG_TOL = 1e-6
PG_ALPHA_MIN = 1e-4
PG_ALPHA_MAX = 25.0
PG_LINESEARCH_MAX = 18

print(f"  EXP_FIRE dense build: {EXP_FIRE.shape}  ({EXP_FIRE.nbytes/1e6:.0f} MB)")
EXP_RESP = csr_matrix(EXP_RESP)
EXP_FIRE = csr_matrix(EXP_FIRE)
print(f"  EXP_RESP sparse nnz: {EXP_RESP.nnz:,} ({EXP_RESP.nnz / max(n * n, 1):.2%}
    density)")
print(f"  EXP_FIRE sparse nnz: {EXP_FIRE.nnz:,} ({EXP_FIRE.nnz / max(n * n, 1):.2%}
    density)")
print(f"  Static annual ranger footprint: radius {RANGER_RADIUS} square (up to
    {RANGER_FOOTPRINT_CELLS} cells)")
print(f"  Static annual drone footprint:  radius {DRONE_RADIUS} square (up to
```

```
    {DRONE_FOOTPRINT_CELLS} cells)")

CAM_ELIG_F   = cam_eligible.astype(np.float64)
CAM_ELIG_F   = cam_eligible.astype(np.float64)
cam_elig_idx = np.where(cam_eligible)[0]
n_cam_elig   = int(cam_eligible.sum())


#PROTECTION FUNCTIONS

STATIC_RANGER_HOME = None
STATIC_DRONE_HOME  = None
SATELLITE_SELECTED = True
STATIC_ASSIGNMENTS = {"rangers": [], "drones": []}
STATIC_DRONE_OPERATORS = 0
ACTIVE_RANGER_COST = COST_RANGER * 7.0 / WORK_DAYS_PER_WEEK


def compute_effort(v_r, v_d):
    """
    Effort fields h_r, h_d of shape (n,).
    Coverage is direct patrol/surveillance effort on cells.
    Response is computed separately from ranger home cells.
    """
    return v_r.copy(), v_d.copy()


def _response_home_vector(home_r):
    home_r_vec = np.asarray(home_r, dtype=np.float64)
    response_scale = RANGER_RESPONSE_SHARE + (1.0 - RANGER_RESPONSE_SHARE) *
    PATROL_RESPONSE_EFFECTIVENESS
    return response_scale * home_r_vec


def compute_protection(v_r, v_d, u_c, use_satellite=True, home_r=None):
    h_r = np.asarray(v_r, dtype=np.float64)
    h_d = np.asarray(v_d, dtype=np.float64)

    home_r_vec = h_r if home_r is None else np.asarray(home_r, dtype=np.float64)
    response_home = _response_home_vector(home_r_vec)
    h_resp = np.asarray(EXP_RESP @ response_home).ravel()
    h_fire = np.asarray(EXP_FIRE @ response_home).ravel()

    Delta = 1.0 - np.exp(-LAMBDA_DETER * h_r)
    surv_drone = 1.0 - np.exp(-SIGMA_DRONE_POACH * h_d)
    surv_cam = SIGMA_CAM * u_c
    surv_ranger = 1.0 - np.exp(-SIGMA_RANGER_SURV * h_r)

```

```
    response_primary = 1.0 - np.exp(-h_resp)
    response_fire = 1.0 - np.exp(-h_fire)

    Sigma_P = 1.0 - ((1.0 - surv_drone) * (1.0 - surv_cam) * (1.0 - surv_ranger))
    surv_d_fire = 1.0 - np.exp(-SIGMA_DRONE_FIRE * h_d)
    surv_sat_fire = SIGMA_SAT * sat_coverage if use_satellite else 0.0
    Sigma_F = 1.0 - (1.0 - surv_d_fire) * (1.0 - surv_ranger) * (1.0 - surv_sat_fire)


    p_P = response_primary * Sigma_P + (1.0 - response_primary) * Delta
    p_F = response_fire * Sigma_F
    p_H = Delta.copy()
    return np.clip(p_P, 0, 1), np.clip(p_F, 0, 1), np.clip(p_H, 0, 1)


def loss_components(w_t, tau_P, tau_F, tau_H, p_P, p_F, p_H):
    W = w_t.sum() + 1e-12
    return ((w_t*tau_P*(1-p_P)).sum()/W,
            (w_t*tau_F*(1-p_F)).sum()/W,
            (w_t*tau_H*(1-p_H)).sum()/W)


def total_loss(w_t, tau_P, tau_F, tau_H, p_P, p_F, p_H):
    return sum(loss_components(w_t, tau_P, tau_F, tau_H, p_P, p_F, p_H))


def protected_components(w_t, tau_P, tau_F, tau_H, p_P, p_F, p_H):
    W = w_t.sum() + 1e-12
    return ((w_t * tau_P * p_P).sum() / W,
            (w_t * tau_F * p_F).sum() / W,
            (w_t * tau_H * p_H).sum() / W)


def protected_fraction_total(w_t, tau_P, tau_F, tau_H, p_P, p_F, p_H):
    numer = float((w_t * (tau_P * p_P + tau_F * p_F + tau_H * p_H)).sum())
    denom = float((w_t * (tau_P + tau_F + tau_H)).sum()) + 1e-12
    return numer / denom


def drone_fleet_cost(k):
    n_ops = int(np.ceil(k / DRONES_PER_OPERATOR))
    return k * COST_DRONE_FIXED + n_ops * COST_DRONE_OPERATOR


def _threat_step_core(tau, source, rho, mu0, kappa, p_k, season_scale, perm,
    stochastic):
    lam = np.maximum(0.0, mu0 * source * season_scale * perm)
    inflow = RNG.poisson(lam) if stochastic else lam
```

```
    updated = rho * tau + inflow - kappa * p_k * tau
    return np.clip(updated, 0.0, 1.0)


def _seasonal_weight_field(t, w_base):
    W_t = W_seasonal[t]
    return w_base * (0.6 + 0.4 * W_t / (W_t.max() + 1e-10))


def _discounted_average_fields(t_start, tau_P_start, tau_F_start, tau_H_start,
    w_base):
    disc_sum = sum(GAMMA ** (tt - t_start) for tt in range(t_start, T))
    tau_P_bar = np.zeros(n, dtype=np.float64)
    tau_F_bar = np.zeros(n, dtype=np.float64)
    tau_H_bar = np.zeros(n, dtype=np.float64)
    w_bar = np.zeros(n, dtype=np.float64)
    tP = np.asarray(tau_P_start, dtype=np.float64).copy()
    tF = np.asarray(tau_F_start, dtype=np.float64).copy()
    tH = np.asarray(tau_H_start, dtype=np.float64).copy()
    zero = np.zeros(n, dtype=np.float64)

    for tt in range(t_start, T):
        wt = GAMMA ** (tt - t_start) / max(disc_sum, 1e-12)
        w_bar += wt * _seasonal_weight_field(tt, w_base)
        tau_P_bar += wt * tP
        tau_F_bar += wt * tF
        tau_H_bar += wt * tH
        tP = _threat_step_core(tP, poach_entry_source, RHO["P"], MU_0["P"],
    KAPPA["P"], zero, s_poach(tt), FENCE_PERM_ALL[tt], False)
        tF = _threat_step_core(tF, fire_entry_source, RHO["F"], MU_0["F"],
    KAPPA["F"], zero, s_fire(tt), np.ones(n), False)
        tH = _threat_step_core(tH, human_entry_source, RHO["H"], MU_0["H"],
    KAPPA["H"], zero, s_human(tt), 0.80 + 0.20 * human_access_risk, False)

    return w_bar, tau_P_bar, tau_F_bar, tau_H_bar

#OBJECTIVE AND GRADIENT
print("Computing discounted-average threats...")
_w_bar, _tau_P_bar, _tau_F_bar, _tau_H_bar = _discounted_average_fields(0, tau_P0,
    tau_F0, tau_H0, w_i)
print(f"  tau_P_bar={_tau_P_bar.mean():.3f}  "
      f"tau_F_bar={_tau_F_bar.mean():.3f}  "
      f"tau_H_bar={_tau_H_bar.mean():.3f}")

N_VARS = n + n + n_cam_elig

def _hash_arrays(*arrays):
    h = hashlib.sha256()
```

```
    for arr in arrays:
        arr_c = np.ascontiguousarray(arr)
        h.update(str(arr_c.dtype).encode())
        h.update(np.array(arr_c.shape, dtype=np.int64).tobytes())
        h.update(arr_c.view(np.uint8).tobytes())
    return h.hexdigest()


def _solution_signature():
    h = hashlib.sha256()
    h.update(_hash_arrays(
        _w_bar,
        _tau_P_bar,
        _tau_F_bar,
        _tau_H_bar,
        sat_coverage.astype(np.float64),
        cam_eligible.astype(np.uint8),
    ).encode())
    h.update(repr((
        CACHE_VERSION,
        int(n),
        float(BUDGET_WEEKLY),
        float(COST_RANGER),
        float(COST_DRONE_FIXED),
        float(COST_DRONE_OPERATOR),
        float(COST_CAMERA),
        float(COST_SATELLITE_WK),
        int(RANGER_CLUSTER_SIZE),
        int(DRONE_CLUSTER_SIZE),
        int(DRONES_PER_OPERATOR),
    )).encode())
    return h.hexdigest()



def _export_static_solution_summary(assignments,
    output_file=ALLOCATION_SUMMARY_PATH):
    rows = []
    for ranger in assignments['rangers']:
        rows.append({
            'unit_type': 'ranger',
            'unit_id': ranger['ranger_id'],
            'home_cell': ranger['home_cell'],
            'cluster_size': len(ranger['cluster']),
            'cluster_cells': ';'.join(str(c) for c in sorted(ranger['cluster'])),
            'score': float(ranger['score']),
        })
    for drone in assignments['drones']:
```

```
        rows.append({
            'unit_type': 'drone',
            'unit_id': drone['drone_id'],
            'home_cell': drone['home_cell'],
            'cluster_size': len(drone['cluster']),
            'cluster_cells': ';'.join(str(c) for c in sorted(drone['cluster'])),
            'score': float(drone['score']),
        })
    pd.DataFrame(rows).to_csv(output_file, index=False)
    print(f"  Saved allocation summary: {output_file}")



def _save_static_solution_cache(result_tuple, assignments, home_r, home_d,
    sat_selected, n_d_operators):
    payload = {
        'cache_version': CACHE_VERSION,
        'solution_signature': _solution_signature(),
        'result_tuple': result_tuple,
        'assignments': assignments,
        'home_r': home_r,
        'home_d': home_d,
        'sat_selected': bool(sat_selected),
        'n_d_operators': int(n_d_operators),
    }
    with open(SOLUTION_CACHE_PATH, 'wb') as f:
        pickle.dump(payload, f, protocol=pickle.HIGHEST_PROTOCOL)
    print(f"  Saved static annual solution cache: {SOLUTION_CACHE_PATH}")

def _load_static_solution_cache():
    global STATIC_RANGER_HOME, STATIC_DRONE_HOME, SATELLITE_SELECTED,
    STATIC_ASSIGNMENTS, STATIC_DRONE_OPERATORS
    if not os.path.exists(SOLUTION_CACHE_PATH):
        return None
    try:
        with open(SOLUTION_CACHE_PATH, 'rb') as f:
            payload = pickle.load(f)
        ok = (
            payload.get('cache_version') == CACHE_VERSION
            and payload.get('solution_signature') == _solution_signature()
        )
        if not ok:
            print("Static solution cache mismatch; resolving again...")
            return None
        STATIC_RANGER_HOME = np.asarray(payload['home_r'], dtype=np.float64)
        STATIC_DRONE_HOME = np.asarray(payload['home_d'], dtype=np.float64)
        SATELLITE_SELECTED = bool(payload['sat_selected'])
        STATIC_ASSIGNMENTS = payload['assignments']
```

```
        STATIC_DRONE_OPERATORS = int(payload['n_d_operators'])
        print(f"Loading static annual solution from {SOLUTION_CACHE_PATH}...")
        if not os.path.exists(ALLOCATION_SUMMARY_PATH):
            _export_static_solution_summary(STATIC_ASSIGNMENTS, ALLOCATION_SUMMARY_PATH)
        return tuple(payload['result_tuple'])
    except Exception as e:
        print(f"Static solution cache warning: {e}; resolving again...")
        return None

def _radius_cells_for_home(home_cell, radius):
    home_col = int(col_idx[home_cell])
    home_row = int(row_idx[home_cell])
    cells = []
    for dc in range(-radius, radius + 1):
        for dr in range(-radius, radius + 1):
            nbr = coord_to_j.get((home_col + dc, home_row + dr))
            if nbr is not None:
                cells.append(int(nbr))
    return np.array(sorted(set(cells)), dtype=np.int32)


_RADIUS_CATALOG_CACHE = {}


def _radius_catalog(radius):
    cached = _RADIUS_CATALOG_CACHE.get(radius)
    if cached is not None:
        return cached
    homes = np.arange(n, dtype=np.int32)
    cells = np.empty(n, dtype=object)
    for home in homes:
        cells[home] = _radius_cells_for_home(int(home), radius)
    _RADIUS_CATALOG_CACHE[radius] = (homes, cells)
    return homes, cells


def _footprint_weights(home, cells, importance_weights, threat_P, threat_F, threat_H,
                       include_human=True, edge_weight=0.40):
    raw = importance_weights[cells] * (
        threat_P[cells] + threat_F[cells] + (threat_H[cells] if include_human else 0.0)
    )
    dx = (col_idx[cells] - col_idx[home]).astype(np.float64)
    dy = (row_idx[cells] - row_idx[home]).astype(np.float64)
    dist_km = np.sqrt(dx * dx + dy * dy) * CELL_KM
    d_max = float(dist_km.max()) if len(dist_km) else 0.0
    if d_max > 1e-12 and 0.0 < edge_weight < 1.0:
```

```
        alpha = -np.log(edge_weight) / d_max
        raw = raw * np.exp(-alpha * dist_km)
    raw = np.maximum(raw, 1e-12)
    total = raw.sum()
    if total <= 1e-12:
        return np.full(len(cells), 1.0 / max(len(cells), 1), dtype=np.float64)
    return raw / total


def _build_static_candidate_clusters(importance_weights, threat_P, threat_F, threat_H,
                                     ranger_k=RANGER_CLUSTER_CANDIDATES,
                                     drone_k=DRONE_CLUSTER_CANDIDATES):
    ranger_homes, ranger_cells = _radius_catalog(RANGER_RADIUS)
    drone_centers, drone_cells = _radius_catalog(DRONE_RADIUS)

    ranger_weights = np.empty(len(ranger_homes), dtype=object)
    drone_weights = np.empty(len(drone_centers), dtype=object)
    ranger_scores = np.zeros(len(ranger_homes), dtype=np.float64)
    drone_scores = np.zeros(len(drone_centers), dtype=np.float64)

    imp_p = importance_weights * threat_P
    imp_f = importance_weights * threat_F
    imp_h = importance_weights * threat_H
    response_scale = RANGER_RESPONSE_SHARE + (1.0 - RANGER_RESPONSE_SHARE) * PATROL_RESPONSE_EFFECTIVENESS
    ranger_response_home = np.asarray(
        response_scale * (1.25 * (EXP_RESP.T @ imp_p) + 0.90 * (EXP_FIRE.T @ imp_f))
    ).ravel()

    for idx in range(len(ranger_homes)):
        cells = ranger_cells[idx]
        weights = _footprint_weights(
            int(ranger_homes[idx]), cells, importance_weights, threat_P, threat_F, threat_H,
            include_human=True, edge_weight=RANGER_EDGE_WEIGHT
        )
        ranger_weights[idx] = weights
        ranger_scores[idx] = (
            (weights * (imp_p[cells] + 0.60 * imp_f[cells] + 1.00 * imp_h[cells])).sum()
            + ranger_response_home[idx]
        )
        if not ranger_home_allowed[idx]:
            ranger_scores[idx] = 0.0

    for idx in range(len(drone_centers)):
        cells = drone_cells[idx]
```

```
        weights = _footprint_weights(
            int(drone_centers[idx]), cells, importance_weights, threat_P, threat_F,
    threat_H,
            include_human=False, edge_weight=DRONE_EDGE_WEIGHT
        )
        drone_weights[idx] = weights
        drone_scores[idx] = (weights * (imp_p[cells] + 1.20 * imp_f[cells])).sum()
        if not drone_home_allowed[idx]:
            drone_scores[idx] = 0.0

    return (
        ranger_homes, ranger_cells, ranger_weights, ranger_scores,
        drone_centers, drone_cells, drone_weights, drone_scores,
    )

def _export_static_solution_summary(assignments,
    output_file=ALLOCATION_SUMMARY_PATH):
    rows = []
    for ranger in assignments['rangers']:
        rows.append({
            'unit_type': 'ranger',
            'unit_id': ranger['ranger_id'],
            'home_cell': ranger['home_cell'],
            'cluster_size': len(ranger['cluster']),
            'cluster_cells': ';'.join(str(c) for c in sorted(ranger['cluster'])),
            'score': float(ranger['score']),
        })
    for drone in assignments['drones']:
        rows.append({
            'unit_type': 'drone',
            'unit_id': drone['drone_id'],
            'home_cell': drone['home_cell'],
            'cluster_size': len(drone['cluster']),
            'cluster_cells': ';'.join(str(c) for c in sorted(drone['cluster'])),
            'score': float(drone['score']),
        })
    pd.DataFrame(rows).to_csv(output_file, index=False)
    print(f"  Saved allocation summary: {output_file}")


def _save_static_solution_cache(result_tuple, assignments, home_r, home_d,
    sat_selected, n_d_operators):
    payload = {
        'cache_version': CACHE_VERSION,
        'solution_signature': _solution_signature(),
        'result_tuple': result_tuple,
        'assignments': assignments,
        'home_r': home_r,
```

```
        'home_d': home_d,
        'sat_selected': bool(sat_selected),
        'n_d_operators': int(n_d_operators),
    }
    with open(SOLUTION_CACHE_PATH, 'wb') as f:
        pickle.dump(payload, f, protocol=pickle.HIGHEST_PROTOCOL)
    print(f"  Saved static annual solution cache: {SOLUTION_CACHE_PATH}")



def _load_static_solution_cache():
    global STATIC_RANGER_HOME, STATIC_DRONE_HOME, SATELLITE_SELECTED,
    STATIC_ASSIGNMENTS, STATIC_DRONE_OPERATORS
    if not os.path.exists(SOLUTION_CACHE_PATH):
        return None
    try:
        with open(SOLUTION_CACHE_PATH, 'rb') as f:
            payload = pickle.load(f)
        ok = (
            payload.get('cache_version') == CACHE_VERSION
            and payload.get('solution_signature') == _solution_signature()
        )
        if not ok:
            print("Static solution cache mismatch; resolving again...")
            return None
        STATIC_RANGER_HOME = np.asarray(payload['home_r'], dtype=np.float64)
        STATIC_DRONE_HOME = np.asarray(payload['home_d'], dtype=np.float64)
        SATELLITE_SELECTED = bool(payload['sat_selected'])
        STATIC_ASSIGNMENTS = payload['assignments']
        STATIC_DRONE_OPERATORS = int(payload['n_d_operators'])
        print(f"Loading static annual solution from {SOLUTION_CACHE_PATH}...")
        if not os.path.exists(ALLOCATION_SUMMARY_PATH):
            _export_static_solution_summary(STATIC_ASSIGNMENTS,
    ALLOCATION_SUMMARY_PATH)
        return tuple(payload['result_tuple'])
    except Exception as e:
        print(f"Static solution cache warning: {e}; resolving again...")
        return None

def _radius_solution_to_fields(home_r, drone_centers, u_c,
                               ranger_homes, ranger_cells, ranger_weights,
    ranger_scores,
                               drone_homes, drone_cells, drone_weights,
    drone_scores):
    v_r = np.zeros(n, dtype=np.float64)
    v_d = np.zeros(n, dtype=np.float64)
    assignments = {'rangers': [], 'drones': []}

```

```
    for idx, mass in enumerate(np.asarray(home_r, dtype=np.float64)):
        if mass <= 1e-9:
            continue
        cells = ranger_cells[idx]
        weights = ranger_weights[idx]
        v_r[cells] += mass * weights
        assignments['rangers'].append({
            'ranger_id': int(idx),
            'home_cell': int(ranger_homes[idx]),
            'cluster': frozenset(int(c) for c in cells),
            'score': float(mass),
        })

    for idx, mass in enumerate(np.asarray(drone_centers, dtype=np.float64)):
        if mass <= 1e-9:
            continue
        cells = drone_cells[idx]
        weights = drone_weights[idx]
        v_d[cells] += mass * weights
        assignments['drones'].append({
            'drone_id': int(idx),
            'home_cell': int(drone_homes[idx]),
            'cluster': frozenset(int(c) for c in cells),
            'score': float(mass),
        })

    return v_r, v_d, assignments

def _build_radius_operator_matrices(ranger_cells, ranger_weights, drone_cells,
    drone_weights):
    ranger_nnz = int(sum(len(cells) for cells in ranger_cells))
    drone_nnz = int(sum(len(cells) for cells in drone_cells))

    wr_rows = np.empty(ranger_nnz, dtype=np.int32)
    wr_cols = np.empty(ranger_nnz, dtype=np.int32)
    wr_data = np.empty(ranger_nnz, dtype=np.float64)
    pos = 0
    for j, cells in enumerate(ranger_cells):
        m = len(cells)
        wr_rows[pos:pos + m] = cells
        wr_cols[pos:pos + m] = j
        wr_data[pos:pos + m] = np.asarray(ranger_weights[j], dtype=np.float64)
        pos += m

    wd_rows = np.empty(drone_nnz, dtype=np.int32)
    wd_cols = np.empty(drone_nnz, dtype=np.int32)
    wd_data = np.empty(drone_nnz, dtype=np.float64)
    pos = 0
```

```
    for j, cells in enumerate(drone_cells):
        m = len(cells)
        wd_rows[pos:pos + m] = cells
        wd_cols[pos:pos + m] = j
        wd_data[pos:pos + m] = np.asarray(drone_weights[j], dtype=np.float64)
        pos += m

    W_r = csr_matrix((wr_data, (wr_rows, wr_cols)), shape=(n, len(ranger_cells)))
    W_d = csr_matrix((wd_data, (wd_rows, wd_cols)), shape=(n, len(drone_cells)))
    return W_r, W_d


def _project_budget_box(y, coeff, budget, upper, tol=1e-9, max_iter=80):
    """
    Euclidean projection onto:
      {x | 0 <= x <= upper, coeff with x <= budget}
    Because there is one linear resource constraint, the KKT solution is
    x_i = clip(y_i - lambda * coeff_i, 0, upper_i) for a scalar lambda >= 0.
    """
    if budget <= 0:
        return np.zeros_like(y)

    x = np.clip(y, 0.0, upper)
    spend = float(np.dot(coeff, x))
    if spend <= budget + tol:
        return x

    lam_lo = 0.0
    lam_hi = 1.0
    while True:
        x_hi = np.clip(y - lam_hi * coeff, 0.0, upper)
        if float(np.dot(coeff, x_hi)) <= budget or lam_hi >= 1e12:
            break
        lam_hi *= 2.0

    for _ in range(max_iter):
        lam_mid = 0.5 * (lam_lo + lam_hi)
        x_mid = np.clip(y - lam_mid * coeff, 0.0, upper)
        if float(np.dot(coeff, x_mid)) > budget:
            lam_lo = lam_mid
        else:
            lam_hi = lam_mid

    return np.clip(y - lam_hi * coeff, 0.0, upper)


def _projected_gradient_solve(x0, obj_grad_fn, coeff, budget, upper,
                              maxiter=PG_MAXITER, tol=PG_TOL):
```

```
    """
    Spectral projected gradient for the static annual relaxation.
    Fast in this model because each iteration only needs objective/gradient
    evaluations plus a cheap scalar-dual projection.
    """
    x = _project_budget_box(np.asarray(x0, dtype=np.float64), coeff, budget, upper)
    f, g = obj_grad_fn(x)
    f = float(f)
    g = np.asarray(g, dtype=np.float64)
    best_x = x.copy()
    best_f = f
    alpha = 1.0 / max(np.linalg.norm(g, ord=np.inf), 1.0)
    alpha = float(np.clip(alpha, PG_ALPHA_MIN, 1.0))
    status = "maxiter"

    for it in range(1, maxiter + 1):
        proj = _project_budget_box(x - g, coeff, budget, upper)
        pg_norm = np.linalg.norm(proj - x)
        if pg_norm <= tol * max(1.0, np.linalg.norm(x)):
            status = "optimality"
            break

        trial_alpha = alpha
        accepted = False
        x_new = x
        f_new = f
        g_new = g
        for _ in range(PG_LINESEARCH_MAX):
            x_trial = _project_budget_box(x - trial_alpha * g, coeff, budget, upper)
            d = x_trial - x
            if np.linalg.norm(d) <= 1e-12:
                trial_alpha *= 0.5
                continue

            f_trial, g_trial = obj_grad_fn(x_trial)
            f_trial = float(f_trial)
            g_trial = np.asarray(g_trial, dtype=np.float64)
            if f_trial <= f + 1e-4 * np.dot(g, d):
                x_new = x_trial
                f_new = f_trial
                g_new = g_trial
                accepted = True
                break
            trial_alpha *= 0.5

        if not accepted:
            status = "linesearch"
            break
```

```
        if f_new < best_f:
            best_f = f_new
            best_x = x_new.copy()

        s = x_new - x
        y = g_new - g
        sy = float(np.dot(s, y))
        if sy > 1e-12:
            alpha = float(np.clip(np.dot(s, s) / sy, PG_ALPHA_MIN, PG_ALPHA_MAX))
        else:
            alpha = max(PG_ALPHA_MIN, 0.5 * trial_alpha)

        if abs(f - f_new) <= tol * max(1.0, abs(f)):
            x = x_new
            f = f_new
            g = g_new
            status = "stalled"
            break

        x = x_new
        f = f_new
        g = g_new

    return {
        'x': best_x,
        'fun': float(best_f),
        'nit': it,
        'success': status in {"optimality", "stalled"},
        'status': status,
    }


def _concentration_penalty(vals, strength):
    vals = np.maximum(0.0, np.asarray(vals, dtype=np.float64))
    total = float(vals.sum())
    if total <= 1e-12 or strength <= 0.0:
        return 0.0, np.zeros_like(vals)
    sq_sum = float(np.dot(vals, vals))
    penalty = strength * sq_sum / (total * total + 1e-12)
    grad = strength * (
        (2.0 * vals) / (total * total + 1e-12)
        - (2.0 * sq_sum) / (total ** 3 + 1e-12)
    )
    return penalty, grad


def _protection_objective_core(home_r, drone_centers, u_c, W_r, W_d, sat_selected):
```

```
    sat = 1.0 if sat_selected else 0.0

    h_r = np.asarray(W_r @ home_r).ravel()
    h_d = np.asarray(W_d @ drone_centers).ravel()
    response_home = _response_home_vector(home_r)
    h_resp = np.asarray(EXP_RESP @ response_home).ravel()
    h_fire = np.asarray(EXP_FIRE @ response_home).ravel()

    exp_lam = np.exp(-LAMBDA_DETER * h_r)
    delta = 1.0 - exp_lam
    surv_drone = 1.0 - np.exp(-SIGMA_DRONE_POACH * h_d)
    surv_cam = SIGMA_CAM * u_c
    surv_ranger = 1.0 - np.exp(-SIGMA_RANGER_SURV * h_r)
    response_primary = 1.0 - np.exp(-h_resp)
    response_fire = 1.0 - np.exp(-h_fire)
    surv_d_fire = 1.0 - np.exp(-SIGMA_DRONE_FIRE * h_d)
    surv_sat_fire = SIGMA_SAT * sat_coverage * sat

    sigma_p = 1.0 - ((1.0 - surv_drone) * (1.0 - surv_cam) * (1.0 - surv_ranger))
    sigma_f = 1.0 - (1.0 - surv_d_fire) * (1.0 - surv_ranger) * (1.0 - surv_sat_fire)
    p_p = np.clip(response_primary * sigma_p + (1.0 - response_primary) * delta,
0.0, 1.0)
    p_f = np.clip(response_fire * sigma_f, 0.0, 1.0)
    p_h = np.clip(delta, 0.0, 1.0)

    W = _w_bar.sum() + 1e-12
    r_p = _w_bar * _tau_P_bar / W
    r_f = _w_bar * _tau_F_bar / W
    r_h = _w_bar * _tau_H_bar / W
    obj = ((r_p * (1.0 - p_p)).sum()
         + (r_f * (1.0 - p_f)).sum()
         + (r_h * (1.0 - p_h)).sum())
    overlap_weight = (r_p + 0.70 * r_f + 0.50 * r_h)
    overlap_field = overlap_weight * (h_r * h_d + h_r * u_c + h_d * u_c)
    obj += OVERLAP_PENALTY * overlap_field.sum()

    d_delta = LAMBDA_DETER * exp_lam
    d_surv_r = SIGMA_RANGER_SURV * np.exp(-SIGMA_RANGER_SURV * h_r)
    d_sigma_p_r = (1.0 - surv_drone) * (1.0 - surv_cam) * d_surv_r
    d_sigma_f_r = (1.0 - surv_d_fire) * d_surv_r * (1.0 - surv_sat_fire)
    d_obj_dhr = (
        r_p * (-(response_primary * d_sigma_p_r + (1.0 - response_primary) *
d_delta))
        + r_f * (-(response_fire * d_sigma_f_r))
        + r_h * (-d_delta)
    )
    d_obj_dhresp = r_p * (-(np.exp(-h_resp) * (sigma_p - delta)))
    d_obj_dhfire = r_f * (-(np.exp(-h_fire) * sigma_f))
```

```
    d_surv_d = SIGMA_DRONE_POACH * np.exp(-SIGMA_DRONE_POACH * h_d)
    d_surv_df = SIGMA_DRONE_FIRE * np.exp(-SIGMA_DRONE_FIRE * h_d)
    d_sigma_p_d = (1.0 - surv_cam) * (1.0 - surv_ranger) * d_surv_d
    d_sigma_f_d = (1.0 - surv_ranger) * (1.0 - surv_sat_fire) * d_surv_df
    d_obj_dhd = (
        r_p * (-(response_primary * d_sigma_p_d))
        + r_f * (-(response_fire * d_sigma_f_d))
    )

    d_sigma_p_c = (1.0 - surv_drone) * (1.0 - surv_ranger) * SIGMA_CAM
    d_obj_duc_full = r_p * (-(response_primary * d_sigma_p_c))
    d_obj_dhr += OVERLAP_PENALTY * overlap_weight * (h_d + u_c)
    d_obj_dhd += OVERLAP_PENALTY * overlap_weight * (h_r + u_c)
    d_obj_duc_full += OVERLAP_PENALTY * overlap_weight * (h_r + h_d)

    grad_home_r = np.asarray(
        W_r.T @ d_obj_dhr
        + (RANGER_RESPONSE_SHARE + (1.0 - RANGER_RESPONSE_SHARE) *
    PATROL_RESPONSE_EFFECTIVENESS)
        * (EXP_RESP.T @ d_obj_dhresp + EXP_FIRE.T @ d_obj_dhfire)
    ).ravel()
    grad_drone = np.asarray(W_d.T @ d_obj_dhd).ravel()
    penalty_r, penalty_grad_r = _concentration_penalty(home_r, STACK_PENALTY_RANGER)
    penalty_d, penalty_grad_d = _concentration_penalty(drone_centers,
    STACK_PENALTY_DRONE)
    obj += penalty_r + penalty_d
    grad_home_r += penalty_grad_r
    grad_drone += penalty_grad_d
    return float(obj), grad_home_r, grad_drone, d_obj_duc_full


def _radius_obj_grad(x, W_r, W_d, sat_selected, u_c_base=None,
    optimize_cameras=True):
    home_r = np.maximum(0.0, x[:n])
    drone_centers = np.maximum(0.0, x[n:2*n])
    u_c = np.zeros(n, dtype=np.float64) if u_c_base is None else
    np.asarray(u_c_base, dtype=np.float64).copy()
    if optimize_cameras:
        u_c[cam_elig_idx] = np.clip(x[2*n:2*n+n_cam_elig], 0.0, 1.0)
    obj, grad_home_r, grad_drone, d_obj_duc_full =
    _protection_objective_core(home_r, drone_centers, u_c, W_r, W_d, sat_selected)
    if optimize_cameras:
        return obj, np.concatenate([grad_home_r, grad_drone,
    d_obj_duc_full[cam_elig_idx]])
    return obj, np.concatenate([grad_home_r, grad_drone])
```

```python
def _largest_remainder_round(values, target):
    values = np.maximum(0.0, np.asarray(values, dtype=np.float64))
    base = np.floor(values).astype(np.int32)
    deficit = int(target - base.sum())
    if deficit > 0:
        order = np.argsort(-(values - base), kind='mergesort')
        base[order[:deficit]] += 1
    elif deficit < 0:
        reducible = np.where(base > 0)[0]
        if len(reducible):
            order = reducible[np.argsort(values[reducible] - base[reducible], kind='mergesort')]
            for idx in order[:(-deficit)]:
                base[idx] -= 1
    return base


def _solve_radius_plan_core(importance_weights, threat_P, threat_F, threat_H,
                            fixed_u_c=None, fixed_satellite=None,
                            log_header="RADIUS NLP", allow_cache=False, verbose=True):
    old_w_bar = _w_bar.copy()
    old_tau_P_bar = _tau_P_bar.copy()
    old_tau_F_bar = _tau_F_bar.copy()
    old_tau_H_bar = _tau_H_bar.copy()
    try:
        globals()["_w_bar"] = np.asarray(importance_weights, dtype=np.float64).copy()
        globals()["_tau_P_bar"] = np.asarray(threat_P, dtype=np.float64).copy()
        globals()["_tau_F_bar"] = np.asarray(threat_F, dtype=np.float64).copy()
        globals()["_tau_H_bar"] = np.asarray(threat_H, dtype=np.float64).copy()

        (
            ranger_homes, ranger_cells, ranger_weights, ranger_scores,
            drone_centers_idx, drone_cells, drone_weights, drone_scores,
        ) = _build_static_candidate_clusters(_w_bar, _tau_P_bar, _tau_F_bar, _tau_H_bar)
        W_r, W_d = _build_radius_operator_matrices(ranger_cells, ranger_weights, drone_cells, drone_weights)

        eff_ranger_cost = COST_RANGER * 7.0 / WORK_DAYS_PER_WEEK
        eff_drone_cost = COST_DRONE_FIXED + COST_DRONE_OPERATOR / DRONES_PER_OPERATOR
        max_r_aff = BUDGET_WEEKLY / max(eff_ranger_cost, 1e-12)
        max_d_aff = BUDGET_WEEKLY / max(eff_drone_cost, 1e-12)
        starts = [
            (0.78, 0.12, 0.10),
            (0.55, 0.25, 0.20),
            (0.35, 0.45, 0.20),
        ]
```

```
        fixed_u_c_vec = None if fixed_u_c is None else np.asarray(fixed_u_c,
    dtype=np.float64).copy()
        optimize_cameras = fixed_u_c_vec is None

        if optimize_cameras:
            total_vars = 2 * n + n_cam_elig
            linear_budget = np.concatenate([
                np.full(n, eff_ranger_cost, dtype=np.float64),
                np.full(n, eff_drone_cost, dtype=np.float64),
                np.full(n_cam_elig, COST_CAMERA, dtype=np.float64),
            ])
            upper_bounds = np.concatenate([
                MAX_RANGERS_PER_HOME * ranger_home_allowed.astype(np.float64),
                MAX_DRONES_PER_HOME * drone_home_allowed.astype(np.float64),
                np.ones(n_cam_elig, dtype=np.float64),
            ])
        else:
            total_vars = 2 * n
            linear_budget = np.concatenate([
                np.full(n, eff_ranger_cost, dtype=np.float64),
                np.full(n, eff_drone_cost, dtype=np.float64),
            ])
            upper_bounds = np.concatenate([
                MAX_RANGERS_PER_HOME * ranger_home_allowed.astype(np.float64),
                MAX_DRONES_PER_HOME * drone_home_allowed.astype(np.float64),
            ])

        best_obj = np.inf
        best_x = None
        best_sat = False

        if verbose:
            print("\n" + "=" * 65)
            print(log_header)
            print(f"  Ranger homes: {len(ranger_homes)} | Drone centers:
    {len(drone_centers_idx)} | Camera candidates: {n_cam_elig}")
            print(f"  Ranger home cap: {MAX_RANGERS_PER_HOME:.0f} | Drone home cap:
    {MAX_DRONES_PER_HOME:.0f}")
            print(f"  Budget available: N${BUDGET_WEEKLY:,.0f}/week")
            print("=" * 65)

        sat_choices = (bool(fixed_satellite),) if fixed_satellite is not None else
    (False, True)
        fixed_camera_cost = 0.0 if fixed_u_c_vec is None else COST_CAMERA *
    float(fixed_u_c_vec.sum())

        for sat_selected in sat_choices:
            sat_budget = COST_SATELLITE_WK if sat_selected else 0.0
```

```
            budget_available = BUDGET_WEEKLY - fixed_camera_cost - sat_budget
            if budget_available <= 0:
                continue
            obj_grad_fn = lambda x, sat=sat_selected: _radius_obj_grad(
                x, W_r, W_d, sat, u_c_base=fixed_u_c_vec,
    optimize_cameras=optimize_cameras
            )

            for i, (fr, fd, fc) in enumerate(starts):
                x0 = np.zeros(total_vars, dtype=np.float64)
                x0[:n] = ranger_scores / (ranger_scores.sum() + 1e-12) * (max_r_aff
    * fr)
                x0[n:2*n] = drone_scores / (drone_scores.sum() + 1e-12) * (max_d_aff
    * fd)
                if optimize_cameras and n_cam_elig:
                    x0[2*n:] = min(1.0, (budget_available * fc) / max(COST_CAMERA *
    n_cam_elig, 1e-12))
                x0 = _project_budget_box(x0, linear_budget, budget_available,
    upper_bounds)

                t0 = time.perf_counter()
                res = _projected_gradient_solve(
                    x0,
                    obj_grad_fn,
                    linear_budget,
                    budget_available,
                    upper_bounds,
                )
                elapsed = time.perf_counter() - t0
                obj = float(res["fun"])
                x_res = res["x"]
                sat_label = "on" if sat_selected else "off"
                status_label = "OK" if res["success"] else f"WARN/{res['status']}"
                cam_sum = x_res[2*n:].sum() if optimize_cameras else
    float(fixed_camera_cost / max(COST_CAMERA, 1e-12))
                if verbose:
                    print(f"  Sat {sat_label} start {i}: L={obj:.6f} {status_label}
    iters={res['nit']} t={elapsed:.2f}s R={x_res[:n].sum():.1f}
    D={x_res[n:2*n].sum():.1f} C={cam_sum:.1f}")
                if obj < best_obj:
                    best_obj = obj
                    best_x = x_res.copy()
                    best_sat = sat_selected

        home_r_c = np.maximum(0.0, best_x[:n])
        drone_c = np.maximum(0.0, best_x[n:2*n])
        sat_selected = bool(best_sat)
        if optimize_cameras:
```

```
            u_c_rel = np.clip(best_x[2*n:], 0.0, 1.0)
        else:
            u_c_rel = None

        N_r_active = int(np.floor(home_r_c.sum() + 1e-9))
        N_d_active = int(np.floor(drone_c.sum() + 1e-9))
        home_r_i = _largest_remainder_round(home_r_c, N_r_active).astype(np.float64)
        drone_i = _largest_remainder_round(drone_c, N_d_active).astype(np.float64)

        while True:
            N_r_employed = int(np.ceil(N_r_active * 7.0 / WORK_DAYS_PER_WEEK))
            N_d_operators = int(np.ceil(N_d_active / DRONES_PER_OPERATOR)) if
    N_d_active > 0 else 0
            spend_base = COST_RANGER * N_r_employed + drone_fleet_cost(N_d_active) +
    fixed_camera_cost + (COST_SATELLITE_WK if sat_selected else 0.0)
            if spend_base <= BUDGET_WEEKLY + 1e-9:
                break
            if sat_selected and fixed_satellite is None:
                sat_selected = False
                continue
            drop_drone_cost = COST_DRONE_FIXED + (COST_DRONE_OPERATOR if (N_d_active
    > 0 and (N_d_active - 1) % DRONES_PER_OPERATOR == 0) else 0.0)
            drop_ranger_cost = COST_RANGER if N_r_active > 0 else -1.0
            if N_d_active > 0 and (N_r_active == 0 or drop_drone_cost >=
    drop_ranger_cost):
                N_d_active -= 1
                drone_i = _largest_remainder_round(drone_c,
    N_d_active).astype(np.float64)
            elif N_r_active > 0:
                N_r_active -= 1
                home_r_i = _largest_remainder_round(home_r_c,
    N_r_active).astype(np.float64)
            else:
                break

        N_r_employed = int(np.ceil(N_r_active * 7.0 / WORK_DAYS_PER_WEEK))
        N_d_operators = int(np.ceil(N_d_active / DRONES_PER_OPERATOR)) if N_d_active
    > 0 else 0
        N_d_employed = N_d_active

        if optimize_cameras:
            budget_left = BUDGET_WEEKLY - COST_RANGER * N_r_employed -
    drone_fleet_cost(N_d_active) - (COST_SATELLITE_WK if sat_selected else 0.0)
            camera_order = np.argsort(-u_c_rel, kind="mergesort")
            N_c = min(n_cam_elig, int(np.floor(max(budget_left, 0.0) / COST_CAMERA)))
            chosen_c = camera_order[:N_c]
            u_c_i = np.zeros(n, dtype=np.float64)
            if N_c > 0:
```

```
                u_c_i[cam_elig_idx[chosen_c]] = 1.0
        else:
            u_c_i = fixed_u_c_vec.copy()
            N_c = int(np.sum(u_c_i > 0.5))

        spend_i = COST_RANGER * N_r_employed + drone_fleet_cost(N_d_active) +
    COST_CAMERA * N_c + (COST_SATELLITE_WK if sat_selected else 0.0)
        v_r_i, v_d_i, assignments = _radius_solution_to_fields(
            home_r_i, drone_i, u_c_i,
            ranger_homes, ranger_cells, ranger_weights, ranger_scores,
            drone_centers_idx, drone_cells, drone_weights, drone_scores,
        )
        p_P, p_F, p_H = compute_protection(v_r_i, v_d_i, u_c_i, sat_selected,
    home_r=home_r_i)
        f_int = total_loss(_w_bar, _tau_P_bar, _tau_F_bar, _tau_H_bar, p_P, p_F, p_H)
        gap = max(0.0, (f_int - best_obj) / (abs(best_obj) + 1e-12))

        return {
            "v_r": v_r_i,
            "v_d": v_d_i,
            "u_c": u_c_i,
            "N_r_employed": N_r_employed,
            "N_r_active": N_r_active,
            "N_d_employed": N_d_employed,
            "N_d_active": N_d_active,
            "N_c": N_c,
            "L_star": float(best_obj),
            "f_int": float(f_int),
            "gap": float(gap),
            "home_r": home_r_i,
            "home_d": drone_i,
            "sat_selected": bool(sat_selected),
            "assignments": assignments,
            "N_d_operators": N_d_operators,
            "spend": float(spend_i),
        }
    finally:
        globals()["_w_bar"] = old_w_bar
        globals()["_tau_P_bar"] = old_tau_P_bar
        globals()["_tau_F_bar"] = old_tau_F_bar
        globals()["_tau_H_bar"] = old_tau_H_bar


def solve_minlp():
    global STATIC_RANGER_HOME, STATIC_DRONE_HOME, SATELLITE_SELECTED,
    STATIC_ASSIGNMENTS, STATIC_DRONE_OPERATORS

    cached = _load_static_solution_cache()
```

```
    if cached is not None:
        return cached

    plan = _solve_radius_plan_core(
        _w_bar.copy(), _tau_P_bar.copy(), _tau_F_bar.copy(), _tau_H_bar.copy(),
        fixed_u_c=None,
        fixed_satellite=None,
        log_header="STATIC ANNUAL RADIUS NLP  --  sparse projected-gradient optimizer",
        verbose=True,
    )

    STATIC_RANGER_HOME = plan["home_r"].copy()
    STATIC_DRONE_HOME = plan["home_d"].copy()
    SATELLITE_SELECTED = plan["sat_selected"]
    STATIC_ASSIGNMENTS = plan["assignments"]
    STATIC_DRONE_OPERATORS = plan["N_d_operators"]

    print(f"\n  Continuous static loss:      {plan['L_star']:.6f}")
    print(f"  Rounded static loss:         {plan['f_int']:.6f}")
    print(f"  Rangers: {plan['N_r_employed']} employed, {plan['N_r_active']} active (radius {RANGER_RADIUS})")
    print(f"  Drones:  {plan['N_d_active']} active, {plan['N_d_operators']} operators (radius {DRONE_RADIUS})")
    print(f"  Cameras: {plan['N_c']}  Satellite selected: {plan['sat_selected']}")
    print(f"  Budget: N${plan['spend']:,.0f} / N${BUDGET_WEEKLY:,.0f}")

    result_tuple = (
        plan["v_r"], plan["v_d"], plan["u_c"],
        plan["N_r_employed"], plan["N_r_active"], plan["N_d_employed"], plan["N_d_active"], plan["N_c"],
        plan["L_star"], plan["f_int"], plan["gap"],
    )
    _export_static_solution_summary(plan["assignments"], ALLOCATION_SUMMARY_PATH)
    _save_static_solution_cache(result_tuple, plan["assignments"], plan["home_r"], plan["home_d"], plan["sat_selected"], plan["N_d_operators"])
    return result_tuple

def step_threats(tau_P, tau_F, tau_H, p_P, p_F, p_H, t):
    fp = FENCE_PERM_ALL[t]
    nP = _threat_step_core(tau_P, poach_entry_source, RHO["P"], MU_0["P"], KAPPA["P"], p_P, s_poach(t), fp, True)
    nF = _threat_step_core(tau_F, fire_entry_source, RHO["F"], MU_0["F"], KAPPA["F"], p_F, s_fire(t), np.ones(n), True)
    nH = _threat_step_core(tau_H, human_entry_source, RHO["H"], MU_0["H"], KAPPA["H"], p_H, s_human(t), 0.80 + 0.20 * human_access_risk, True)
    return nP, nF, nH

```

```
#SPECIES MOVEMENT


K_CELLS = {gname: GROUPS[gname][2]*CELL_KM**2*np.clip(S_i,0.01,1.0)
           for gname in GROUP_NAMES}

ATTR_BASE = np.zeros((G, n), dtype=np.float64)
for gi, gname in enumerate(GROUP_NAMES):
    D_g, r_g, K_max, wh_a, _ = GROUPS[gname]
    a = S_i * np.exp(-wh_a * d_water_km)
    a *= (0.3+0.7*frac_pan) if gname=="waterbird" else (1.0-0.5*frac_pan)
    ATTR_BASE[gi,:] = np.clip(a, 1e-10, None)

HERB_IDX = [GROUP_NAMES.index(g) for g in
   ("large_mig_herb","large_res_herb","small_herb")]
CARN_IDX = [GROUP_NAMES.index(g) for g in ("large_carn","small_carn","raptor")]
FLEE_IDX = set(HERB_IDX)
CARN_SET  = set(CARN_IDX)




_diag_vals = np.ones(n)
_diag_rows = np.arange(n, dtype=np.int32)
_DIAG_ROWS = _diag_rows; _DIAG_COLS = _diag_rows.copy()

_ALL_ROWS = np.concatenate([NB_ROWS, _DIAG_ROWS])
_ALL_COLS = np.concatenate([NB_COLS, _DIAG_COLS])
_D_g_arr  = np.array([GROUPS[g][0] for g in GROUP_NAMES])
_r_g_arr  = np.array([GROUPS[g][1] for g in GROUP_NAMES])


def move_species(N, t, tau_P_cur, W_t):
    """
    Vectorised reaction-diffusion. Sparse matrix rebuilt per step but
    using pre-concatenated index arrays.
    """
    N_new = np.zeros_like(N)
    prey  = N[HERB_IDX,:].sum(axis=0)
    pm    = prey.max()
    flee  = np.exp(-FLEE_ALPHA * tau_P_cur)

    for gi, gname in enumerate(GROUP_NAMES):
        D_g = _D_g_arr[gi]; r_g = _r_g_arr[gi]
        K_i = K_CELLS[gname]
        attr = ATTR_BASE[gi,:] * W_t
        if gi in CARN_SET and pm > 0:
            attr = attr * (1.0 + 0.5 * prey / pm)
```

```
        if gi in FLEE_IDX:
            attr = attr * flee
        attr = np.maximum(attr, 1e-10)


        ead = attr[NB_ROWS]
        sps = np.zeros(n); np.add.at(sps, NB_COLS, ead)
        np.maximum(sps, 1e-10, out=sps)
        off_vals = D_g * ead / sps[NB_COLS]

        diag_vals = 1.0 - D_g


        all_vals = np.concatenate([off_vals, np.full(n, diag_vals)])
        T_m = csr_matrix((all_vals, (_ALL_ROWS, _ALL_COLS)), shape=(n, n))

        N_new[gi,:] = T_m @ N[gi,:]

        Ni = N_new[gi,:]
        Ni += r_g * Ni * (1.0 - Ni / (K_i + 1e-10))
        np.maximum(Ni, 0.0, out=Ni)
    return N_new



_nb_counts = np.array([len(nb) for nb in neighbours], dtype=np.float64)
_nb_counts = np.maximum(_nb_counts, 1.0)
_NB_AVG = csr_matrix(
    (np.ones(len(NB_ROWS)) / _nb_counts[NB_ROWS],
     (NB_ROWS, NB_COLS)), shape=(n, n))
_W_water = np.exp(-ALPHA_W * d_water_km)

def compute_dynamic_wi(N_current):
    """
    Recompute importance weights from current species distribution.
    Multiplicative waterhole: W_i amplifies cells with high species density.
    """
    S_t  = N_current.sum(axis=0)
    sm   = S_t.max()
    S_n  = S_t / sm if sm > 0 else S_t
    C_t  = _NB_AVG @ S_n
    cm   = C_t.max()
    C_t  = C_t / cm if cm > 0 else C_t
    Psi_n = Psi_i / (Psi_i.max() + 1e-10) if Psi_i.max() > 0 else Psi_i
    w_t  = (S_n * (1.0 + W_WATER * _W_water)
           + W_CONN * C_t
           + W_FIRE * Psi_n)
    mx   = w_t.max()
```

```
    return w_t / mx if mx > 0 else w_t

#CALIBRATION

print("\nCalibrating lambda (deterrence)...")
n_rangers_uniform = int(BUDGET_WEEKLY / max(ACTIVE_RANGER_COST, 1e-12))
cal_ranger_homes, cal_ranger_cells, cal_ranger_weights, _, _, _, _, _ =
    _build_static_candidate_clusters(
    _w_bar, _tau_P_bar, _tau_F_bar, _tau_H_bar
)
home_r_cal = np.zeros(n, dtype=np.float64)
allowed_ranger_count = int(ranger_home_allowed.sum())
if allowed_ranger_count > 0:
    home_r_cal[ranger_home_allowed] = n_rangers_uniform / allowed_ranger_count
v_r_cal = np.zeros(n, dtype=np.float64)
for idx, mass in enumerate(home_r_cal):
    if mass <= 0:
        continue
    v_r_cal[cal_ranger_cells[idx]] += mass * cal_ranger_weights[idx]
TARGET_DETERRENCE = 0.35

def _weighted_mean(arr):
    return float((w_i * arr).sum() / (w_i.sum() + 1e-12))

def _delta_bar(lam):
    return _weighted_mean(1.0 - np.exp(-lam * v_r_cal))

delta_bar = _delta_bar(LAMBDA_DETER)
p_P_cal, _, _ = compute_protection(v_r_cal, np.zeros(n), np.zeros(n),
    use_satellite=False, home_r=home_r_cal)
p_bar = _weighted_mean(p_P_cal)
print(f"  Uniform {n_rangers_uniform} active rangers over {allowed_ranger_count}
    allowed homes -> mean Delta = {delta_bar:.3f} (target {TARGET_DETERRENCE:.2f})")
print(f"  Implied mean p_P under current model = {p_bar:.3f}")
if abs(delta_bar - TARGET_DETERRENCE) > 0.01:
    from scipy.optimize import brentq
    def _delta_root(lam):
        return _delta_bar(lam) - TARGET_DETERRENCE
    try:
        lo, hi = 1e-6, 1.0
        flo = _delta_root(lo)
        fhi = _delta_root(hi)
        while flo * fhi > 0 and hi < 1e6:
            hi *= 2.0
            fhi = _delta_root(hi)
        if flo * fhi <= 0:
            LAMBDA_DETER = brentq(_delta_root, lo, hi)
            delta_bar = _delta_bar(LAMBDA_DETER)
```

```
            p_P_cal, _, _ = compute_protection(v_r_cal, np.zeros(n), np.zeros(n),
    use_satellite=False, home_r=home_r_cal)
            p_bar = _weighted_mean(p_P_cal)
            print(f"  Recalibrated lambda = {LAMBDA_DETER:.4f} -> mean Delta =
    {delta_bar:.3f}")
            print(f"  Implied mean p_P after deterrence calibration = {p_bar:.3f}")
        else:
            print(f"  Warning: deterrence target not bracketed, keeping lambda =
    {LAMBDA_DETER:.4f}")
    except Exception:
        print(f"  Warning: deterrence calibration failed, keeping lambda =
    {LAMBDA_DETER:.4f}")

#OPTIMISE RESOURCE COUNT - PHASE 1

(v_r_opt, v_d_opt, u_c_opt,
    N_r_employed, N_r_active, N_d_employed, N_d_active, N_c,
 L_star_0, f_int_0, gap_0) = solve_minlp()

u_c_fixed = u_c_opt.copy()
v_r_fixed = v_r_opt.copy()
v_d_fixed = v_d_opt.copy()
u_c_fixed = u_c_opt.copy()


#RUN SCENARIOS

def _solve_mobile_replan(t_start, tau_P_cur, tau_F_cur, tau_H_cur, w_cur, fixed_u_c,
    fixed_satellite, verbose=True):
    w_bar_plan, tau_P_bar_plan, tau_F_bar_plan, tau_H_bar_plan =
    _discounted_average_fields(
        t_start, tau_P_cur, tau_F_cur, tau_H_cur, w_cur
    )
    return _solve_radius_plan_core(
        w_bar_plan, tau_P_bar_plan, tau_F_bar_plan, tau_H_bar_plan,
        fixed_u_c=fixed_u_c,
        fixed_satellite=fixed_satellite,
        log_header=f"ROLLING BIWEEKLY REPLAN  --  starting week {t_start + 1:02d}",
        verbose=verbose,
    )


def run_scenario(label, verbose=True, show_progress=True):
    if verbose:
        print(f"\n{'='*65}\nRUNNING {label}\n{'='*65}")

    tau_P = tau_P0.copy(); tau_F = tau_F0.copy(); tau_H = tau_H0.copy()
    N_    = N0.copy();     w_    = w_i.copy()
```

```
    def _plan_dict(v_r, v_d, u_c, home_r, home_d, use_sat, n_r_employed, n_r_active,
n_d_employed, n_d_active, n_c, n_d_ops, spend):
        return {
            "v_r": np.asarray(v_r, dtype=np.float64).copy(),
            "v_d": np.asarray(v_d, dtype=np.float64).copy(),
            "u_c": np.asarray(u_c, dtype=np.float64).copy(),
            "home_r": np.asarray(home_r, dtype=np.float64).copy(),
            "home_d": np.asarray(home_d, dtype=np.float64).copy(),
            "sat_selected": bool(use_sat),
            "N_r_employed": int(n_r_employed),
            "N_r_active": int(n_r_active),
            "N_d_employed": int(n_d_employed),
            "N_d_active": int(n_d_active),
            "N_c": int(n_c),
            "N_d_operators": int(n_d_ops),
            "spend": float(spend),
        }

    if label == "S0":
        current_plan = _plan_dict(
            np.zeros(n, dtype=np.float64),
            np.zeros(n, dtype=np.float64),
            np.zeros(n, dtype=np.float64),
            np.zeros(n, dtype=np.float64),
            np.zeros(n, dtype=np.float64),
            False, 0, 0, 0, 0, 0, 0, 0.0
        )
        if verbose:
            print("  S0: no intervention, no active rangers, no drones, no cameras,
no satellite")
    elif label == "S2":
        current_plan = _solve_mobile_replan(0, tau_P, tau_F, tau_H, w_,
u_c_fixed.copy(), bool(SATELLITE_SELECTED), verbose=verbose)
        if verbose:
            print(f"  S2: biweekly redeployment with fixed cameras
({int(u_c_fixed.sum())}) and satellite={int(SATELLITE_SELECTED)}")
            print(f"  Initial S2: R={current_plan['N_r_active']} active
({current_plan['N_r_employed']} employed)  D={current_plan['N_d_active']} active
({current_plan['N_d_operators']} operators)")
    else:
        current_plan = _plan_dict(
            v_r_fixed.copy(),
            v_d_fixed.copy(),
            u_c_fixed.copy(),
            STATIC_RANGER_HOME.copy() if STATIC_RANGER_HOME is not None else
np.zeros(n, dtype=np.float64),
            STATIC_DRONE_HOME.copy() if STATIC_DRONE_HOME is not None else
```

```
    np.zeros(n, dtype=np.float64),
1817            SATELLITE_SELECTED,
1818            N_r_employed, N_r_active, N_d_employed, N_d_active, N_c,
    STATIC_DRONE_OPERATORS,
1819            COST_RANGER * N_r_employed + drone_fleet_cost(N_d_active) + COST_CAMERA
    * u_c_fixed.sum() + (COST_SATELLITE_WK if SATELLITE_SELECTED else 0.0),
1820        )
1821        p_P3, p_F3, p_H3 = compute_protection(current_plan["v_r"],
    current_plan["v_d"], current_plan["u_c"], current_plan["sat_selected"],
    home_r=current_plan["home_r"])
1822        home_r0 = np.zeros(n, dtype=np.float64)
1823        v_r0 = np.zeros(n, dtype=np.float64)
1824        p_P0, p_F0, p_H0 = compute_protection(v_r0, np.zeros(n), np.zeros(n), False,
    home_r=home_r0)
1825        L3 = total_loss(w_i, tau_P0, tau_F0, tau_H0, p_P3, p_F3, p_H3)
1826        L0 = total_loss(w_i, tau_P0, tau_F0, tau_H0, p_P0, p_F0, p_H0)
1827        if verbose:
1828            print(f"  S1: R={N_r_active} active ({N_r_employed} employed)
    D={N_d_active} active ({STATIC_DRONE_OPERATORS} operators)  C={N_c}")
1829            print(f"  t=0 check: S1={L3:.4f}  S0={L0:.4f}"
1830                  f"  ({'OK: S1<S0' if L3 < L0 else 'BUG: S1>=S0'})")
1831
1832    hist = {k: [] for k in [
1833        "loss","P_park","N_total","budget",
1834        "loss_P","loss_F","loss_H","fence_km",
1835        "tau_P_mean","tau_F_mean","protected_total",
1836        "rangers_active","drones_active","cameras","satellite",
1837    ]}
1838    snapshots = []
1839
1840    t0_time = time.time()
1841
1842    for t in range(T):
1843        v_r = current_plan["v_r"]
1844        v_d = current_plan["v_d"]
1845        u_c = current_plan["u_c"]
1846        home_r = current_plan["home_r"]
1847        home_d = current_plan["home_d"]
1848        use_sat = current_plan["sat_selected"]
1849        N_r_total = current_plan["N_r_active"]
1850        N_d_total = current_plan["N_d_active"]
1851        current_budget = current_plan["spend"]
1852
1853        p_P, p_F, p_H = compute_protection(v_r, v_d, u_c, use_sat, home_r=home_r)
1854        h_r, h_d = compute_effort(v_r, v_d)
1855        W_t = W_seasonal[t]
1856        Lp, Lf, Lh = loss_components(w_, tau_P, tau_F, tau_H, p_P, p_F, p_H)
1857        protected_total = protected_fraction_total(w_, tau_P, tau_F, tau_H, p_P,
```

```
    p_F, p_H)

        hist["loss"].append(Lp + Lf + Lh)
        hist["P_park"].append((w_ * (p_P + p_F + p_H) / 3).sum() / w_.sum())
        hist["N_total"].append(float(N_.sum()))
        hist["budget"].append(current_budget)
        hist["loss_P"].append(Lp); hist["loss_F"].append(Lf);
    hist["loss_H"].append(Lh)
        hist["fence_km"].append(min(FENCE_BUILT_KM + FENCE_RATE_KM_WK * t,
    FENCE_TOTAL_KM))
        hist["tau_P_mean"].append(tau_P.mean());
    hist["tau_F_mean"].append(tau_F.mean())
        hist["protected_total"].append(protected_total)
        hist["rangers_active"].append(current_plan["N_r_active"])
        hist["drones_active"].append(current_plan["N_d_active"])
        hist["cameras"].append(current_plan["N_c"])
        hist["satellite"].append(int(use_sat))

        snapshots.append({
            "t": t,
            "v_r": v_r.copy(),
            "v_d": v_d.copy(),
            "h_r": h_r.copy(),
            "h_d": h_d.copy(),
            "home_r": home_r.copy(),
            "home_d": home_d.copy(),
            "p_P": p_P.copy(),
            "tau_P": tau_P.copy(),
            "tau_F": tau_F.copy(),
            "tau_H": tau_H.copy(),
            "u_c": u_c.copy(),
            "sat_selected": bool(use_sat),
            "budget": float(current_budget),
        })

        elapsed = time.time() - t0_time
        eta = elapsed / (t + 1) * (T - t - 1) if t > 0 else 0
        if show_progress:
            progress_bar(
                t + 1, T,
                f"L={hist['loss'][-1]:.3f} P={hist['P_park'][-1]:.2f}
    Prot={hist['protected_total'][-1]:.2f}"
                f" R={N_r_total}(rad{RANGER_RADIUS})
    D={N_d_total}(rad{DRONE_RADIUS}) ETA:{eta:.0f}s"
            )

        tau_P, tau_F, tau_H = step_threats(tau_P, tau_F, tau_H, p_P, p_F, p_H, t)
        N_ = move_species(N_, t, tau_P, W_t)
```

```
        _poach_loss_base = PoachValue_i * (1.0 - p_P) * 0.05
        for gi, gname in enumerate(GROUP_NAMES):
            if GROUPS[gname][4]:
                N_[gi, :] = np.maximum(0, N_[gi, :] * (1 - _poach_loss_base * tau_P))
        w_ = compute_dynamic_wi(N_)

        if label == "S2" and (t + 1) < T and (t + 1) % REPLAN_INTERVAL_WEEKS == 0:
            current_plan = _solve_mobile_replan(t + 1, tau_P, tau_F, tau_H, w_,
    u_c_fixed.copy(), bool(SATELLITE_SELECTED), verbose=verbose)

    if verbose:
        print(f"\n  OK Done  Mean Loss:{np.mean(hist['loss']):.4f}"
              f"  Mean P:{np.mean(hist['P_park']):.4f}"
              f"  Time:{time.time() - t0_time:.1f}s")
    return hist, snapshots


results = {}; snaps = {}
results["S0"], snaps["S0"] = run_scenario("S0")
results["S1"], snaps["S1"] = run_scenario("S1")
results["S2"], snaps["S2"] = run_scenario("S2")

weeks = np.arange(T)
df_ts = pd.DataFrame({"week": weeks})
for sc, h in results.items():
    for k, v in h.items():
        df_ts[f"{sc}_{k}"] = v
df_ts.to_csv("dynamic_timeseries.csv", index=False)

disc   = lambda h: sum(GAMMA**t * h["loss"][t] for t in range(T))
L_S0   = disc(results["S0"]); L_S1 = disc(results["S1"]); L_S2 = disc(results["S2"])
improv_S1 = (L_S0-L_S1)/max(L_S0, 1e-12)*100
improv_S2 = (L_S0-L_S2)/max(L_S0, 1e-12)*100
gain_S2_over_S1 = (L_S1-L_S2)/max(L_S1, 1e-12)*100
print(f"\nS0={L_S0:.4f}  S1={L_S1:.4f}  S2={L_S2:.4f}"
      f"  S1 improvement={improv_S1:.1f}%  S2 improvement={improv_S2:.1f}%"
      f"  S2 vs S1={gain_S2_over_S1:.1f}%  Gap={gap_0:.4%}")

#MONTE CARLO UNCERTAINTY
mc_rows = []
for seed in range(MONTE_CARLO_SEED0, MONTE_CARLO_SEED0 + MONTE_CARLO_RUNS):
    for scenario in (["S0", "S1", "S2"] if MONTE_CARLO_INCLUDE_DYNAMIC else ["S0",
    "S1"]):
        RNG = np.random.default_rng(seed)
        hist_mc, _ = run_scenario(scenario, verbose=False, show_progress=False)
        L_mc = sum(GAMMA**t * hist_mc["loss"][t] for t in range(T))
        mc_rows.append({
            "seed": int(seed),
```

```
            "scenario": scenario,
            "discounted_loss": float(L_mc),
            "mean_loss": float(np.mean(hist_mc["loss"])),
            "mean_protected_share": float(np.mean(hist_mc["protected_total"])),
        })

mc_df = pd.DataFrame(mc_rows)
mc_summary = mc_df.groupby("scenario", as_index=False).agg(
    discounted_loss_mean=("discounted_loss", "mean"),
    discounted_loss_sd=("discounted_loss", "std"),
    discounted_loss_p05=("discounted_loss", lambda x: float(np.quantile(x, 0.05))),
    discounted_loss_p95=("discounted_loss", lambda x: float(np.quantile(x, 0.95))),
    mean_loss_mean=("mean_loss", "mean"),
    protected_share_mean=("mean_protected_share", "mean"),
    protected_share_sd=("mean_protected_share", "std"),
)
mc_summary["discounted_loss_ci95"] = 1.96 * mc_summary["discounted_loss_sd"] /
    np.sqrt(max(MONTE_CARLO_RUNS, 1))
mc_summary["protected_share_ci95"] = 1.96 * mc_summary["protected_share_sd"] /
    np.sqrt(max(MONTE_CARLO_RUNS, 1))
mc_paired = mc_df.pivot(index="seed", columns="scenario",
    values="discounted_loss").dropna().reset_index()
mc_paired["improvement_abs"] = mc_paired["S0"] - mc_paired["S1"]
mc_paired["improvement_pct"] = 100.0 * mc_paired["improvement_abs"] /
    np.maximum(mc_paired["S0"], 1e-12)
mc_mean_improvement = float(mc_paired["improvement_pct"].mean())
mc_p05_improvement = float(np.quantile(mc_paired["improvement_pct"], 0.05))
mc_p95_improvement = float(np.quantile(mc_paired["improvement_pct"], 0.95))
mc_win_rate = float(np.mean(mc_paired["S1"] < mc_paired["S0"]))
mc_meaningful_win_rate = float(np.mean(mc_paired["improvement_pct"] > 1.0))
if "S2" in mc_paired.columns:
    mc_paired["improvement_pct_S2"] = 100.0 * (mc_paired["S0"] - mc_paired["S2"]) /
    np.maximum(mc_paired["S0"], 1e-12)
    mc_s2_mean_improvement = float(mc_paired["improvement_pct_S2"].mean())
    mc_s2_win_rate = float(np.mean(mc_paired["S2"] < mc_paired["S0"]))
else:
    mc_s2_mean_improvement = np.nan
    mc_s2_win_rate = np.nan
mc_df.to_csv("monte_carlo_uncertainty_raw.csv", index=False)
mc_summary.to_csv("monte_carlo_uncertainty_summary.csv", index=False)

#VISUALISATION

from matplotlib.colors import LinearSegmentedColormap
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D

gp_geo       = grid.copy().to_crs(CRS_GEO)
```

```
cell_lons_geo = np.array([row.geometry.centroid.x for _, row in gp_geo.iterrows()])
cell_lats_geo = np.array([row.geometry.centroid.y for _, row in gp_geo.iterrows()])


base_lons_geo, base_lats_geo = [], []
for b in range(n_bases):
    pt     = camps_proj.geometry.iloc[b]
    pt_geo = gpd.GeoSeries([pt], crs=CRS_PROJ).to_crs(CRS_GEO).iloc[0]
    base_lons_geo.append(pt_geo.x); base_lats_geo.append(pt_geo.y)
base_lons_geo = np.array(base_lons_geo)
base_lats_geo = np.array(base_lats_geo)

wh_high = wh_proj[wh_proj["quality"] == 1.0]
wh_med  = wh_proj[wh_proj["quality"] == 0.5]


cmap_prot   = LinearSegmentedColormap.from_list("prot",
    ["#b71c1c","#f44336","#ff9800","#ffeb3b","#66bb6a","#1b5e20"])
cmap_threat = LinearSegmentedColormap.from_list("threat",
    ["#e8f5e9","#ffcc02","#ff6f00","#b71c1c","#4a0000"])
cmap_cover  = LinearSegmentedColormap.from_list("cover",
    ["#ffffff","#bbdefb","#1565c0","#0d47a1"])
cmap_diff   = LinearSegmentedColormap.from_list("diff",
    ["#b71c1c","#ef5350","#ffcdd2","#f5f5f5","#c8e6c9","#388e3c","#1b5e20"])


def _add_park_boundary(ax):
    etosha_geo.boundary.plot(ax=ax, color="black", linewidth=1.0, zorder=5)


def _plot_cell_heatmap(ax, vals, cmap, vmin=None, vmax=None,
                       edgecolor="#f2f2f2", linewidth=0.10, alpha=1.0):
    vals = np.asarray(vals, dtype=np.float64)
    vmin = float(np.nanmin(vals)) if vmin is None else vmin
    vmax = float(np.nanmax(vals)) if vmax is None else vmax
    plot_gdf = gp_geo[["geometry"]].copy()
    plot_gdf["_val"] = vals
    plot_gdf.plot(
        ax=ax, column="_val", cmap=cmap, vmin=vmin, vmax=vmax,
        linewidth=linewidth, edgecolor=edgecolor, alpha=alpha, zorder=2
    )
    sm = ScalarMappable(norm=Normalize(vmin=vmin, vmax=vmax), cmap=cmap)
    sm.set_array([])
    return sm


def _add_cell_overlay(ax, mask, facecolor, edgecolor=None,
                      alpha=0.35, linewidth=0.45, zorder=6):
```

```
    mask = np.asarray(mask, dtype=bool)
    if not mask.any():
        return
    gp_geo.loc[mask].plot(
        ax=ax,
        facecolor=facecolor,
        edgecolor=edgecolor if edgecolor is not None else facecolor,
        linewidth=linewidth,
        alpha=alpha,
        zorder=zorder,
    )


def _add_intensity_overlay(ax, vals, cmap, edgecolor,
                           alpha=0.65, linewidth=0.55, zorder=7):
    vals = np.asarray(vals, dtype=np.float64)
    mask = vals > 1e-9
    if not mask.any():
        return
    sub = gp_geo.loc[mask, ["geometry"]].copy()
    sub["_val"] = vals[mask]
    sub.plot(
        ax=ax,
        column="_val",
        cmap=cmap,
        linewidth=linewidth,
        edgecolor=edgecolor,
        alpha=alpha,
        zorder=zorder,
    )


def _mark_home_cells(ax, vals, edgecolor, marker, size=34, zorder=9):
    vals = np.asarray(vals, dtype=np.float64)
    mask = vals > 1e-9
    if not mask.any():
        return
    ax.scatter(
        cell_lons_geo[mask],
        cell_lats_geo[mask],
        s=size,
        marker=marker,
        facecolors="none",
        edgecolors=edgecolor,
        linewidths=1.0,
        zorder=zorder,
    )

```

```
2085
2086 def _add_waterholes(ax):
2087     if len(wh_high):
2088         wh_high.to_crs(CRS_GEO).plot(ax=ax, color="#1565c0", markersize=14,
2089             marker="o", zorder=6, edgecolors="white", linewidths=0.4)
2090     if len(wh_med):
2091         wh_med.to_crs(CRS_GEO).plot(ax=ax, color="#64b5f6", markersize=7,
2092             marker="o", zorder=6, edgecolors="none")
2093
2094
2095 def _style(ax, title, season_col="#333333"):
2096     ax.set_title(title, fontsize=10, fontweight="bold", pad=5)
2097     ax.set_xlabel("Longitude", fontsize=8)
2098     ax.set_ylabel("Latitude",  fontsize=8)
2099     ax.tick_params(labelsize=7)
2100     ax.set_aspect("equal")
2101     for spine in ax.spines.values():
2102         spine.set_edgecolor(season_col); spine.set_linewidth(1.5)
2103
2104
2105
2106
2107 def _save_context_map(filename, vals, cmap, title, cbar_label,
2108                       vmin=None, vmax=None, note=None):
2109     fig, ax = plt.subplots(figsize=(10.5, 5.8))
2110     sm = _plot_cell_heatmap(ax, vals, cmap, vmin=vmin, vmax=vmax,
2111                             edgecolor="#f4f4f4", linewidth=0.10, alpha=1.0)
2112     _add_park_boundary(ax)
2113     _style(ax, title)
2114     cbar = plt.colorbar(sm, ax=ax, shrink=0.86, pad=0.02)
2115     cbar.set_label(cbar_label, fontsize=9)
2116     cbar.ax.tick_params(labelsize=8)
2117     if note:
2118         ax.text(0.02, 0.98, note, transform=ax.transAxes, va="top", fontsize=8,
2119                 bbox=dict(facecolor="white", edgecolor="#cccccc",
    boxstyle="round,pad=0.35"))
2120     plt.tight_layout()
2121     plt.savefig(filename, dpi=170, bbox_inches="tight")
2122     plt.close()
2123     print(f"  Saved {filename}")
2124
2125 #MAP 1: PATROL ALLOCATION
2126
2127 print("Generating patrol allocation map...")
2128 fig1, axes1 = plt.subplots(1, 3, figsize=(30, 8))
2129 fig1.suptitle("Patrol Effort Footprint and Deployment Homes",
2130               fontsize=13, fontweight="semibold")
2131 ranger_home_cmap = LinearSegmentedColormap.from_list("r_home", ["#fce4ec",
```

```
    "#ec407a", "#880e4f"])
drone_home_cmap = LinearSegmentedColormap.from_list("d_home", ["#ede7f6", "#7e57c2",
    "#311b92"])
patrol_vmax = max(snaps["S0"][0]["v_r"].max(), snaps["S1"][0]["v_r"].max(),
    snaps["S2"][0]["v_r"].max(), 1e-9)

for ax, sc, v_r, home_r_map, home_d_map, u_c_map, sat_flag in [
    (axes1[0], "S0", snaps["S0"][0]["v_r"], snaps["S0"][0]["home_r"],
    snaps["S0"][0]["home_d"], snaps["S0"][0]["u_c"], snaps["S0"][0]["sat_selected"]),
    (axes1[1], "S1", snaps["S1"][0]["v_r"], snaps["S1"][0]["home_r"],
    snaps["S1"][0]["home_d"], snaps["S1"][0]["u_c"], snaps["S1"][0]["sat_selected"]),
    (axes1[2], "S2", snaps["S2"][0]["v_r"], snaps["S2"][0]["home_r"],
    snaps["S2"][0]["home_d"], snaps["S2"][0]["u_c"], snaps["S2"][0]["sat_selected"]),
]:
    sc2 = _plot_cell_heatmap(ax, v_r, cmap_cover, vmin=0, vmax=patrol_vmax)
    _add_park_boundary(ax)
    if sat_flag:
        _add_cell_overlay(ax, sat_coverage > 0, facecolor="#80deea",
    edgecolor="none", alpha=0.12, linewidth=0.0, zorder=3)
    _add_waterholes(ax)
    if sc != "S0":
        _add_intensity_overlay(ax, home_r_map, ranger_home_cmap,
    edgecolor="#ad1457", alpha=0.55)
        _add_intensity_overlay(ax, home_d_map, drone_home_cmap, edgecolor="#4527a0",
    alpha=0.48)
        _mark_home_cells(ax, home_r_map, edgecolor="#4a001f", marker="s", size=44)
        _mark_home_cells(ax, home_d_map, edgecolor="#1a237e", marker="o", size=34)
        _add_cell_overlay(ax, u_c_map > 0.5, facecolor="#26c6da",
    edgecolor="#00838f", alpha=0.78, linewidth=0.5, zorder=8)
    else:
        ax.text(0.02, 0.97, "No deployed resources\nunder the no-intervention case",
                transform=ax.transAxes, fontsize=8, va="top",
                bbox=dict(facecolor="#ffffff", edgecolor="#bbbbbb",
    boxstyle="round,pad=0.3"))
    plt.colorbar(sc2, ax=ax, label="Ranger patrol intensity", shrink=0.72)
    panel_title = {
        "S0": "No-intervention patrol effort",
        "S1": "Static optimized patrol effort",
        "S2": "Biweekly redeployed patrol effort",
    }[sc]
    _style(ax, panel_title)

handles = [
    mpatches.Patch(facecolor="#ec407a", edgecolor="#ad1457", alpha=0.55,
    label="Ranger home cells"),
    mpatches.Patch(facecolor="#7e57c2", edgecolor="#4527a0", alpha=0.50,
    label="Drone home cells"),
    Line2D([0],[0], marker="s", color="#4a001f", markerfacecolor="none",
```

```
            markersize=7, linewidth=0, label="Ranger home marker"),
    Line2D([0],[0], marker="o", color="#1a237e", markerfacecolor="none",
            markersize=7, linewidth=0, label="Drone home marker"),
    mpatches.Patch(facecolor="#26c6da", edgecolor="#00838f", alpha=0.78,
    label="Camera cells"),
    mpatches.Patch(facecolor="#80deea", edgecolor="none", alpha=0.22,
    label="Satellite footprint"),
    Line2D([0],[0], marker="o",  color="w", markerfacecolor="#1565c0",
            markersize=8,  label="Managed waterhole"),
]
fig1.legend(handles=handles, loc="lower center", bbox_to_anchor=(0.5, -0.01),
            ncol=4, fontsize=7.5, framealpha=0.94)
plt.subplots_adjust(bottom=0.16, wspace=0.18)
plt.savefig("map_patrol_allocation.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_patrol_allocation.png")


# MAP 2: PROTECTION  +  RESIDUAL THREAT  (S0 vs S1)
print("Generating protection and threat maps...")

p_P_s0, p_F_s0, p_H_s0 = compute_protection(
    snaps["S0"][0]["v_r"], snaps["S0"][0]["v_d"], np.zeros(n), False,
    home_r=snaps["S0"][0]["home_r"])
p_P_S1, p_F_S1, p_H_S1 = compute_protection(
    snaps["S1"][0]["v_r"], snaps["S1"][0]["v_d"], snaps["S1"][0]["u_c"],
    snaps["S1"][0]["sat_selected"], home_r=snaps["S1"][0]["home_r"])
p_P_S2, p_F_S2, p_H_S2 = compute_protection(
    snaps["S2"][0]["v_r"], snaps["S2"][0]["v_d"], snaps["S2"][0]["u_c"],
    snaps["S2"][0]["sat_selected"], home_r=snaps["S2"][0]["home_r"])

ranger_cover_mask = v_r_opt > 1e-9
drone_cover_mask = v_d_opt > 1e-9
camera_cover_mask = u_c_opt > 0.5
sat_cover_mask = (sat_coverage > 0) & bool(SATELLITE_SELECTED)
combined_cover_mask = ranger_cover_mask | drone_cover_mask | camera_cover_mask |
    sat_cover_mask
coverage_cells = int(combined_cover_mask.sum())
coverage_frac = coverage_cells / max(n, 1)
coverage_weight_frac = float(w_i[combined_cover_mask].sum() / (w_i.sum() + 1e-12))
loss_S1_t0 = w_i * (tau_P0 * (1.0 - p_P_S1) + tau_F0 * (1.0 - p_F_S1) + tau_H0 *
    (1.0 - p_H_S1))
loss_S2_t0 = w_i * (tau_P0 * (1.0 - p_P_S2) + tau_F0 * (1.0 - p_F_S2) + tau_H0 *
    (1.0 - p_H_S2))
loss_s0_t0 = w_i * (tau_P0 * (1.0 - p_P_s0) + tau_F0 * (1.0 - p_F_s0) + tau_H0 *
    (1.0 - p_H_s0))
loss_t0_scale = max(float(loss_s0_t0.max()), float(loss_S1_t0.max()),
    float(loss_S2_t0.max()), 1e-12)
```

```
loss_t0_norm = loss_S1_t0 / loss_t0_scale
L0p_t0, L0f_t0, L0h_t0 = loss_components(w_i, tau_P0, tau_F0, tau_H0, p_P_s0, p_F_s0, p_H_s0)
L3p_t0, L3f_t0, L3h_t0 = loss_components(w_i, tau_P0, tau_F0, tau_H0, p_P_S1, p_F_S1, p_H_S1)
L2p_t0, L2f_t0, L2h_t0 = loss_components(w_i, tau_P0, tau_F0, tau_H0, p_P_S2, p_F_S2, p_H_S2)
P0p_t0, P0f_t0, P0h_t0 = protected_components(w_i, tau_P0, tau_F0, tau_H0, p_P_s0, p_F_s0, p_H_s0)
P3p_t0, P3f_t0, P3h_t0 = protected_components(w_i, tau_P0, tau_F0, tau_H0, p_P_S1, p_F_S1, p_H_S1)
P2p_t0, P2f_t0, P2h_t0 = protected_components(w_i, tau_P0, tau_F0, tau_H0, p_P_S2, p_F_S2, p_H_S2)
protected_s0_t0 = protected_fraction_total(w_i, tau_P0, tau_F0, tau_H0, p_P_s0, p_F_s0, p_H_s0)
protected_S1_t0 = protected_fraction_total(w_i, tau_P0, tau_F0, tau_H0, p_P_S1, p_F_S1, p_H_S1)
protected_S2_t0 = protected_fraction_total(w_i, tau_P0, tau_F0, tau_H0, p_P_S2, p_F_S2, p_H_S2)
L0_t0 = L0p_t0 + L0f_t0 + L0h_t0
L3_t0 = L3p_t0 + L3f_t0 + L3h_t0
L2_t0 = L2p_t0 + L2f_t0 + L2h_t0


tau_P_end_s0 = np.array(results["S0"]["tau_P_mean"])
tau_P_end_S1 = np.array(results["S1"]["tau_P_mean"])
tau_P_end_S2 = np.array(results["S2"]["tau_P_mean"])


tau_P_s0_spatial = snaps["S0"][-1]["tau_P"]
tau_P_S1_spatial = snaps["S1"][-1]["tau_P"]
tau_P_S2_spatial = snaps["S2"][-1]["tau_P"]

prot_comp_max = max(float(np.max(p_P_s0)), float(np.max(p_P_S1)), float(np.max(p_P_S2)), 1e-12)
tau_comp_max = max(float(np.max(tau_P_s0_spatial)), float(np.max(tau_P_S1_spatial)), float(np.max(tau_P_S2_spatial)), 1e-12)

fig2, axeS1 = plt.subplots(2, 3, figsize=(24, 12))
fig2.suptitle("Protection and Residual Poaching Threat",
              fontsize=13, fontweight="semibold")

panels = [
    (axeS1[0,0], p_P_s0 / prot_comp_max,           cmap_prot,   0, 1, "No-intervention protection"),
    (axeS1[0,1], p_P_S1 / prot_comp_max,           cmap_prot,   0, 1, "Static optimized protection"),
    (axeS1[0,2], p_P_S2 / prot_comp_max,           cmap_prot,   0, 1, "Biweekly
```

```
    redeployed protection"),
    (axeS1[1,0], tau_P_s0_spatial / tau_comp_max, cmap_threat, 0, 1,
    "No-intervention poaching threat at year end"),
    (axeS1[1,1], tau_P_S1_spatial / tau_comp_max, cmap_threat, 0, 1, "Static
    optimized poaching threat at year end"),
    (axeS1[1,2], tau_P_S2_spatial / tau_comp_max, cmap_threat, 0, 1, "Biweekly
    redeployed poaching threat at year end"),
]
for ax, vals, cmap, vmin, vmax, title in panels:
    sc2 = _plot_cell_heatmap(ax, vals, cmap, vmin, vmax)
    _add_park_boundary(ax)
    _add_waterholes(ax)
    plt.colorbar(sc2, ax=ax, shrink=0.6)
    _style(ax, title)

plt.tight_layout()
plt.savefig("map_protection_threat.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_protection_threat.png")


#MAP 3: DRONE COVERAGE RADIUS HEATMAP

print("Generating drone coverage map...")
h_d_S1 = v_d_opt.copy()

fig3, ax3 = plt.subplots(figsize=(12, 6))
sc2 = _plot_cell_heatmap(ax3, h_d_S1,
    LinearSegmentedColormap.from_list("drone",
        ["#ffffff","#e8eaf6","#7986cb","#3949ab","#1a237e"]),
    vmin=0)
_add_park_boundary(ax3)
_add_waterholes(ax3)
_add_intensity_overlay(ax3, snaps["S1"][0]["home_d"], drone_home_cmap,
    edgecolor="#4527a0", alpha=0.55)
_mark_home_cells(ax3, snaps["S1"][0]["home_d"], edgecolor="#1a237e", marker="o",
    size=34)
plt.colorbar(sc2, ax=ax3, label="Drone surveillance intensity", shrink=0.6)
_style(ax3, "Optimized drone surveillance")
handles_d = [
    mpatches.Patch(facecolor="#7e57c2", edgecolor="#4527a0", alpha=0.55,
                   label="Drone home cells"),
    Line2D([0],[0], marker="o", color="#1a237e", markerfacecolor="none",
           markersize=7, linewidth=0, label="Drone home marker"),
    Line2D([0],[0], marker="o", color="w", markerfacecolor="#1565c0",
           markersize=8, label="Managed waterhole"),
]
fig3.legend(handles=handles_d, loc="lower center", bbox_to_anchor=(0.5, -0.01),
```

```
            ncol=3, fontsize=8, framealpha=0.92)
plt.subplots_adjust(bottom=0.18, right=0.88)
plt.savefig("map_drone_coverage.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_drone_coverage.png")


print("Generating satellite coverage map...")
fig_sat, ax_sat = plt.subplots(figsize=(12, 6))
sat_sm = _plot_cell_heatmap(
    ax_sat,
    sat_coverage,
    LinearSegmentedColormap.from_list("sat_cov", ["#f7fcfd", "#80deea", "#00838f"]),
    vmin=0, vmax=1
)
_add_park_boundary(ax_sat)
_style(ax_sat, "Satellite coverage footprint")
plt.colorbar(sat_sm, ax=ax_sat, label="Satellite coverage", shrink=0.72)
plt.savefig("map_satellite_coverage.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_satellite_coverage.png")


print("Generating resource deployment plate...")
fig_plate = plt.figure(figsize=(24, 13))
gs_plate = gridspec.GridSpec(
    4, 4, figure=fig_plate,
    width_ratios=[1.5, 1.5, 1.0, 1.0],
    hspace=0.24, wspace=0.22,
    left=0.04, right=0.96, top=0.93, bottom=0.10
)
fig_plate.suptitle("Optimized Resource Deployment Summary", fontsize=14,
   fontweight="semibold")

deployment_field = np.maximum.reduce([
    snaps["S1"][0]["v_r"] / max(float(np.max(snaps["S1"][0]["v_r"])), 1e-12),
    snaps["S1"][0]["v_d"] / max(float(np.max(snaps["S1"][0]["v_d"])), 1e-12),
    0.65 * snaps["S1"][0]["u_c"],
    0.45 * sat_coverage * float(SATELLITE_SELECTED),
])

prot_plate_max = max(float(np.max(p_P_S1)), 1e-12)

ax_deploy_big = fig_plate.add_subplot(gs_plate[:2, :2])
sm_deploy_big = _plot_cell_heatmap(
    ax_deploy_big,
    deployment_field,
    LinearSegmentedColormap.from_list("deploy_mix", ["#f7f7f7", "#d7ccc8",
```

```
    "#8d6e63", "#3e2723"]),
     0, 1
)
_add_park_boundary(ax_deploy_big)
if SATELLITE_SELECTED:
     _add_cell_overlay(ax_deploy_big, sat_coverage > 0, facecolor="#80deea",
    edgecolor="none", alpha=0.18, linewidth=0.0, zorder=3)
_add_waterholes(ax_deploy_big)
_add_intensity_overlay(ax_deploy_big, snaps["S1"][0]["home_r"], ranger_home_cmap,
    edgecolor="#ad1457", alpha=0.55)
_add_intensity_overlay(ax_deploy_big, snaps["S1"][0]["home_d"], drone_home_cmap,
    edgecolor="#4527a0", alpha=0.48)
_mark_home_cells(ax_deploy_big, snaps["S1"][0]["home_r"], edgecolor="#4a001f",
    marker="s", size=44)
_mark_home_cells(ax_deploy_big, snaps["S1"][0]["home_d"], edgecolor="#1a237e",
    marker="o", size=34)
_add_cell_overlay(ax_deploy_big, u_c_opt > 0.5, facecolor="#26c6da",
    edgecolor="#00838f", alpha=0.78, linewidth=0.5, zorder=8)
_style(ax_deploy_big, "Combined deployment footprint")
plt.colorbar(sm_deploy_big, ax=ax_deploy_big, label="Normalized deployment
    intensity", shrink=0.78, pad=0.02)

ax_protect_big = fig_plate.add_subplot(gs_plate[2:, :2])
sm_protect_big = _plot_cell_heatmap(ax_protect_big, p_P_S1, cmap_prot, 0,
    prot_plate_max)
_add_park_boundary(ax_protect_big)
_add_waterholes(ax_protect_big)
_style(ax_protect_big, "Protection intensity")
plt.colorbar(sm_protect_big, ax=ax_protect_big, label="Protection probability",
    shrink=0.78, pad=0.02)

small_panels = [
     (fig_plate.add_subplot(gs_plate[0, 2]), snaps["S1"][0]["home_r"],
    ranger_home_cmap, "Ranger home network",
      "Ranger home intensity", "#4a001f", "s"),
     (fig_plate.add_subplot(gs_plate[0, 3]), snaps["S1"][0]["home_d"],
    drone_home_cmap, "Drone home network",
      "Drone home intensity", "#1a237e", "o"),
     (fig_plate.add_subplot(gs_plate[1, 2]), u_c_opt,
      LinearSegmentedColormap.from_list("cam_cells", ["#f1f8e9", "#80deea",
    "#00838f"]),
      "Camera placement", "Camera placement", "#006064", "P"),
     (fig_plate.add_subplot(gs_plate[1, 3]), sat_coverage,
      LinearSegmentedColormap.from_list("sat_cells", ["#f7fcfd", "#80deea",
    "#00838f"]),
      "Satellite footprint", "Satellite coverage", "#006064", "X"),
]

```

```
for ax_r, vals, cmap_r, title_r, cbar_r, edge_r, marker_r in small_panels:
    sm_r = _plot_cell_heatmap(ax_r, vals, cmap_r, vmin=0,
    vmax=max(float(np.max(vals)), 1.0))
    _add_park_boundary(ax_r)
    _add_waterholes(ax_r)
    _mark_home_cells(ax_r, vals, edgecolor=edge_r, marker=marker_r, size=28)
    _style(ax_r, title_r)
    plt.colorbar(sm_r, ax=ax_r, label=cbar_r, shrink=0.72, pad=0.02)

ax_blank_1 = fig_plate.add_subplot(gs_plate[2, 2])
ax_blank_1.axis("off")
ax_blank_2 = fig_plate.add_subplot(gs_plate[2, 3])
ax_blank_2.axis("off")
ax_blank_3 = fig_plate.add_subplot(gs_plate[3, 2])
ax_blank_3.axis("off")
ax_blank_4 = fig_plate.add_subplot(gs_plate[3, 3])
ax_blank_4.axis("off")

handles_plate = [
    mpatches.Patch(facecolor="#ec407a", edgecolor="#ad1457", alpha=0.55,
    label="Ranger home cells"),
    mpatches.Patch(facecolor="#7e57c2", edgecolor="#4527a0", alpha=0.48,
    label="Drone home cells"),
    mpatches.Patch(facecolor="#26c6da", edgecolor="#00838f", alpha=0.78,
    label="Camera cells"),
    mpatches.Patch(facecolor="#80deea", edgecolor="none", alpha=0.25,
    label="Satellite footprint"),
    Line2D([0],[0], marker="o", color="w", markerfacecolor="#1565c0", markersize=8,
    label="Managed waterhole"),
]
fig_plate.legend(handles=handles_plate, loc="lower center", bbox_to_anchor=(0.5,
    -0.01),
                 ncol=5, fontsize=8, framealpha=0.94)
plt.savefig("map_resource_plate.png", dpi=160, bbox_inches="tight")
plt.close()
print("  Saved map_resource_plate.png")



# MAP 4: CUMULATIVE ANNUAL COVERAGE
print("Generating cumulative coverage map...")
cum_s0 = np.mean([snaps["S0"][t]["p_P"] for t in range(T)], axis=0)
cum_S1 = np.mean([snaps["S1"][t]["p_P"] for t in range(T)], axis=0)
cum_S2 = np.mean([snaps["S2"][t]["p_P"] for t in range(T)], axis=0)
gain   = cum_S2 - cum_S1
cum_comp_max = max(float(cum_s0.max()), float(cum_S1.max()), float(cum_S2.max()),
    1e-12)
vmax_g = max(abs(gain).max(), 0.01)
```

```
2403
2404 fig4, axes4 = plt.subplots(1, 4, figsize=(28, 7))
2405 fig4.suptitle("Average Annual Protection and Improvement from Optimized Deployment",
2406              fontsize=13, fontweight="semibold")
2407
2408 for ax, vals, cmap, vmin, vmax, title in [
2409     (axes4[0], cum_s0 / cum_comp_max, cmap_prot, 0, 1,   "No-intervention average
     annual protection"),
2410     (axes4[1], cum_S1 / cum_comp_max, cmap_prot, 0, 1,   "Static optimized average
     annual protection"),
2411     (axes4[2], cum_S2 / cum_comp_max, cmap_prot, 0, 1,   "Biweekly redeployed
     average annual protection"),
2412     (axes4[3], gain,   cmap_diff, -vmax_g, vmax_g, "Improvement from biweekly
     redeployment"),
2413 ]:
2414     sc2 = _plot_cell_heatmap(ax, vals, cmap, vmin, vmax)
2415     _add_park_boundary(ax)
2416     _add_waterholes(ax)
2417     plt.colorbar(sc2, ax=ax, shrink=0.60, pad=0.02)
2418     _style(ax, title)
2419
2420 axes4[3].text(0.02, 0.97,
2421     (f"Cells improved: {(gain>0).sum()}/{n}\n"
2422      f"Mean gain: +{gain.mean():.3f}\n"
2423      f"Max gain:  +{gain.max():.3f}"),
2424     transform=axes4[3].transAxes, fontsize=8, va="top",
2425     bbox=dict(facecolor="#f5f5f5", edgecolor="#aaa",
2426               boxstyle="round,pad=0.4"))
2427 plt.tight_layout()
2428 plt.savefig("map_cumulative_coverage.png", dpi=150, bbox_inches="tight")
2429 plt.close()
2430 print("  Saved map_cumulative_coverage.png")
2431
2432
2433 # MAP 5: COMBINED
2434
2435 print("Generating combined overview map...")
2436
2437 fig5, axes5 = plt.subplots(1, 3, figsize=(28, 8))
2438 fig5.suptitle("Optimized Deployment Overview",
2439              fontsize=13, fontweight="semibold")
2440
2441 ax = axes5[0]
2442 sc2 = _plot_cell_heatmap(ax, deployment_field,
2443     LinearSegmentedColormap.from_list("deploy_mix_overview", ["#f7f7f7", "#d7ccc8",
     "#8d6e63", "#3e2723"]),
2444     0, 1)
2445 _add_park_boundary(ax)
```

```
if SATELLITE_SELECTED:
    _add_cell_overlay(ax, sat_coverage > 0, facecolor="#80deea", edgecolor="none", alpha=0.12, linewidth=0.0, zorder=3)
_add_waterholes(ax)
_add_intensity_overlay(ax, snaps["S1"][0]["home_r"], ranger_home_cmap, edgecolor="#ad1457", alpha=0.55)
_add_intensity_overlay(ax, snaps["S1"][0]["home_d"], drone_home_cmap, edgecolor="#4527a0", alpha=0.48)
_mark_home_cells(ax, snaps["S1"][0]["home_r"], edgecolor="#4a001f", marker="s", size=44)
_mark_home_cells(ax, snaps["S1"][0]["home_d"], edgecolor="#1a237e", marker="o", size=34)
_add_cell_overlay(ax, u_c_opt > 0.5, facecolor="#26c6da", edgecolor="#00838f", alpha=0.78, linewidth=0.5, zorder=8)
plt.colorbar(sc2, ax=ax, label="Normalized deployment intensity", shrink=0.6)
handles_all = [
    mpatches.Patch(facecolor="#ec407a", edgecolor="#ad1457", alpha=0.55,
                   label=f"Ranger home cells  (N_r active = {N_r_active})"),
    mpatches.Patch(facecolor="#7e57c2", edgecolor="#4527a0", alpha=0.48,
                   label=f"Drone home cells   (N_d active = {N_d_active})"),
    Line2D([0],[0], marker="s", color="#4a001f", markerfacecolor="none",
           markersize=7, linewidth=0, label="Ranger home marker"),
    Line2D([0],[0], marker="o", color="#1a237e", markerfacecolor="none",
           markersize=7, linewidth=0, label="Drone home marker"),
    mpatches.Patch(facecolor="#26c6da", edgecolor="#00838f", alpha=0.78,
                   label=f"Camera cells       (N_c = {N_c})"),
    mpatches.Patch(facecolor="#80deea", alpha=0.25,
                   label=f"Satellite coverage ({sat_coverage.sum():.0f} cells)"),
    Line2D([0],[0], marker="o", color="w", markerfacecolor="#1565c0",
           markersize=8,  label="Managed waterhole"),
]
ax.text(0.02, 0.97,
    (f"Covered cells: {coverage_cells}/{n} ({coverage_frac*100:.1f}%)\n"
     f"Weighted coverage: {coverage_weight_frac*100:.1f}%"),
    transform=ax.transAxes, fontsize=8, va="top",
    bbox=dict(facecolor="#f5f5f5", edgecolor="#aaa", boxstyle="round,pad=0.4"))
_style(ax, "Deployment footprint")

ax = axes5[1]
sc2 = _plot_cell_heatmap(ax, p_P_S1, cmap_prot, 0, 1)
_add_park_boundary(ax)
_add_waterholes(ax)
plt.colorbar(sc2, ax=ax,
    label="Protection probability",
    shrink=0.6)
_style(ax, "Protection intensity")

ax = axes5[2]
```

```
sc2 = _plot_cell_heatmap(ax, loss_t0_norm,
    LinearSegmentedColormap.from_list("loss_now",
        ["#fff3e0", "#ffb74d", "#ef6c00", "#b71c1c"]), 0, 1)
_add_park_boundary(ax)
_add_waterholes(ax)
plt.colorbar(sc2, ax=ax, label="Residual loss at the start of the planning horizon",
    shrink=0.6)
ax.text(0.02, 0.97,
    (f"S0 t=0 loss: {L0_t0:.4f}\n"
     f"S1 t=0 loss: {L3_t0:.4f}\n"
     f"Improvement: {(L0_t0-L3_t0)/max(L0_t0,1e-12)*100:.1f}%"),
    transform=ax.transAxes, fontsize=8, va="top",
    bbox=dict(facecolor="#f5f5f5", edgecolor="#aaa", boxstyle="round,pad=0.4"))
_style(ax, "Residual loss at the start of the planning horizon")

fig5.legend(handles=handles_all, loc="lower center", bbox_to_anchor=(0.5, -0.01),
            ncol=5, fontsize=7.5, framealpha=0.94)
plt.subplots_adjust(right=0.90, bottom=0.16, wspace=0.18)
plt.savefig("map_combined_overview.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_combined_overview.png")


# MAP 6: THREAT + IMPORTANCE + LOSS HEATMAPS
print("Generating threat, importance, and loss heatmaps...")
loss_s0_norm = loss_s0_t0 / loss_t0_scale
loss_S1_norm = loss_S1_t0 / loss_t0_scale

fig7, axes7 = plt.subplots(2, 3, figsize=(24, 12))
fig7.suptitle("Contextual Spatial Fields",
              fontsize=13, fontweight="semibold")

heat_panels = [
    (axes7[0, 0], tau_P0, cmap_threat, 0, 1, "Poaching threat at the start of the
    planning horizon"),
    (axes7[0, 1], tau_F0, cmap_threat, 0, 1, "Fire threat at the start of the
    planning horizon"),
    (axes7[0, 2], tau_H0, cmap_threat, 0, 1, "Human pressure at the start of the
    planning horizon"),
    (axes7[1, 0], w_i,
     LinearSegmentedColormap.from_list("imp_full", ["#f9fbe7", "#c5e1a5", "#689f38",
    "#1b5e20"]),
     0, 1, "Conservation importance"),
    (axes7[1, 1], loss_s0_norm,
     LinearSegmentedColormap.from_list("loss_s0", ["#fff8e1", "#ffcc80", "#f57c00",
    "#bf360c"]),
     0, 1, "Residual loss under no intervention"),
    (axes7[1, 2], loss_S1_norm,
```

```
     LinearSegmentedColormap.from_list("loss_S1", ["#fff8e1", "#ffcc80", "#f57c00",
    "#bf360c"]),
     0, 1, "Residual loss under the optimized deployment"),
]

for ax, vals, cmap, vmin, vmax, title in heat_panels:
    sm = _plot_cell_heatmap(ax, vals, cmap, vmin, vmax)
    _add_park_boundary(ax)
    _add_waterholes(ax)
    plt.colorbar(sm, ax=ax, shrink=0.65)
    _style(ax, title)

plt.tight_layout()
plt.savefig("map_threat_importance_loss.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved map_threat_importance_loss.png")

print("Generating standalone context maps...")
terrain_cmap = LinearSegmentedColormap.from_list("terrain_pan", ["#f7f4ea",
    "#d7ccc8", "#a1887f", "#5d4037"])
species_cmap = LinearSegmentedColormap.from_list("species_score", ["#f1f8e9",
    "#aed581", "#558b2f", "#1b5e20"])
distance_cmap = LinearSegmentedColormap.from_list("distance_map", ["#0d47a1",
    "#42a5f5", "#e3f2fd", "#fffde7"])
fire_potential_cmap = LinearSegmentedColormap.from_list("fire_potential",
    ["#fff8e1", "#ffb74d", "#f57c00", "#bf360c"])

_save_context_map(
    "context_poaching_threat.png",
    tau_P0, cmap_threat,
    "Context: Poaching Threat",
    "Poaching threat index tau_P",
    vmin=0, vmax=1,
    note="Boundary-adjusted baseline threat."
)
_save_context_map(
    "context_fire_threat.png",
    tau_F0, cmap_threat,
    "Context: Fire Threat",
    "Fire threat index tau_F",
    vmin=0, vmax=1,
)
_save_context_map(
    "context_human_threat.png",
    tau_H0, cmap_threat,
    "Context: Human Pressure",
    "Human pressure index tau_H",
    vmin=0, vmax=1,
```

```
)
_save_context_map(
    "context_importance.png",
    w_i,
    LinearSegmentedColormap.from_list("imp_context", ["#f9fbe7", "#c5e1a5",
    "#689f38", "#1b5e20"]),
    "Context: Conservation Importance",
    "Importance score w_i",
    vmin=0, vmax=1,
)
_save_context_map(
    "context_species_score.png",
    S_i, species_cmap,
    "Context: Species Score",
    "Species score S_i",
    vmin=float(np.min(S_i)), vmax=float(np.max(S_i)),
)
_save_context_map(
    "context_terrain_pan.png",
    frac_pan, terrain_cmap,
    "Context: Pan / Open Terrain Fraction",
    "Pan fraction",
    vmin=0, vmax=1,
    note="Higher values indicate salt-pan / open-terrain cells."
)
_save_context_map(
    "context_fire_potential.png",
    Psi_i, fire_potential_cmap,
    "Context: Fire Damage Potential",
    "Fire potential Psi_i",
    vmin=float(np.min(Psi_i)), vmax=float(np.max(Psi_i)),
)
_save_context_map(
    "context_boundary_distance.png",
    d_boundary_km, distance_cmap,
    "Context: Distance To Boundary",
    "Distance to boundary (km)",
    vmin=float(np.min(d_boundary_km)), vmax=float(np.max(d_boundary_km)),
)
_save_context_map(
    "context_boundary_risk.png",
    boundary_risk, cmap_threat,
    "Context: Boundary Infiltration Risk",
    "Boundary risk weight",
    vmin=0, vmax=1,
    note="Higher near the boundary to reflect slip-in risk."
)
_save_context_map(
```

```
    "context_water_distance.png",
    d_water_km, distance_cmap,
    "Context: Distance To Water",
    "Distance to water (km)",
    vmin=float(np.min(d_water_km)), vmax=float(np.max(d_water_km)),
)


# TIME-SERIES COMPARISON PLOT
print("Generating time-series comparison...")
COLS   = {"S0": "#9e9e9e", "S1": "#d32f2f", "S2": "#1565c0"}
LABELS = {"S0": "S0: No intervention", "S1": "S1: Static optimized deployment",
    "S2": "S2: Biweekly redeployment"}
weeks  = np.arange(T)

fig6 = plt.figure(figsize=(20, 10))
gs6  = gridspec.GridSpec(2, 3, figure=fig6, hspace=0.40, wspace=0.30,
                         left=0.06, right=0.97, top=0.92, bottom=0.07)
fig6.suptitle("Annual Protection Dynamics: No Intervention, Static Deployment, and
    Biweekly Redeployment",
              fontsize=14, fontweight="semibold")

def _sax(ax):
    ax.tick_params(labelsize=8)
    ax.spines[["top","right"]].set_visible(False)
    ax.grid(alpha=0.25, color="#dddddd")
    return ax

ax = _sax(fig6.add_subplot(gs6[0,0]))
for sc, h in results.items():
    ax.plot(weeks, h["loss"], color=COLS[sc], label=LABELS[sc], lw=2)
ax.axvspan(26, 39, alpha=0.07, color="#e65100", label="Dry season")
ax.set_title("Total conservation loss"); ax.set_xlabel("Week")
ax.set_ylabel("Loss index (weighted mean residual threat)")
ax.legend(fontsize=8)

ax = _sax(fig6.add_subplot(gs6[0,1]))
for sc, h in results.items():
    ax.plot(weeks, h["P_park"], color=COLS[sc], lw=2, label=LABELS[sc])
ax.axvspan(26, 39, alpha=0.07, color="#e65100")
ax.set_title("Park-wide mean protection"); ax.set_xlabel("Week")
ax.set_ylabel("Weighted mean protection probability")
ax.legend(fontsize=8)

ax = _sax(fig6.add_subplot(gs6[0,2]))
ax.plot(weeks, results["S1"]["rangers_active"], color="#d32f2f", lw=2, label="S1
    active rangers")
ax.plot(weeks, results["S1"]["drones_active"], color="#7b1fa2", lw=2,
```

```
    linestyle="--", label="S1 active drones")
ax.plot(weeks, results["S2"]["rangers_active"], color="#1565c0", lw=2, label="S2 active rangers")
ax.plot(weeks, results["S2"]["drones_active"], color="#283593", lw=2, linestyle="--", label="S2 active drones")
ax.plot(weeks, results["S1"]["cameras"], color="#006064", lw=1.5, linestyle=":", label=f"Fixed cameras ({int(np.max(results['S1']['cameras']))})")
ax.set_ylim(0, None)
ax.set_title("Resource mix over time")
ax.set_xlabel("Week"); ax.set_ylabel("Active units / installed cameras"); ax.legend(fontsize=8)
ax.text(0.02, 0.97,
    (f"S1 mean budget: N${np.mean(results['S1']['budget']):,.0f}/wk\n"
     f"S2 mean budget: N${np.mean(results['S2']['budget']):,.0f}/wk\n"
     f"Satellite footprint: {sat_coverage.sum():.0f} cells"),
    transform=ax.transAxes, fontsize=7.5, va="top",
    bbox=dict(facecolor="#f5f5f5", edgecolor="#aaa", boxstyle="round"))

ax = _sax(fig6.add_subplot(gs6[1,0]))
ax.plot(weeks, results["S1"]["loss_P"], color="#b71c1c", lw=2, label="S1 poaching loss")
ax.plot(weeks, results["S1"]["loss_F"], color="#e65100", lw=2, label="S1 fire loss")
ax.plot(weeks, results["S1"]["loss_H"], color="#f57f17", lw=2, label="S1 human loss")
ax.plot(weeks, results["S2"]["loss_P"], color="#880e4f", lw=1.6, linestyle="--", label="S2 poaching loss")
ax.plot(weeks, results["S2"]["loss_F"], color="#ef6c00", lw=1.6, linestyle="--", label="S2 fire loss")
ax.plot(weeks, results["S2"]["loss_H"], color="#ff8f00", lw=1.6, linestyle="--", label="S2 human loss")
ax.set_title("Loss by threat type under active management"); ax.set_xlabel("Week");
    ax.set_ylabel("Loss contribution"); ax.legend(fontsize=7.5, ncol=2)

ax = _sax(fig6.add_subplot(gs6[1,1]))
for sc, h in results.items():
    N_norm = np.array(h["N_total"]) / h["N_total"][0]
    ax.plot(weeks, N_norm, color=COLS[sc], lw=2, label=LABELS[sc])
ax.axhline(1.0, color="black", lw=0.8, ls="--", alpha=0.5)
ax.set_title("Relative species abundance"); ax.set_xlabel("Week")
ax.set_ylabel("Relative species abundance (N_t / N_0)")
ax.legend(fontsize=8)

ax = _sax(fig6.add_subplot(gs6[1,2]))
ax.plot(weeks, results["S1"]["fence_km"], color="#1565c0", lw=2, label="Rebuilt (km)")
ax.axhline(FENCE_TOTAL_KM, color="#555", lw=1, ls="--", label="Total perimeter")
ax.fill_between(weeks, results["S1"]["fence_km"], FENCE_TOTAL_KM,
                alpha=0.15, color="#ef5350", label="Unreconstructed")
ax.set_title("Fence reconstruction"); ax.set_xlabel("Week")
```

```
ax.set_ylabel("km"); ax.legend(fontsize=8)

plt.savefig("dynamic_optimisation.png", dpi=150, bbox_inches="tight")
plt.close()
print("  Saved dynamic_optimisation.png")

print("Generating Monte Carlo uncertainty plot...")
fig_mc, axes_mc = plt.subplots(2, 2, figsize=(16, 11))

for sc, color in [("S0", "#9e9e9e"), ("S1", "#d32f2f"), ("S2", "#1565c0")]:
    if sc not in mc_df["scenario"].unique():
        continue
    sub = mc_df[mc_df["scenario"] == sc]
    axes_mc[0, 0].hist(sub["discounted_loss"], bins=14, alpha=0.65, color=color,
    label=sc)
axes_mc[0, 0].set_title("Discounted loss across Monte Carlo runs")
axes_mc[0, 0].set_xlabel("Discounted loss")
axes_mc[0, 0].set_ylabel("Frequency")
axes_mc[0, 0].legend(fontsize=8)

axes_mc[0, 1].boxplot(
    [mc_df.loc[mc_df["scenario"] == sc, "discounted_loss"] for sc in ["S0", "S1",
    "S2"] if sc in mc_df["scenario"].unique()],
    labels=[sc for sc in ["S0", "S1", "S2"] if sc in mc_df["scenario"].unique()],
)
axes_mc[0, 1].set_title("Discounted loss distribution by scenario")
axes_mc[0, 1].set_ylabel("Discounted loss")

axes_mc[1, 0].hist(mc_paired["improvement_pct"], bins=14, color="#1565c0",
    alpha=0.82)
axes_mc[1, 0].axvline(0.0, color="#555555", linewidth=1.0)
axes_mc[1, 0].axvline(1.0, color="#2e7d32", linewidth=1.0, linestyle="--")
axes_mc[1, 0].set_title("Percentage improvement of S1 over S0")
axes_mc[1, 0].set_xlabel("Improvement in discounted loss (%)")
axes_mc[1, 0].set_ylabel("Frequency")

xmin_mc = float(min(mc_paired["S0"].min(), mc_paired["S1"].min()))
xmax_mc = float(max(mc_paired["S0"].max(), mc_paired["S1"].max()))
axes_mc[1, 1].scatter(mc_paired["S0"], mc_paired["S1"], color="#5e35b1", alpha=0.75)
axes_mc[1, 1].plot([xmin_mc, xmax_mc], [xmin_mc, xmax_mc], color="#555555",
    linestyle="--")
if "S2" in mc_paired.columns:
    axes_mc[1, 1].scatter(mc_paired["S0"], mc_paired["S2"], color="#1565c0",
    alpha=0.60)
axes_mc[1, 1].set_title("Seed-matched discounted loss outcomes")
axes_mc[1, 1].set_xlabel("S0 discounted loss")
axes_mc[1, 1].set_ylabel("Managed-scenario discounted loss")

```

```
print("Generating biweekly deployment animation...")
decision_weeks = list(range(0, T, REPLAN_INTERVAL_WEEKS))
decision_snaps = [snaps["S2"][wk] for wk in decision_weeks]

fig_anim, ax_anim = plt.subplots(figsize=(13, 7))

def _dynamic_deployment_field(snap):
    return np.maximum.reduce([
        snap["v_r"] / max(float(np.max(snap["v_r"])), 1e-12),
        snap["v_d"] / max(float(np.max(snap["v_d"])), 1e-12),
        0.65 * snap["u_c"],
        0.45 * sat_coverage * float(snap["sat_selected"]),
    ])

def _draw_anim_frame(frame_idx):
    snap = decision_snaps[frame_idx]
    ax_anim.clear()
    sm = _plot_cell_heatmap(
        ax_anim,
        _dynamic_deployment_field(snap),
        LinearSegmentedColormap.from_list("deploy_anim", ["#f7f7f7", "#d7ccc8", "#8d6e63", "#3e2723"]),
        0, 1
    )
    _add_park_boundary(ax_anim)
    if snap["sat_selected"]:
        _add_cell_overlay(ax_anim, sat_coverage > 0, facecolor="#80deea", edgecolor="none", alpha=0.16, linewidth=0.0, zorder=3)
    _add_waterholes(ax_anim)
    _add_intensity_overlay(ax_anim, snap["home_r"], ranger_home_cmap, edgecolor="#ad1457", alpha=0.55)
    _add_intensity_overlay(ax_anim, snap["home_d"], drone_home_cmap, edgecolor="#4527a0", alpha=0.48)
    _mark_home_cells(ax_anim, snap["home_r"], edgecolor="#4a001f", marker="s", size=42)
    _mark_home_cells(ax_anim, snap["home_d"], edgecolor="#1a237e", marker="o", size=34)
    _add_cell_overlay(ax_anim, snap["u_c"] > 0.5, facecolor="#26c6da", edgecolor="#00838f", alpha=0.78, linewidth=0.5, zorder=8)
    _style(ax_anim, f"Biweekly redeployment footprint  |  Week {snap['t'] + 1}")
    ax_anim.text(
        0.02, 0.97,
        f"Active rangers: {int(np.sum(snap['home_r']))}\nActive drones: {int(np.sum(snap['home_d']))}\nWeekly budget: N${snap['budget']:,.0f}",
        transform=ax_anim.transAxes, fontsize=8, va="top",
        bbox=dict(facecolor="#ffffff", edgecolor="#bbbbbb", boxstyle="round,pad=0.3")
    )
    return sm
```

```
2785
2786 _draw_anim_frame(0)
2787 anim_obj = animation.FuncAnimation(fig_anim, _draw_anim_frame,
        frames=len(decision_snaps), interval=1200, blit=False)
2788 try:
2789     writer = animation.FFMpegWriter(fps=1, bitrate=1800)
2790     anim_obj.save("deployment_animation.mp4", writer=writer, dpi=160)
2791     print("  Saved deployment_animation.mp4")
2792 except Exception as e:
2793     print(f"  Warning: could not save deployment_animation.mp4 ({e})")
2794 plt.close(fig_anim)
2795
2796 plt.tight_layout()
2797 plt.savefig("monte_carlo_uncertainty.png", dpi=150, bbox_inches="tight")
2798 plt.close()
2799 print("  Saved monte_carlo_uncertainty.png")
2800
2801 # FINAL
2802 print("\n" + "="*65)
2803 print("FINAL SUMMARY")
2804 print("="*65)
2805 print(f"  HIRED/BOUGHT (static for full year):")
2806 print(f"    Rangers:   {N_r_active} on duty / {N_r_employed} employed")
2807 print(f"    Drones:    {N_d_active} active / {STATIC_DRONE_OPERATORS} operators")
2808 print(f"    Cameras:   {N_c}  (permanently installed on selected cells)")
2809 print(f"    Satellite:  {int(SATELLITE_SELECTED)}   (Planet SuperDove,
        {sat_coverage.sum():.0f} candidate cells covered)")
2810 print(f"  OPTIMALITY:")
2811 print(f"    L* (continuous):  {L_star_0:.6f}")
2812 print(f"    f(x_hat) (integer):   {f_int_0:.6f}")
2813 print(f"    Gap delta:            {gap_0:.4%}  ->  {(1-gap_0)*100:.2f}%-optimal")
2814 print(f"  PERFORMANCE:")
2815 print(f"    S0 discounted loss:  {L_S0:.4f}")
2816 print(f"    S1 discounted loss:  {L_S1:.4f}")
2817 print(f"    S2 discounted loss:  {L_S2:.4f}")
2818 print(f"    S1 improvement over S0: {improv_S1:.1f}%")
2819 print(f"    S2 improvement over S0: {improv_S2:.1f}%")
2820 print(f"    S2 improvement over S1: {gain_S2_over_S1:.1f}%")
2821 print(f"    Mean protected share S0:
        {np.mean(results['S0']['protected_total'])*100:.1f}%")
2822 print(f"    Mean protected share S1:
        {np.mean(results['S1']['protected_total'])*100:.1f}%")
2823 print(f"    Mean protected share S2:
        {np.mean(results['S2']['protected_total'])*100:.1f}%")
2824 print(f"    S0 t=0 loss:         {L0_t0:.4f}")
2825 print(f"    S1 t=0 loss:         {L3_t0:.4f}")
2826 print(f"    S2 t=0 loss:         {L2_t0:.4f}")
2827 print(f"    S0 t=0 protected:    {protected_s0_t0*100:.1f}% of weighted threat")
```

```
print(f"    S1 t=0 protected:    {protected_S1_t0*100:.1f}% of weighted threat")
print(f"    S2 t=0 protected:    {protected_S2_t0*100:.1f}% of weighted threat")
print(f"    S1 t=0 protection by threat:"
      f" poach={P3p_t0:.4f}, fire={P3f_t0:.4f}, human={P3h_t0:.4f}")
print(f"    S2 t=0 protection by threat:"
      f" poach={P2p_t0:.4f}, fire={P2f_t0:.4f}, human={P2h_t0:.4f}")
print(f"  COVERAGE:")
print(f"    Ranger-covered cells:   {int(ranger_cover_mask.sum())}/{n}")
print(f"    Drone-covered cells:    {int(drone_cover_mask.sum())}/{n}")
print(f"    Camera cells:           {int(camera_cover_mask.sum())}/{n}")
print(f"    Satellite cells:        {int(sat_cover_mask.sum())}/{n}")
print(f"    Combined covered cells: {coverage_cells}/{n}
    ({coverage_frac*100:.1f}%)")
print(f"    Weighted coverage:      {coverage_weight_frac*100:.1f}% of w_i")
print(f"  STACKING PENALTY:")
print(f"    Ranger home concentration penalty: {STACK_PENALTY_RANGER:.3f}")
print(f"    Drone home concentration penalty:  {STACK_PENALTY_DRONE:.3f}")
print(f"  OUTPUTS:")
print(f"    deployment_animation.mp4")
print(f"    map_key_*.png")
print(f"    map_cumulative_coverage.png")
print(f"    context_*.png")
print(f"    dynamic_optimisation.png")
print(f"    dynamic_timeseries.csv")
print(f"    monte_carlo_uncertainty.png")
print(f"    monte_carlo_uncertainty_summary.csv")
print("="*65)
```

A.2 Sensitivity Code

Artificial Intelligence was not use to generate the following code.

```
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
import textwrap
from matplotlib.ticker import MaxNLocator

# UTILITIES
def _clone(v):
    if isinstance(v, np.ndarray):
        return v.copy()
    return copy.deepcopy(v)

def _scenario_tag(name):
    return (
```

```
        str(name)
        .lower()
        .replace(" ", "_")
        .replace("%", "pct")
        .replace("-", "minus")
        .replace("+", "plus")
        .replace("/", "_")
        .replace("(", "")
        .replace(")", "")
    )

def wrap_label(s, width=18):
    return "\n".join(textwrap.wrap(str(s), width=width))

# STATE SNAPSHOT
state_keys = [
    "BUDGET_WEEKLY",
    "WORK_DAYS_PER_WEEK",
    "ACTIVE_RANGER_COST",
    "COST_DRONE_FIXED",
    "COST_DRONE_OPERATOR",
    "COST_DRONE",
    "COST_CAMERA",
    "COST_SATELLITE_WK",
    "LAMBDA_DETER",
    "SIGMA_DRONE_POACH",
    "SIGMA_DRONE_FIRE",
    "SIGMA_CAM",
    "BOUNDARY_POACH_BOOST",
    "RANGER_RESPONSE_SHARE",
    "PATROL_RESPONSE_EFFECTIVENESS",
    "CACHE_VERSION",
    "STATIC_RANGER_HOME",
    "STATIC_DRONE_HOME",
    "SATELLITE_SELECTED",
    "STATIC_ASSIGNMENTS",
    "STATIC_DRONE_OPERATORS",
    "v_r_opt",
    "v_d_opt",
    "u_c_opt",
    "v_r_fixed",
    "v_d_fixed",
    "u_c_fixed",
    "N_r_employed",
    "N_r_active",
    "N_d_employed",
    "N_d_active",
    "N_c",
```

```
    "L_star_0",
    "f_int_0",
    "gap_0",
]

base_state = {k: _clone(globals()[k]) for k in state_keys if k in globals()}

def restore_base_state():
    for k, v in base_state.items():
        globals()[k] = _clone(v)

def apply_overrides(overrides):
    for k, v in overrides.items():
        globals()[k] = v
    globals()["ACTIVE_RANGER_COST"] = COST_RANGER * 7.0 / WORK_DAYS_PER_WEEK
    globals()["COST_DRONE"] = COST_DRONE_FIXED + COST_DRONE_OPERATOR /
    DRONES_PER_OPERATOR

def sync_solution_tuple(sol):
    v_r_opt_, v_d_opt_, u_c_opt_, N_r_emp_, N_r_act_, N_d_emp_, N_d_act_, N_c_, Ls_,
    fi_, gap_ = sol
    globals()["v_r_opt"] = np.asarray(v_r_opt_).copy()
    globals()["v_d_opt"] = np.asarray(v_d_opt_).copy()
    globals()["u_c_opt"] = np.asarray(u_c_opt_).copy()
    globals()["v_r_fixed"] = np.asarray(v_r_opt_).copy()
    globals()["v_d_fixed"] = np.asarray(v_d_opt_).copy()
    globals()["u_c_fixed"] = np.asarray(u_c_opt_).copy()
    globals()["N_r_employed"] = int(N_r_emp_)
    globals()["N_r_active"] = int(N_r_act_)
    globals()["N_d_employed"] = int(N_d_emp_)
    globals()["N_d_active"] = int(N_d_act_)
    globals()["N_c"] = int(N_c_)
    globals()["L_star_0"] = float(Ls_)
    globals()["f_int_0"] = float(fi_)
    globals()["gap_0"] = float(gap_)

def capture_static_plan_state():
    return {
        "STATIC_RANGER_HOME": _clone(STATIC_RANGER_HOME),
        "STATIC_DRONE_HOME": _clone(STATIC_DRONE_HOME),
        "SATELLITE_SELECTED": bool(SATELLITE_SELECTED),
        "STATIC_ASSIGNMENTS": _clone(STATIC_ASSIGNMENTS),
        "STATIC_DRONE_OPERATORS": int(STATIC_DRONE_OPERATORS),
    }

def apply_static_plan_state(plan_state):
    globals()["STATIC_RANGER_HOME"] = _clone(plan_state["STATIC_RANGER_HOME"])
    globals()["STATIC_DRONE_HOME"] = _clone(plan_state["STATIC_DRONE_HOME"])
```

```
    globals()["SATELLITE_SELECTED"] = bool(plan_state["SATELLITE_SELECTED"])
    globals()["STATIC_ASSIGNMENTS"] = _clone(plan_state["STATIC_ASSIGNMENTS"])
    globals()["STATIC_DRONE_OPERATORS"] = int(plan_state["STATIC_DRONE_OPERATORS"])

# OPTIMISATION UNDER ASSUMPTIONS
def solve_plan_under_assumptions(name, overrides):
    restore_base_state()
    apply_overrides(overrides)

    globals()["CACHE_VERSION"] = f"{base_state.get('CACHE_VERSION',
    'base')}_sens_{_scenario_tag(name)}"
    globals()["STATIC_RANGER_HOME"] = None
    globals()["STATIC_DRONE_HOME"] = None
    globals()["STATIC_ASSIGNMENTS"] = {"rangers": [], "drones": []}
    globals()["STATIC_DRONE_OPERATORS"] = 0

    sol = solve_minlp()
    sync_solution_tuple(sol)

    return {
        "scenario": name,
        "overrides": _clone(overrides),
        "solution": tuple(_clone(x) for x in sol),
        "static_plan_state": capture_static_plan_state(),
        "objective_cont": float(L_star_0),
        "objective_rounded": float(f_int_0),
        "gap": float(gap_0),
        "rangers_active": int(N_r_active),
        "rangers_employed": int(N_r_employed),
        "drones_active": int(N_d_active),
        "cameras": int(N_c),
        "satellite": int(bool(SATELLITE_SELECTED)),
    }

# FULL SIM EVAL
def evaluate_plan_full_sim(plan_row, seed):
    restore_base_state()
    apply_overrides(plan_row["overrides"])
    sync_solution_tuple(plan_row["solution"])
    apply_static_plan_state(plan_row["static_plan_state"])

    globals()["RNG"] = np.random.default_rng(seed)
    hist, _ = run_scenario("S1")

    home_r_eval = STATIC_RANGER_HOME.copy() if STATIC_RANGER_HOME is not None else
    np.zeros(n, dtype=float)
    p_p, p_f, p_h = compute_protection(
        v_r_opt, v_d_opt, u_c_opt, bool(SATELLITE_SELECTED), home_r=home_r_eval
```

```
    )

    discounted_loss = float(sum((GAMMA ** t) * hist["loss"][t] for t in range(T)))
    protection_field = (p_p + p_f + p_h) / 3.0
    residual_loss_t0 = w_i * (
        tau_P0 * (1.0 - p_p) +
        tau_F0 * (1.0 - p_f) +
        tau_H0 * (1.0 - p_h)
    )

    return {
        "scenario": plan_row["scenario"],
        "seed": int(seed),
        "discounted_loss": discounted_loss,
        "mean_loss": float(np.mean(hist["loss"])),
        "mean_protected_share": float(np.mean(hist["protected_total"])),
        "t0_protected_share": float(protected_fraction_total(w_i, tau_P0, tau_F0,
    tau_H0, p_p, p_f, p_h)),
        "objective_cont": float(plan_row["objective_cont"]),
        "objective_rounded": float(plan_row["objective_rounded"]),
        "gap": float(plan_row["gap"]),
        "rangers_active": int(plan_row["rangers_active"]),
        "rangers_employed": int(plan_row["rangers_employed"]),
        "drones_active": int(plan_row["drones_active"]),
        "cameras": int(plan_row["cameras"]),
        "satellite": int(plan_row["satellite"]),
        "protection_field": protection_field,
        "residual_loss_t0": residual_loss_t0,
        "history_loss": np.asarray(hist["loss"], dtype=float),
        "history_protected": np.asarray(hist["protected_total"], dtype=float),
    }

# SCENARIO DEFINITIONS
scenario_specs = [
    ("Base case", {}),
    ("LAMBDA_DETER -10%", {"LAMBDA_DETER": base_state["LAMBDA_DETER"] * 0.90}),
    ("LAMBDA_DETER +10%", {"LAMBDA_DETER": base_state["LAMBDA_DETER"] * 1.10}),
    ("SIGMA_DRONE_POACH -10%", {"SIGMA_DRONE_POACH": base_state["SIGMA_DRONE_POACH"]
    * 0.90}),
    ("SIGMA_DRONE_POACH +10%", {"SIGMA_DRONE_POACH": base_state["SIGMA_DRONE_POACH"]
    * 1.10}),
    ("BOUNDARY_POACH_BOOST -10%", {"BOUNDARY_POACH_BOOST":
    base_state["BOUNDARY_POACH_BOOST"] * 0.90}),
    ("BOUNDARY_POACH_BOOST +10%", {"BOUNDARY_POACH_BOOST":
    base_state["BOUNDARY_POACH_BOOST"] * 1.10}),
    ("SIGMA_CAM -10%", {"SIGMA_CAM": base_state["SIGMA_CAM"] * 0.90} if "SIGMA_CAM"
    in base_state else {}),
    ("SIGMA_CAM +10%", {"SIGMA_CAM": base_state["SIGMA_CAM"] * 1.10} if "SIGMA_CAM"
```

```
    in base_state else {}),
    ("SIGMA_DRONE_FIRE -10%", {"SIGMA_DRONE_FIRE": base_state["SIGMA_DRONE_FIRE"] *
    0.90} if "SIGMA_DRONE_FIRE" in base_state else {}),
    ("SIGMA_DRONE_FIRE +10%", {"SIGMA_DRONE_FIRE": base_state["SIGMA_DRONE_FIRE"] *
    1.10} if "SIGMA_DRONE_FIRE" in base_state else {}),
    ("Budget -10%", {"BUDGET_WEEKLY": base_state["BUDGET_WEEKLY"] * 0.90}),
    ("Budget +10%", {"BUDGET_WEEKLY": base_state["BUDGET_WEEKLY"] * 1.10}),
    ("Drone fixed cost -10%", {"COST_DRONE_FIXED": base_state["COST_DRONE_FIXED"] *
    0.90}),
    ("Drone fixed cost +10%", {"COST_DRONE_FIXED": base_state["COST_DRONE_FIXED"] *
    1.10}),
    ("Camera cost -10%", {"COST_CAMERA": base_state["COST_CAMERA"] * 0.90}),
    ("Camera cost +10%", {"COST_CAMERA": base_state["COST_CAMERA"] * 1.10}),
    ("Four-day patrol week", {"WORK_DAYS_PER_WEEK": 4}),
    ("Drones 50% more expensive", {"COST_DRONE_FIXED":
    base_state["COST_DRONE_FIXED"] * 1.50}),
]

# RESPONSE
if "RANGER_RESPONSE_SHARE" in base_state and "PATROL_RESPONSE_EFFECTIVENESS" in
    base_state:
    scenario_specs.extend([
        ("Response improved", {
            "RANGER_RESPONSE_SHARE": 0.60,
            "PATROL_RESPONSE_EFFECTIVENESS": 0.75
        }),
        ("Response strong but detection unchanged", {
            "RANGER_RESPONSE_SHARE": 0.65,
            "PATROL_RESPONSE_EFFECTIVENESS": 0.80
        }),
    ])

scenario_specs = [(name, overrides) for name, overrides in scenario_specs if
    overrides is not None]

# SOLVE SCENARIO
plans = [solve_plan_under_assumptions(name, overrides) for name, overrides in
    scenario_specs]
restore_base_state()

# EVALUATION FINAL
FIXED_SEED = 42
rows = [evaluate_plan_full_sim(plan, FIXED_SEED) for plan in plans]

raw_df = pd.DataFrame([{
    "scenario": r["scenario"],
    "discounted_loss": r["discounted_loss"],
    "mean_loss": r["mean_loss"],
```

```
    "mean_protected_share": r["mean_protected_share"],
    "t0_protected_share": r["t0_protected_share"],
    "objective_cont": r["objective_cont"],
    "objective_rounded": r["objective_rounded"],
    "gap": r["gap"],
    "rangers_active": r["rangers_active"],
    "rangers_employed": r["rangers_employed"],
    "drones_active": r["drones_active"],
    "cameras": r["cameras"],
    "satellite": r["satellite"],
} for r in rows])

raw_df.to_csv("sensitivity_fullsim_raw.csv", index=False)

summary_df = raw_df.copy()
base_loss = float(summary_df.loc[summary_df["scenario"] == "Base case",
    "discounted_loss"].iloc[0])
base_obj = float(summary_df.loc[summary_df["scenario"] == "Base case",
    "objective_rounded"].iloc[0])

summary_df["loss_change_vs_base_pct"] = 100.0 * (summary_df["discounted_loss"] -
    base_loss) / max(base_loss, 1e-12)
summary_df["objective_change_pct"] = 100.0 * (summary_df["objective_rounded"] -
    base_obj) / max(base_obj, 1e-12)
summary_df["protected_share_mean_pct"] = 100.0 * summary_df["mean_protected_share"]
summary_df["t0_protected_mean_pct"] = 100.0 * summary_df["t0_protected_share"]
summary_df = summary_df.sort_values("discounted_loss").reset_index(drop=True)

summary_df.to_csv("sensitivity_fullsim_summary.csv", index=False)
display(summary_df)

# PLOT
plt.style.use("seaborn-v0_8-whitegrid")

summary_plot = summary_df.copy()
summary_plot["scenario_plot"] = summary_plot["scenario"].map(lambda s: wrap_label(s,
    18))

# METRICS
fig, axes = plt.subplots(2, 2, figsize=(18, 12), constrained_layout=True)

plot_loss = summary_plot.sort_values("discounted_loss").reset_index(drop=True)

axes[0, 0].barh(plot_loss["scenario_plot"], plot_loss["discounted_loss"],
    color="#6d4c41")
axes[0, 0].set_title("Discounted loss by scenario", pad=10)
axes[0, 0].set_xlabel("Discounted loss")
axes[0, 0].set_ylabel("")
```

```
axes[0, 0].invert_yaxis()

axes[0, 1].barh(plot_loss["scenario_plot"], plot_loss["protected_share_mean_pct"], color="#2e7d32")
axes[0, 1].set_title("Mean protected share (%)", pad=10)
axes[0, 1].set_xlabel("Protected share (%)")
axes[0, 1].set_ylabel("")
axes[0, 1].invert_yaxis()

x = np.arange(len(plot_loss))
w = 0.26
axes[1, 0].bar(x - w, plot_loss["rangers_active"], width=w, label="Rangers")
axes[1, 0].bar(x, plot_loss["drones_active"], width=w, label="Drones")
axes[1, 0].bar(x + w, plot_loss["cameras"], width=w, label="Cameras")
axes[1, 0].set_xticks(x)
axes[1, 0].set_xticklabels(plot_loss["scenario_plot"], rotation=30, ha="right")
axes[1, 0].set_title("Allocation", pad=10)
axes[1, 0].set_ylabel("Units")
axes[1, 0].legend(frameon=False, ncol=3, loc="upper center", bbox_to_anchor=(0.5, 1.18))

delta_df = plot_loss[plot_loss["scenario"] != "Base case"].sort_values("loss_change_vs_base_pct")
axes[1, 1].barh(delta_df["scenario_plot"], delta_df["loss_change_vs_base_pct"], color="#c62828")
axes[1, 1].axvline(0, color="#444444", linewidth=1)
axes[1, 1].set_title("Loss change vs base (%)", pad=10)
axes[1, 1].set_xlabel("Percent change")
axes[1, 1].set_ylabel("")
axes[1, 1].invert_yaxis()

for ax in axes.ravel():
    ax.tick_params(axis="both", labelsize=10)
    ax.xaxis.set_major_locator(MaxNLocator(nbins=6))
    ax.grid(True, axis="x", alpha=0.25)

fig.suptitle("Scenario sensitivity summary", fontsize=16, y=1.02)
plt.savefig("sensitivity_fullsim_metrics.png", dpi=220, bbox_inches="tight")
plt.show()

# MAPS
preferred_key_names = [
    "Base case",
    "Four-day patrol week",
    "Drones 50% more expensive",
    "Response improved",
]
key_rows = [r for r in rows if r["scenario"] in preferred_key_names]
```

```
if len(key_rows) < 4:
    key_rows = rows[:4]
else:
    key_rows = key_rows[:4]

prot_max = max(float(np.max(r["protection_field"])) for r in key_rows)
loss_max = max(float(np.max(r["residual_loss_t0"])) for r in key_rows)

fig, axes = plt.subplots(2, 2, figsize=(16, 12), constrained_layout=True)
axes = axes.ravel()
sm_last = None

for ax, r in zip(axes, key_rows):
    vals = r["protection_field"] / max(prot_max, 1e-12)
    sm_last = _plot_cell_heatmap(ax, vals, cmap_prot, 0, 1)
    _add_park_boundary(ax)
    _add_waterholes(ax)
    _style(ax, r["scenario"])
    ax.set_title(r["scenario"], fontsize=11, pad=8)

for ax in axes[len(key_rows):]:
    ax.axis("off")

cbar = fig.colorbar(sm_last, ax=axes.tolist(), shrink=0.8, pad=0.02)
cbar.set_label("Protection (scaled)", rotation=90)
fig.suptitle("Protection maps", fontsize=16, y=1.02)
plt.savefig("sensitivity_fullsim_protection_maps.png", dpi=220, bbox_inches="tight")
plt.show()

loss_cmap = LinearSegmentedColormap.from_list("loss", ["#fff8e1", "#ffcc80",
    "#f57c00", "#bf360c"])

fig, axes = plt.subplots(2, 2, figsize=(16, 12), constrained_layout=True)
axes = axes.ravel()
sm_last = None

for ax, r in zip(axes, key_rows):
    vals = r["residual_loss_t0"] / max(loss_max, 1e-12)
    sm_last = _plot_cell_heatmap(ax, vals, loss_cmap, 0, 1)
    _add_park_boundary(ax)
    _add_waterholes(ax)
    _style(ax, r["scenario"])
    ax.set_title(r["scenario"], fontsize=11, pad=8)

for ax in axes[len(key_rows):]:
    ax.axis("off")

cbar = fig.colorbar(sm_last, ax=axes.tolist(), shrink=0.8, pad=0.02)
```

```
cbar.set_label("Residual loss (scaled)", rotation=90)
fig.suptitle("Residual loss maps", fontsize=16, y=1.02)
plt.savefig("sensitivity_fullsim_loss_maps.png", dpi=220, bbox_inches="tight")
plt.show()

# TIME SERIES
fig, axes = plt.subplots(1, 2, figsize=(16, 5.8), constrained_layout=True)

for r in key_rows:
    axes[0].plot(r["history_loss"], linewidth=2, label=r["scenario"])
axes[0].set_title("Weekly conservation loss", pad=10)
axes[0].set_xlabel("Week")
axes[0].set_ylabel("Loss index")
axes[0].legend(frameon=False, fontsize=9, loc="upper left", bbox_to_anchor=(1.02,
    1.0))

for r in key_rows:
    axes[1].plot(100.0 * r["history_protected"], linewidth=2, label=r["scenario"])
axes[1].set_title("Weekly protected share", pad=10)
axes[1].set_xlabel("Week")
axes[1].set_ylabel("Protected share (%)")
axes[1].legend(frameon=False, fontsize=9, loc="upper left", bbox_to_anchor=(1.02,
    1.0))

for ax in axes:
    ax.tick_params(axis="both", labelsize=10)
    ax.grid(True, alpha=0.25)

fig.suptitle("Scenario time series", fontsize=16, y=1.03)
plt.savefig("sensitivity_fullsim_timeseries.png", dpi=220, bbox_inches="tight")
plt.show()

print("Saved:")
print("  sensitivity_fullsim_raw.csv")
print("  sensitivity_fullsim_summary.csv")
print("  sensitivity_fullsim_metrics.png")
print("  sensitivity_fullsim_protection_maps.png")
print("  sensitivity_fullsim_loss_maps.png")
print("  sensitivity_fullsim_timeseries.png")
```